SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

COMPUTER VISION – ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM – SOLUTION REPORT

Table 1. Assignment details (fill in the blanks before submitting)

Field	Details	Field	Details
Student Name	K.MAHESH BABU	Register Number	192525122
Department / Section	Information Technology	Academic Year / Semester	2025–2026
Faculty Name	Dr.MURALI_____	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50

0. Experimental Setup, Data and Honesty Statement

This section states what was actually done, as the assignment requires ("do not report assumed or fabricated measurements as experimental results").

- **Dataset** no real archive photographs were supplied with the assignment. Six public test images that ship with scikit-image (astronaut, camera, coins, coffee, chelsea cat, rocket) were degraded with a synthetic old-photograph model (Figure 1). Every number in this report is a real measurement on these synthetic photographs; none comes from real historical prints, and results on real prints will differ.
- **Grayscale set (Tasks 1–9)** 9 photographs, 512×512, 8-bit: 3 "lightly damaged", 3 "heavily faded", 3 "unevenly exposed" (G01–G09). Degradation model: linear fade ($g' = a \cdot g + b$), optional gamma, one-sided illumination shadow, slight blur, stains (smooth dark blobs), scratches (1–2 px bright lines at random angles), dust (1–3 px bright discs), creases (3 px lines), a torn paper-white corner, Gaussian noise σ=6, 1 % salt-and-pepper noise and horizontal scanner stripes every 6 rows – matching the operating data. Ground-truth damage masks and the clean originals are known for every photograph.
- **Colour set (Task 10)** 15 colour photographs, 512×512 (5 scenes × 3 degradation variants – mirrored/cropped/re-seeded – so they are not 15 independent scenes), with strong sepia/yellow cast, fade, stains, scratches, noise and JPEG quality-35 blocking. Timing for the "4000×3000" requirement was measured on a 4000×3000 up-scaled copy.
- **Important consequence of the synthetic data** all bright damage is rendered at a constant paper-white level (235). This makes simple global thresholding look better than it would on real prints. Thresholds in Tasks 7–8 were tuned on the same 9 photographs (no hold-out set), so they are optimistic.
- **Hardware / software** the sandbox used for the experiments has 2 CPU cores and about 7 GB RAM (not the 4 cores / 16 GB of the brief), Python 3.13, OpenCV 5.0, NumPy 2.5, scikit-image 0.26, no GPU. Timings are best-of-N wall-clock on this machine and would be somewhat faster on 4 cores.
- **Pretrained models** PyTorch and pretrained weights (DnCNN, HED, SAM 2, low-light/illumination/deblurring networks) were not available, and there was no network access to download them. Questions 6, 19, 26, 39, 46, 53, 62 are therefore answered as reasoned discussion, explicitly NOT as experimental results.
- **Test-resolution caveat** quality experiments run at 512×512 for tractability. Processing time for the A4 requirement (2480×3508) was measured separately at full size (Section 9.4).
- **Assumptions** (i) Gaussian/median kernels are in pixels at 512 px width; frequency-domain cut-offs D0 are in cycles per 512 px; (ii) "contrast" = grey-level standard deviation; "faint-detail recovery" = mean Sobel gradient; (iii) precision/recall use a ±2 px matching tolerance because 1–2 px lines are never pixel-exact; (iv) the reference histogram for matching is a well-exposed clean portrait; (v) the OCR table of Task 5 is not applicable – the photographs contain no text.

- **Original retained** the pipeline never overwrites the input; inpainted pixels are limited to the detected damage mask and are labelled as reconstructed.

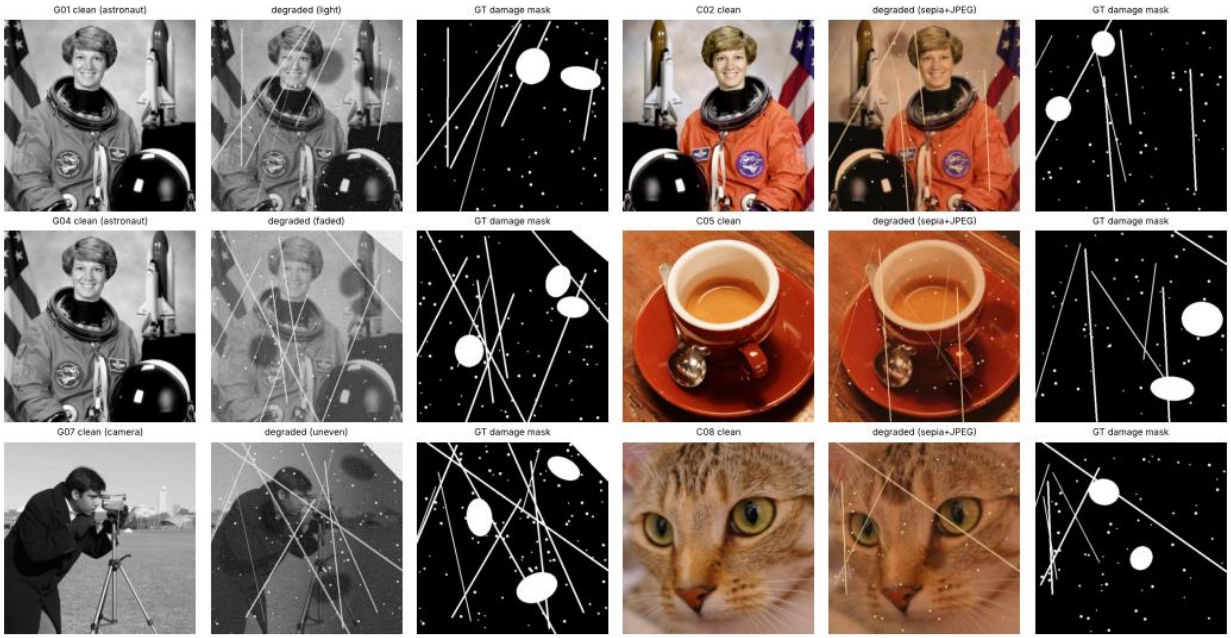


Figure 1. Examples of the synthetic old-photograph set: clean original, degraded photograph and ground-truth damage mask (grayscale G01, G04, G07; colour C02, C05, C08).

Task 1 – Grey Level Transformation Analysis

Method and implementation (Q1)

Negative: s = 255 − r. Log: s = c·log(1 + r), c = 255/log 256. Power-law: s = 255·(r/255)^γ with γ chosen per photograph so that the mean maps to mid-grey (γ ranged 0.57–1.18, values in results). Piecewise-linear stretch: breakpoints at the 2nd and 98th percentile (r1, r2) mapped to 0 and 255. All implemented with OpenCV LUT / NumPy (code in Appendix A, task12.py).

Q2. Comparison – contrast, faint details, clipping, time

Table 2. Task 1 results, mean over the 3 photographs of each set (512×512). Subject–bg = |mean(centre 256×256) – mean(border ring)|.

Set	Technique	RMS contrast	Entropy (bits)	Clipped px %	Mean gradient	Subject–bg diff	Time ms
light	Original	57.5	7.48	1.04	80.1	16.1	0.00
light	A Negative	57.5	7.48	1.04	80.1	16.1	0.04
light	A Log	33.5	6.33	1.15	48.8	8.8	1.12
light	B Gamma(auto)	56.0	7.39	1.04	77.6	16.2	0.13
light	C Piecewise 2-98%	70.0	7.32	5.26	93.9	20.0	4.44
faded	Original	39.7	6.90	1.03	64.3	9.6	0.00
faded	A Negative	39.7	6.90	1.03	64.3	9.6	0.02
faded	A Log	21.0	5.36	1.27	33.1	3.3	1.13
faded	B Gamma(auto)	41.2	6.89	1.02	66.7	10.1	0.11
faded	C Piecewise 2-98%	58.1	6.75	4.99	88.2	14.8	4.28
uneven	Original	52.1	7.34	1.06	77.4	9.7	0.00
uneven	A Negative	52.1	7.34	1.06	77.4	9.7	0.02
uneven	A Log	31.8	6.54	1.28	52.4	5.9	1.13
uneven	B Gamma(auto)	47.8	7.19	1.03	73.3	8.7	0.13
uneven	C Piecewise 2-98%	60.1	7.19	5.37	87.0	11.5	4.90

Reading of the table: piecewise-linear stretching gives the highest contrast and the highest mean gradient on every set (faded: RMS 39.7 → 58.1, subject–background difference 9.6 → 14.8) and costs only ~4 ms, but it clips

about 5 % of the pixels by construction (2 % + 2 % tails plus the pure black/white impulse noise). The auto-gamma changes the mean brightness but barely changes contrast (faded: 39.7 → 41.2) and keeps clipping at the input level (~1 %). Log compresses highlights and reduces contrast on photographs that are already bright (light set: 57.5 → 33.5). The negative only flips polarity: statistics are identical to the original and its SSIM against the clean photograph collapses (≈ 0), so it is unsuitable for restoration (useful only for inspecting very dark detail).

Caution: the RMS/entropy/gradient values are inflated by the added noise, scratches and impulse noise, so they rank the techniques but do not measure "true" detail.

Q3. Transformation curves and histograms

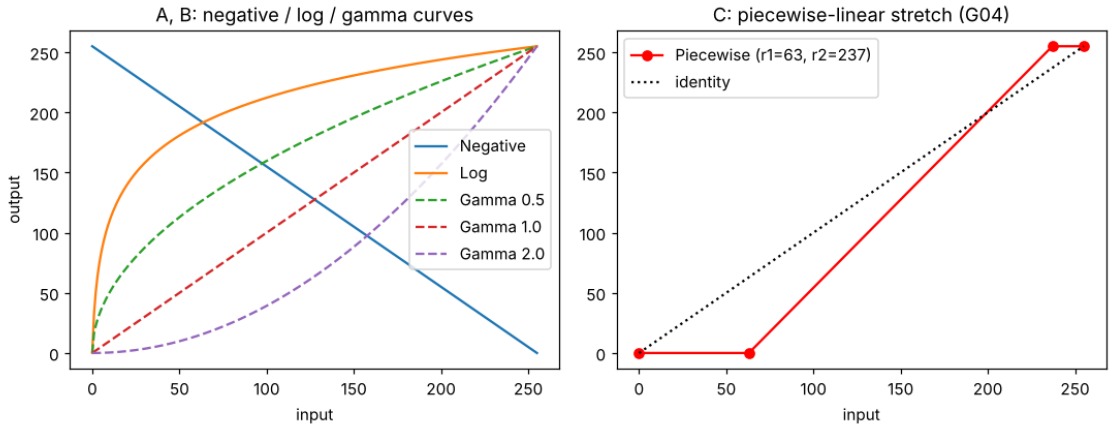


Figure 2. Transformation curves: negative, log, gamma (0.5, 1, 2) and the piecewise-linear stretch fitted to faded photograph G04 (r1, r2 shown in legend).

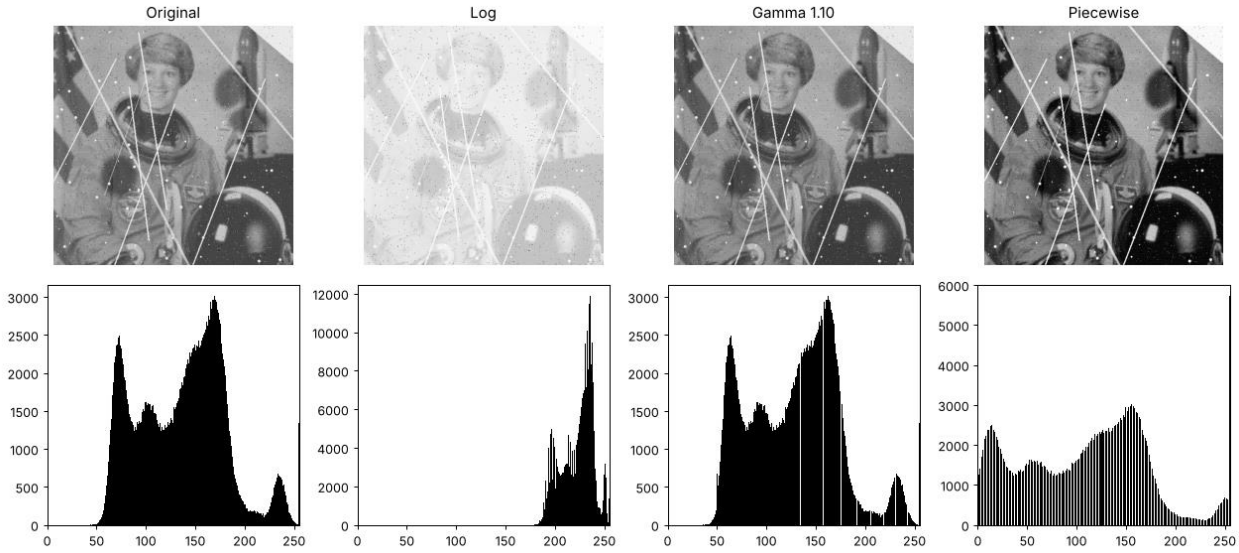


Figure 3. G04 (heavily faded): original, log, gamma and piecewise-stretch images with their histograms. The stretched histogram spans the full range.

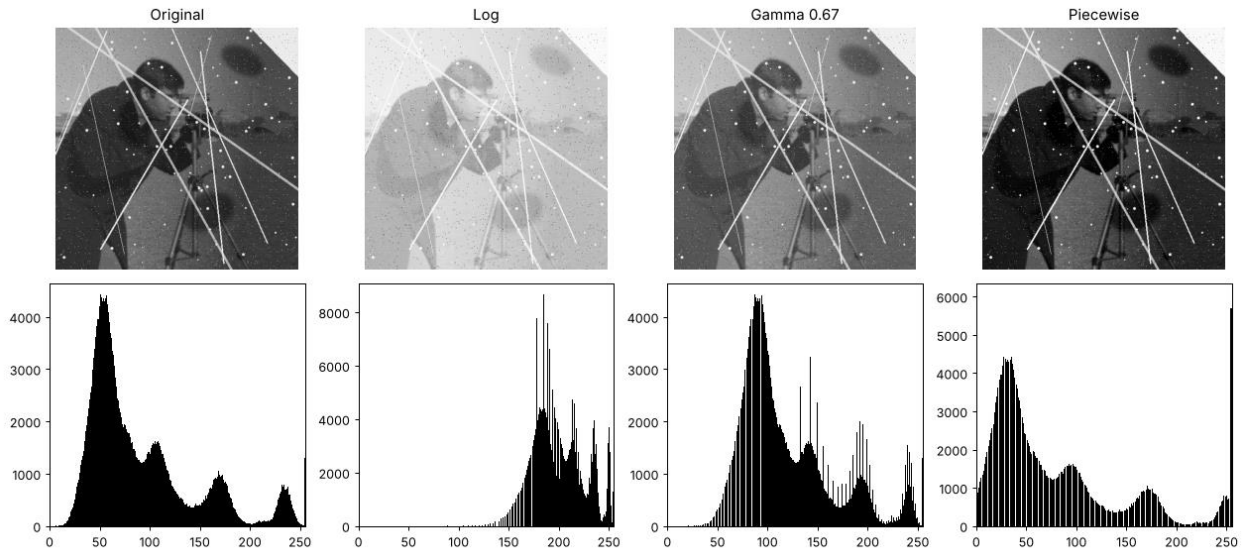


Figure 4. G07 (unevenly exposed): same comparison.

Q4. Bit-plane slicing

Table 3. Fraction of pixels whose bit equals the bit of the clean photograph (G01). 0.50 = chance (pure noise).

Bit plane	0	1	2	3	4	5	6	7 (MSB)
Agreement of degraded plane with clean plane	0.50	0.50	0.50	0.50	0.43	0.62	0.79	0.91

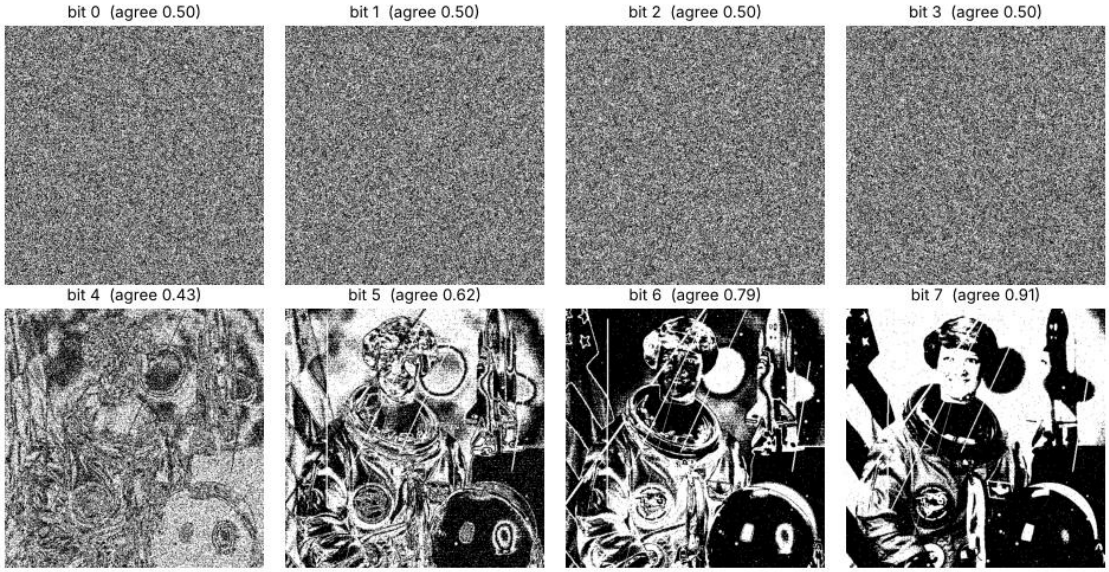


Figure 5. Bit planes 0–7 of G01. Planes 0–3 look like random noise; planes 6–7 hold the photograph structure and also the bright scratches/dust.

Planes 0–3 (agreement exactly 0.50, i.e. chance level) contain essentially only noise (Gaussian noise, stripes, impulses). Bit 4 (0.43) is partly shifted by the fade offset and is noisy; bit 5 (0.62) holds fine tonal detail with noise; bits 6 (0.79) and 7 (0.91) carry the useful image structure and also show the scratches, dust and torn corner clearly – which makes the MSB planes useful for locating bright damage. Keeping only the top 2–3 planes would discard noise but also all fine tone.

Q5. Same gamma on bright and dark photographs

Table 4. Effect of the same gamma values on an over-exposed (bright) and an under-exposed (dark) version of one portrait.

Photograph	γ	Mean	RMS contrast	Clipped %
Bright	0.5	222.8	25.5	11.5
Bright	1.0	198.1	43.8	11.0
Bright	2.0	161.0	66.1	9.9
Dark	0.5	117.7	59.8	12.2
Dark	1.0	68.9	45.0	13.9
Dark	2.0	26.2	23.5	26.0

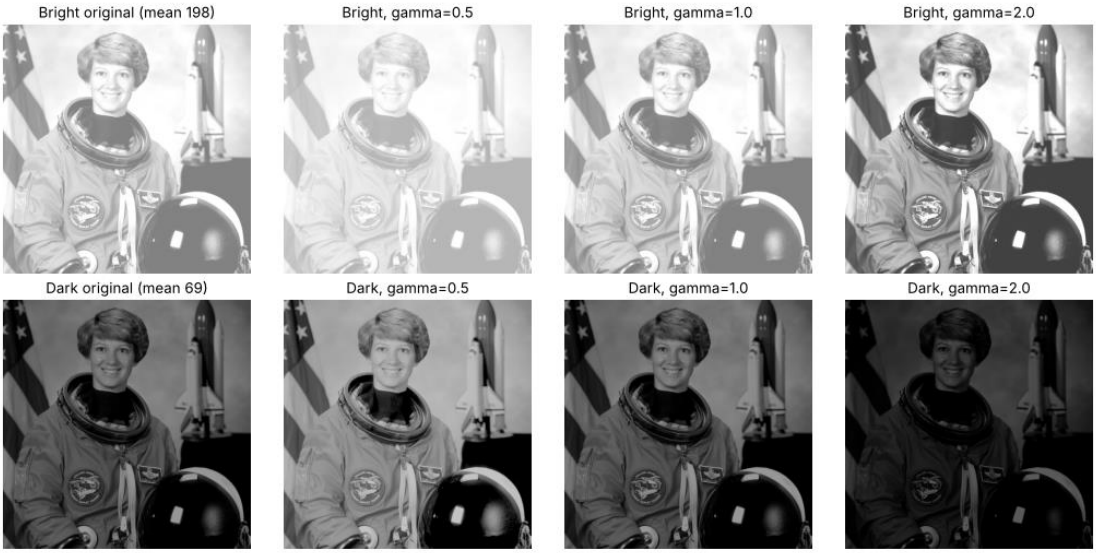


Figure 6. Same γ values on a bright print (top) and a dark print (bottom).

γ = 0.5 lifts a dark print (mean 69 → 118, contrast 45 → 60) but pushes a bright print to near white (mean 198 → 223, contrast 43.8 → 25.5, details washed out). γ = 2.0 does the opposite: it recovers the bright print (contrast 44 → 66) but crushes the dark print (mean 26, contrast 23, 26 % of pixels clipped). A single fixed gamma therefore cannot serve a mixed archive; gamma must be chosen per photograph (e.g. from the mean).

Q6. Pretrained enhancement model vs the selected transformation (reasoned discussion – NOT executed)

A pretrained low-light enhancement network (e.g. Zero-DCE/SCI-type) could not be run here. Reasoned expectation: such networks are trained on natural, noise-free colour photographs, so on faded sepia prints they tend to over-saturate, amplify noise/scratches and may alter tone and facial texture in ways that cannot be audited; the piecewise-linear stretch is deterministic, invertible-looking (it undoes a linear fade), and cannot invent detail. A network could still give better results on severely under-exposed prints, which should be checked experimentally.

Advantages, limitations and selection

- **Negative** trivial; useless for restoration (no contrast gain, polarity reversed).
- **Log** lifts shadows; lowers contrast of bright/faded prints and amplifies dark-region noise.
- **Gamma** preserves end points (≈1 % clipped) and is good for exposure correction; needs a per-image γ and does not widen a compressed histogram.
- **Piecewise-linear stretch** best contrast and faint-detail recovery; inverts a linear fade; costs ~5 % clipping and is sensitive to scratches/impulses (use percentiles).

Selection: piecewise-linear contrast stretching (percentile breakpoints) for faded prints, with per-image gamma as the exposure correction. It is used for the luminance channel of the colour pipeline in Task 10. (In the grayscale pipeline the local method of Task 2 replaces it for uneven prints.)

Task 2 – Histogram Processing and Enhancement

Q7. Implementation

A: cv2.equalizeHist. B: histogram matching to the histogram of a well-exposed clean portrait (skimage.exposure.match_histograms). C: cv2.createCLAHE (clip 2.0, 8×8 tiles; sweep in Q11). Code: Appendix A, task12.py.

Q8. Comparison – detail, contrast, noise amplification, faded / over-exposed regions

Table 5. Task 2 results, mean over the 3 photographs of each set. Noise estimate = mean absolute difference between the image and its 3×3 median.

Set	Technique	RMS contrast	Entropy	Noise (mean |x−med3|)	Clipped px %
light	Original	57.5	7.48	6.3	1.0
light	A Global HE	73.9	7.29	9.7	2.4
light	B Hist. matching	74.9	6.89	8.8	13.6
light	C CLAHE(2.0, 8x8)	60.3	7.76	11.1	1.0
faded	Original	39.7	6.90	6.2	1.0
faded	A Global HE	74.1	6.73	14.5	2.4
faded	B Hist. matching	75.1	6.26	13.3	14.0
faded	C CLAHE(2.0, 8x8)	49.0	7.53	11.8	1.0
uneven	Original	52.1	7.34	6.6	1.1
uneven	A Global HE	73.7	7.12	10.8	2.4
uneven	B Hist. matching	74.7	6.59	11.3	13.9
uneven	C CLAHE(2.0, 8x8)	54.1	7.63	12.0	1.0

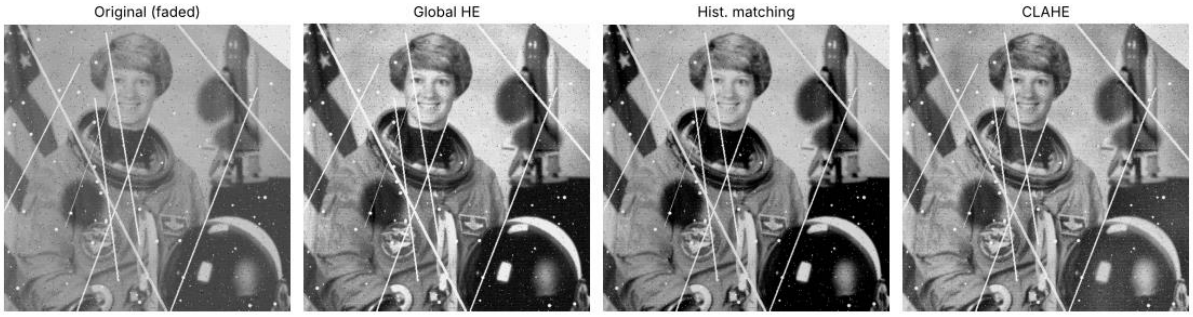


Figure 7. G04 (heavily faded): original, global HE, histogram matching and CLAHE.

Global HE and histogram matching give the largest contrast (RMS ≈ 74) but also roughly double the noise measure (original ≈ 6.3 → 9.7–14.5) and clip 2.4 % (HE) or ≈ 14 % (matching to this reference, whose histogram has a saturated bright background) of the pixels. CLAHE with clip 2 gives a smaller global contrast gain (faded: 39.7 → 49.0) but the highest entropy on every set (7.5–7.8 bits), clips only ≈ 1 % and amplifies noise less than HE on faded and light prints. On the unevenly exposed prints CLAHE evens the two halves best (Q10).

Q9. Histograms and CDF plots

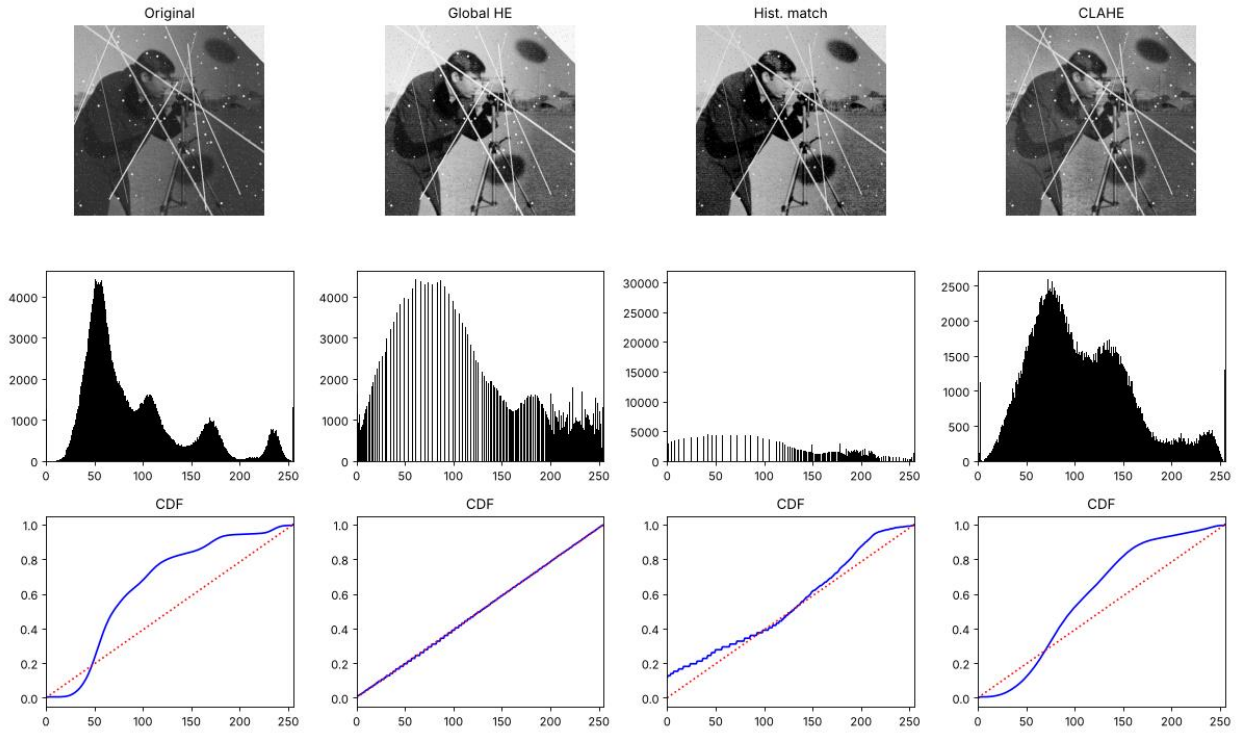


Figure 8. G07 (uneven exposure): image, histogram and CDF for original, global HE, histogram matching and CLAHE. HE and matching force a nearly straight / reference CDF; CLAHE keeps a smoother, gentler CDF.

Q10. Why global HE fails with very dark and very bright regions

Global HE uses one cumulative-histogram mapping for the whole photograph. When one side is in shadow and the other is lit, the pixels of both populations share one mapping, so the mapping is dominated by whichever population is larger and the shadowed region is not lifted independently. Measured on G07 (left half lit, right half in shadow): mean left/right = 92/84 originally; after HE 116/126 (the imbalance changes sign and grows from 7.9 to 10.6 levels), the local contrast (std) of the lit half rises from 62 to 87 while the shadow half only changes from 51 to 54. CLAHE gives a left/right mean of 103/100 (difference 3.5), i.e. a much more even tone, because each tile has its own mapping.

Q11. CLAHE tile size and clip limit

Table 6. CLAHE parameter sweep on G07 (uneven exposure). The same sweep for G04 is in results_t12.json.

Clip	Tile	RMS contrast	Entropy	Noise
1.0	2×2	57.2	7.55	8.3
1.0	4×4	53.7	7.51	8.9
1.0	8×8	51.7	7.46	9.3
1.0	16×16	50.8	7.42	9.6
2.0	2×2	61.6	7.72	10.5
2.0	4×4	55.6	7.66	11.6
2.0	8×8	51.9	7.58	12.3
2.0	16×16	50.4	7.53	13.0
4.0	2×2	66.5	7.84	13.6
4.0	4×4	59.5	7.83	16.2
4.0	8×8	54.6	7.73	17.9
4.0	16×16	51.8	7.65	19.1
8.0	2×2	70.5	7.91	16.3
8.0	4×4	64.3	7.91	21.4
8.0	8×8	61.2	7.89	26.0
8.0	16×16	58.9	7.85	29.2

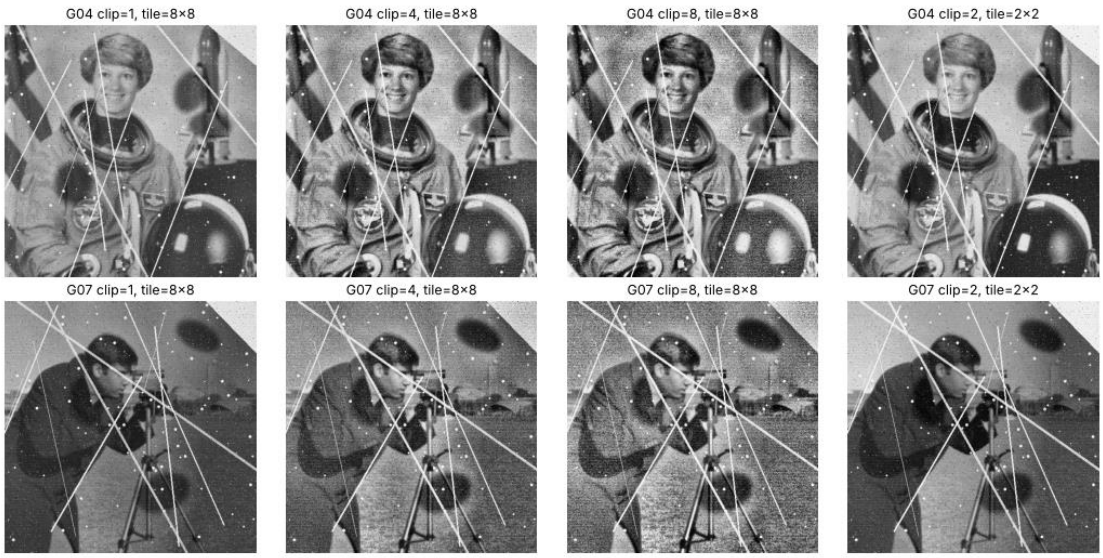


Figure 9. CLAHE results for different (clip, tile) pairs on G04 (top) and G07 (bottom).

Higher clip limits increase entropy and contrast but raise the noise measure steeply (tile 8×8: clip 1 → 9.3, clip 2 → 12.3, clip 4 → 17.9, clip 8 → 26.0). Small tiles (2×2) give the strongest contrast but visible blockiness and noise; large tiles (16×16) approach global equalisation. Choice: 8×8 tiles; clip 2.0 as the pipeline default (entropy 7.58); clip 1.0 gives almost the same RMS contrast (51.7 vs 51.9) with 24 % less measured noise, so it is preferred for the noisiest scans. These were chosen from this sweep only.

Q12. Global HE on a photograph with a large, nearly uniform background

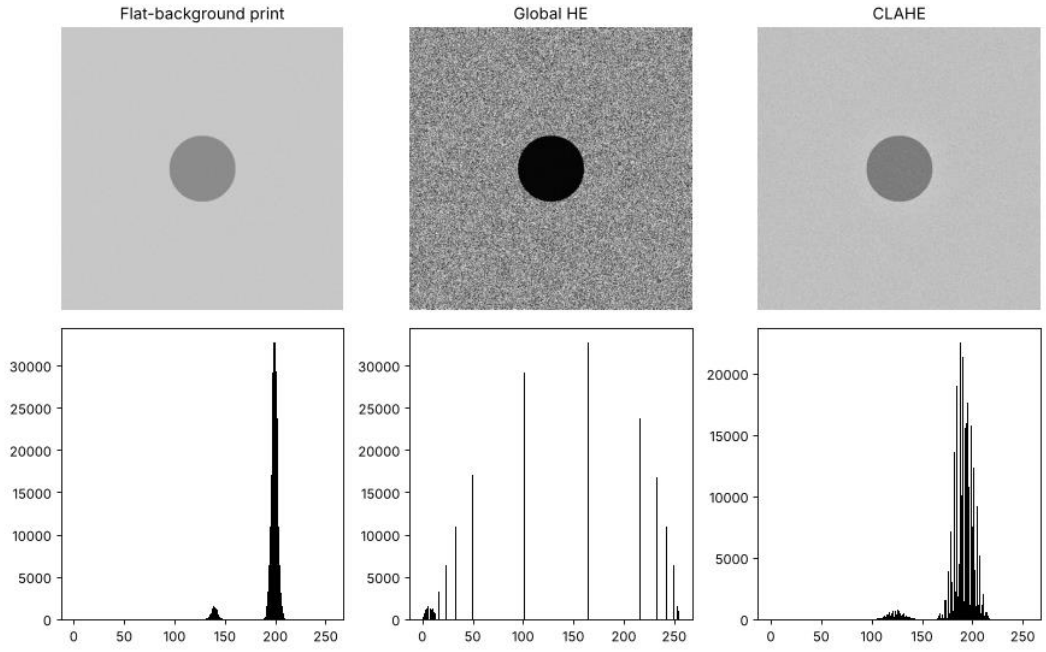


Figure 10. Small subject on a flat paper background: original, global HE and CLAHE with histograms.

On a test print with a flat background (σ ≈ 3 grey levels, mean 200) global HE stretches the single narrow histogram peak over a wide range: background σ rises from 3.0 to 70.0 grey levels (≈ 23×) and the background mean drops from 200 to 144, i.e. noise and paper texture become strong visible patterns and the tone of the paper changes. CLAHE (clip 2) limits this to σ = 8.6.

Advantages, limitations and selection

- **Global HE** fast (0.3 ms); strong contrast; amplifies noise, clips highlights, fails with mixed exposure and flat backgrounds, changes the historical tone.
- **Histogram matching** controllable look if a good reference exists; result depends entirely on the reference (here 14 % clipping).
- **CLAHE** local, highest entropy, least clipping, handles uneven tone; needs clip/tile tuning, still amplifies noise at high clip.

Selection: CLAHE (8×8 tiles, clip 1–2) – best entropy, ~1 % clipping, best local evenness; used in the grayscale pipeline.

Task 3 – Spatial Filtering and Image Smoothing

Test data for Task 3: three clean photographs (astronaut, camera, coins) with film-grain noise added (Gaussian σ=6, 1 % salt-and-pepper, +4 grey-level stripes every 6 rows), so that PSNR/SSIM against the clean image are meaningful.

Q13. Implementation

cv2.blur (box), cv2.GaussianBlur (σ derived from kernel size), cv2.medianBlur with 3×3, 5×5, 7×7 kernels. Code: Appendix A, task34.py.

Q14. Comparison – noise removal, detail preservation, time

Q15. PSNR and SSIM

Table 7. Task 3 PSNR/SSIM/time against the clean photograph, mean over 3 photographs.

Filter	Kernel	PSNR (dB)	SSIM	Time (ms, 512×512)
Noisy input	–	24.18	0.641	–
Box	3×3	28.65	0.777	0.09
Box	5×5	26.66	0.759	0.10
Box	7×7	24.95	0.713	0.32
Gaussian	3×3	29.45	0.792	0.09
Gaussian	5×5	28.84	0.802	0.15
Gaussian	7×7	27.54	0.791	0.25
Median	3×3	31.25	0.869	0.08
Median	5×5	28.57	0.836	0.66
Median	7×7	26.79	0.791	9.81



Figure 11. Filtered results (crop of the astronaut face) for the three filters and three kernel sizes.

Median 3×3 is clearly best: PSNR 31.25 dB and SSIM 0.869 against 24.18 dB / 0.641 for the noisy input, because it removes the salt-and-pepper impulses completely instead of smearing them. Among linear filters Gaussian is better than box at every size (e.g. 5×5: 28.84 dB / 0.802 vs 26.66 dB / 0.759). Larger kernels reduce Gaussian noise further but PSNR falls for all filters beyond 3×3 (median 7×7: 26.8 dB) because detail is lost. Median 7×7 is also much slower (9.8 ms vs 0.3 ms for box 7×7 on this machine).

Q16. Intensity profiles

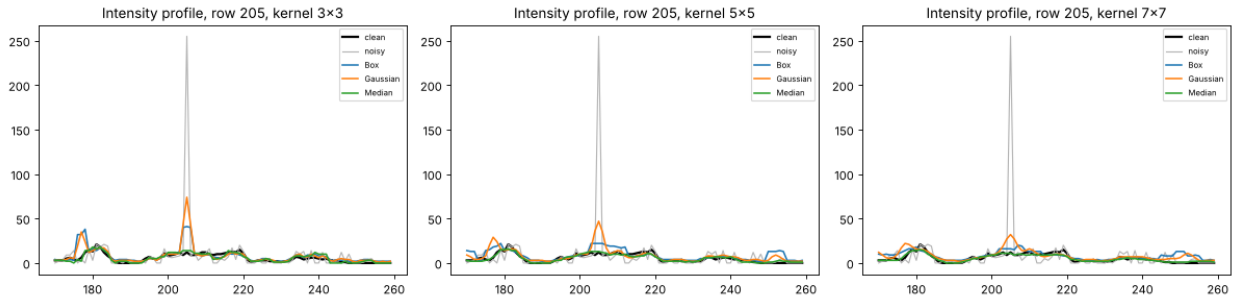


Figure 12. Grey-level profile along one row through the face (astronaut) for 3×3, 5×5 and 7×7 kernels: clean (black), noisy (grey) and the three filters.

Q17. Correlation vs convolution with a 3×3 mask

An impulse image was filtered with the asymmetric mask [[1,2,3],[4,5,6],[7,8,9]]. Correlation (what cv2.filter2D computes) places the mask 180° rotated around the impulse: central 3×3 patch = [9.0, 8.0, 13.0], [6.0, 5.0, 4.0], [3.0, 2.0, 1.0]. Convolution (mask flipped first) reproduces the mask itself around the impulse: [1.0, 2.0, 17.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]. (The extra 13 / 17 in the corner cells come from the second impulse in the test image.) For symmetric masks – box and Gaussian – the two operations give identical results (checked numerically), so the distinction matters only for directional masks such as Sobel or line masks.

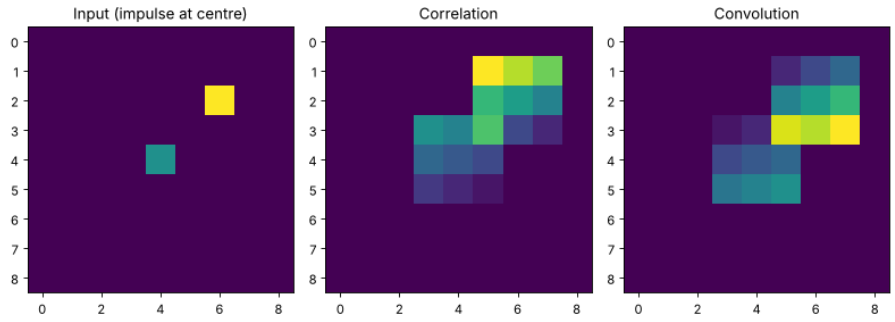


Figure 13. Input, correlation result and convolution result for an asymmetric 3×3 mask.

Q18. Consequences of a large 7×7 filter on small details

Table 8. A 2-px dark line (grey 60 on 150) after filtering.

Filter	Kernel	Remaining contrast of a 2-px line (original 90)
Box	3×3	60
Box	5×5	36
Box	7×7	26
Gaussian	3×3	67
Gaussian	5×5	56
Gaussian	7×7	45
Median	3×3	90
Median	5×5	0
Median	7×7	0

On the astronaut face region the mean gradient (edge) energy retained is 0.58 (box 7×7), 0.68 (Gaussian 7×7) and 0.61 (median 7×7), against 0.81–0.85 for 3×3 kernels. A 2-px line (hair strand, narrow boundary) keeps only 26/90 of its contrast with a 7×7 box and 45/90 with a 7×7 Gaussian, and a median of 5×5 or larger deletes it entirely (a feature thinner than half the window is removed by a median) – while a 3×3 median keeps it at full contrast. Eyes, hair strands and narrow boundaries therefore disappear or blur with 7×7 kernels, which conflicts with the requirement to preserve facial features.

Q19. Classical filter vs pretrained DnCNN (reasoned discussion – NOT executed)

DnCNN was not available. Reasoned expectation: a DnCNN trained on Gaussian noise normally beats classical filters in PSNR/SSIM on Gaussian noise, but it was trained for σ-specific Gaussian noise, not for impulse noise, scanner stripes or scratches, and it can hallucinate texture on faces. A practical combination would be median 3×3 for impulses followed by a learned denoiser, to be verified experimentally. Classical filters need no GPU and run well inside the time budget.

Advantages, limitations and selection

- **Box** fastest, but blurs edges uniformly and leaves ringing-like blockiness; worst SSIM.
- **Gaussian** best linear trade-off (best SSIM 0.802 at 5×5); cannot remove impulses (they are smeared, not removed).
- **Median** removes salt-and-pepper and dust, preserves edges; removes lines thinner than the window; slow at large kernels; weak against Gaussian grain.

Selection: median 3×3 first (impulse/dust), then Gaussian 3×3 for grain; no 7×7 kernels. The pipeline uses exactly this pair.

Task 4 – Image Sharpening and Edge Enhancement

Test data: the clean photographs blurred with a Gaussian (σ = 1.6) plus mild noise (σ = 3), so that sharpening can be scored against the clean original. Edge halo/overshoot and rise distance were measured on a synthetic step edge blurred with the same σ.

Q20. Implementation

A: Laplacian sharpening $g = f + 1.0 \cdot \nabla^2 f$ (4-neighbour kernel). B: unsharp masking $g = f + a \cdot (f - G_\sigma * f)$ with $\sigma = 1.5$, $a = 1.5$; high-boost with $a = 4$. C: gradient-based sharpening $g = f + c \cdot |\nabla f|$ (Sobel magnitude), $c = 0.5$ (scaled). Code: Appendix A, task34.py.

Q21. Comparison – sharpness, noise amplification, halos, natural detail

Q22. Variance of the Laplacian

Table 9. Task 4 results (mean over 3 blurred photographs; overshoot/rise on the synthetic step edge: clean edge has rise 0, blurred edge has rise 4 px).

Method	Var(Laplacian)	PSNR dB	SSIM	Noise in flat patch	Overshoot (grey levels)	10–90 % rise (px)	ms
Blurred input	192.8	26.97	0.771	5.0	0	4	–
A Laplacian	7917.1	23.00	0.378	14.2	6	2	0.82
B Unsharp (s=1.5,a=1.5)	1130.2	27.18	0.641	8.6	13	2	1.56
B High-boost (a=4)	4308.8	23.57	0.449	14.5	49	2	1.50
C Gradient (c=0.5)	276.4	22.69	0.697	6.2	16	3	1.85



Figure 14. Crop of the astronaut photograph: clean, blurred, and the sharpening results.

Variance of Laplacian rises from 193 (blurred) to 1130 (unsharp a = 1.5), 4309 (high-boost a = 4) and 7917 (Laplacian), but a high value alone is not "better": in the Laplacian and high-boost results it is dominated by amplified noise (flat-patch noise 5.0 → 14.2 and 14.5) and the SSIM against the clean photograph falls from 0.771 to 0.378 and 0.449. The unsharp mask with a = 1.5 is the only method that keeps PSNR at or above the blurred input (27.18 vs 26.97 dB) while shortening the edge rise from 4 to 2 px, with 13 grey levels of overshoot (a visible but mild halo); the Laplacian result overshoots less on the clean step (6) but produces strong noise. The gradient-based method at c = 0.5 gave only a small sharpness increase (Laplacian variance 276), a 3 px rise and 16 levels overshoot; it keeps the highest SSIM among sharpeners (0.697) but gives the lowest PSNR (22.7 dB).

Q23. Intensity profiles

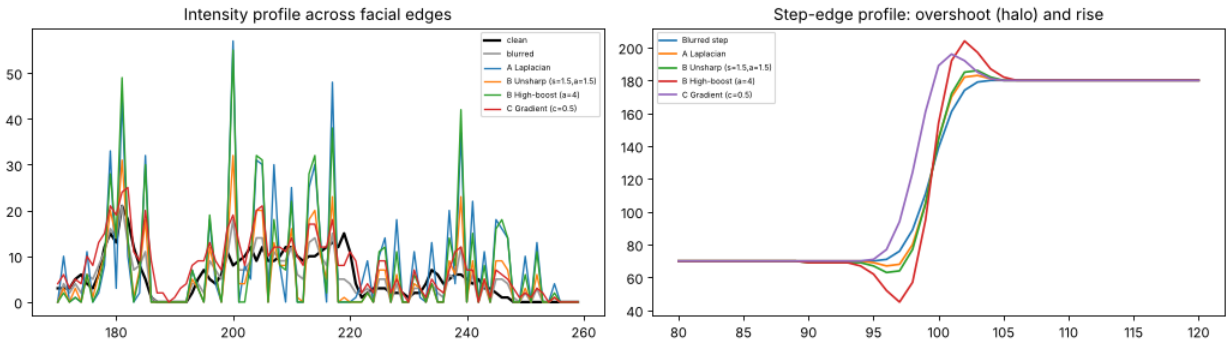


Figure 15. Left: profile across facial edges (row 205). Right: step-edge profile showing overshoot (halo) and edge rise for each method.

Q24. Why smoothing before sharpening helps

Sharpening filters are high-pass: they multiply the noise as much as the edges. Smoothing first with a Gaussian σ = 1.0 removes the highest noise frequencies before they are boosted. Measured on the blurred astronaut: unsharp mask (a = 2) – flat-patch noise 15.6 → 12.2, PSNR 26.53 → 27.72 dB, SSIM 0.660 → 0.876; Laplacian – noise 17.1 → 10.7, PSNR 23.06 → 26.90 dB, SSIM 0.441 → 0.841. (The Laplacian is a second derivative and benefits most; LoG = smoothing + Laplacian is the same idea.)

Q25. High-boost with a large amplification factor on scratched / grainy photographs

Table 10. High-boost on the scratched, grainy faded photograph G04 (σ = 1.5).

Boost amount a	Var(Laplacian)	Noise in flat patch	SSIM vs clean	Clipped %
1	10411	25.7	0.278	3.5
2	15897	33.9	0.237	4.9
4	29131	45.7	0.185	7.9
8	61801	61.7	0.133	17.8

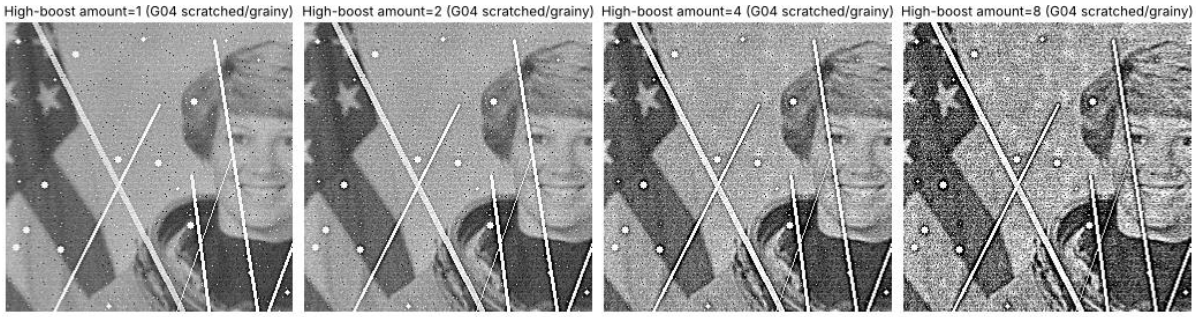


Figure 16. Top-left corner of G04 after high-boost with a = 1, 2, 4, 8: grain, dust and scratch edges are amplified together with real detail.

With a = 8 the measured noise is 2.4× larger than with a = 1 (61.7 vs 25.7), clipping rises from 3.5 % to 17.8 %, and SSIM against the clean photograph falls from 0.278 to 0.133. Scratches gain bright/dark halo edges and dust becomes more conspicuous, i.e. artificial edges are created – the opposite of the brief.

Q26. Traditional sharpening vs a pretrained deblurring network (reasoned discussion – NOT executed)

No deblurring network could be run. Reasoned: networks such as NAFNet/Restormer can recover more detail from motion or defocus blur than unsharp masking, but they synthesise plausible texture that was never in the print, which is unacceptable for faces in historical photographs unless labelled; unsharp masking only re-weights existing frequencies. The need for a GPU and the lack of an audit trail are additional drawbacks for this archive.

Advantages, limitations and selection

- **Laplacian** simple, strongest edge boost; very noise-sensitive; second-derivative sign handling and clipping are needed; worst SSIM.
- **Unsharp/high-boost** controllable (σ, a); halo grows with a; best balance at a ≈ 1–2.
- **Gradient** weak, positive-only edge emphasis; leaves some noise; little sharpness gain at c = 0.5.

Selection: unsharp masking with a ≈ 0.7–1.5, σ ≈ 1.5 applied after light smoothing; amounts above 2 are rejected for scratched/grainy prints. The pipeline uses a = 0.7.

Task 5 – Frequency-Domain Low-Pass Filtering

Q27. Implementation

Images are zero-padded to 2M × 2N, transformed with the FFT, multiplied by H(u,v) and transformed back (ILPF: D ≤ D0; Butterworth BLPF: n = 2; Gaussian GLPF: $\exp(-D^2/2D0^2)$). D0 is given in cycles per 512 px of the original image. Code: Appendix A, task56.py. Test data: the same noisy photographs as Task 3.

Q28. Frequency spectrum – patterns of grain, scratches, periodic artefacts

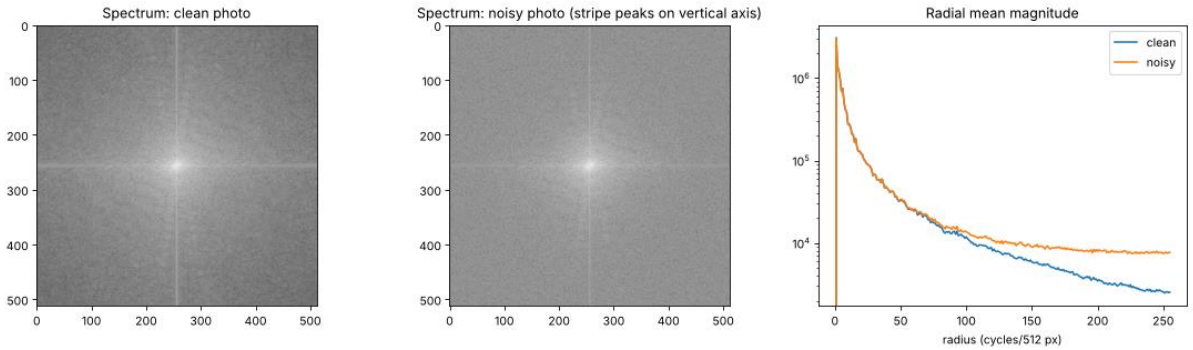


Figure 17. Log spectrum of the clean photograph (left), the noisy photograph (centre; stripe peaks along the vertical axis) and radial mean magnitude (right).

The scanner stripes (period 6 rows) produce a discrete pair of spikes on the vertical frequency axis at ≈ 85 cycles per 512 px (expected 512/6 = 85.3); the peak is 2.0× the neighbouring magnitudes. Film/sensor grain and impulse noise raise the whole high-frequency plateau: at radii 150–250 the noisy spectrum is 2.1× the clean one. Long straight scratches appear as lines through the origin perpendicular to the scratch direction and have broadband energy, so they are not separable from edges by low-pass filtering.

Q29. Different cut-off frequencies

Q33. Edge/detail preservation (edge retention)

Table 11. Task 5 results, mean over 3 noisy photographs. Edge retention = mean gradient of filtered / clean (≈1 is ideal; >1 means noise energy is left).

Filter	D0	PSNR (dB)	SSIM	Edge retention	Time ms (512², padded)
ILPF	20	20.44	0.508	0.44	64
ILPF	40	23.38	0.614	0.64	66
ILPF	80	26.75	0.727	0.96	66
ILPF	120	27.90	0.731	1.20	68
BLPF	20	21.17	0.589	0.34	93
BLPF	40	24.28	0.716	0.52	98
BLPF	80	27.77	0.799	0.80	93
BLPF	120	29.05	0.795	1.02	95
GLPF	20	21.71	0.617	0.35	75
GLPF	40	24.93	0.745	0.54	73
GLPF	80	28.26	0.803	0.84	72
GLPF	120	29.09	0.789	1.03	75



Figure 18. Crop of the astronaut photograph filtered with ILPF/BLPF/GLPF for D0 = 20, 40, 80, 120 cycles.

D0 = 20 over-smooths (PSNR ≈ 20–22 dB, edge retention 0.35–0.44); D0 = 80 is a good compromise for the Gaussian and Butterworth filters (GLPF 28.26 dB, SSIM 0.803, edge retention 0.84); at D0 = 120 PSNR is higher (≈ 29 dB) but edge retention exceeds 1, which means much of the noise passes. The ideal filter has the lowest PSNR and SSIM at every cut-off (D0 = 80: 26.75 dB, SSIM 0.727) because of ringing.

Q30. Ringing, loss of detail, blurring, processing time

Table 12. Ringing on a noise-free step edge (60 → 190 grey levels).

Filter	D0	Overshoot	Undershoot	Ringing energy (Σ|dev|, 40 px)
ILPF	20	11.82	11.73	137.73
ILPF	40	11.53	11.43	71.16
BLPF	20	4.36	4.36	18.39
BLPF	40	4.50	4.50	0.61
GLPF	20	0.00	0.00	0.00
GLPF	40	0.01	0.01	0.07

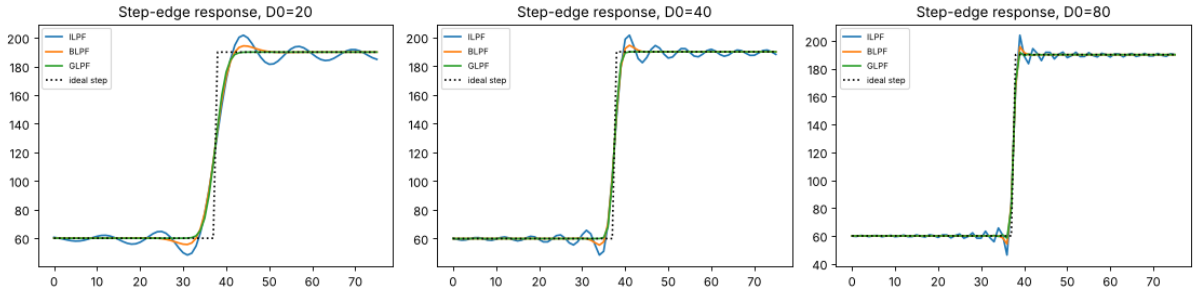


Figure 19. Step-edge responses: the ideal filter oscillates, Butterworth (n = 2) has slight ringing, Gaussian has none.

The ideal filter overshoots by 11.5–11.8 grey levels with persistent oscillation (ring energy 137 at D0 = 20), the second-order Butterworth by 4.4–4.5 levels with rapid decay, and the Gaussian filter essentially none (< 0.02). Edge blurring is the price of smaller D0 (edge retention table). All frequency-domain filters took ≈ 70–107 ms on 512×512 with padding (mean of best-of-2), against 0.2–1.3 ms for the equivalent spatial Gaussian.

Q31. Frequency-domain vs spatial-domain Gaussian

Table 13. Frequency-domain GLPF vs spatial Gaussian with σ = 512/(2π·D0).

D0	σ spatial (px)	PSNR freq	PSNR spatial	SSIM freq	SSIM spatial	Agreement of outputs (dB)	ms freq	ms spatial
20	4.07	20.74	20.77	0.599	0.592	51.0	99.3	2.24
40	2.04	24.22	24.27	0.752	0.745	51.1	79.1	0.45
80	1.02	27.83	27.83	0.813	0.805	51.1	86.4	0.27

A Gaussian is its own Fourier transform, so the two are the same filter: the outputs agree to ≈ 51 dB PSNR (differences are rounding/border handling) and the PSNR/SSIM against the clean image are equal to within 0.05 dB. The frequency-domain version is 60–300× slower at this size (66–75 ms vs 0.2–1.3 ms) – the spatial form is the right choice unless a large kernel or a combined notch/stripe filter is required.

Q32. Effect of omitting padding



Figure 20. Gaussian low-pass with and without padding, and the absolute difference (wrap-around error).

Without padding the DFT assumes a periodic image, so content from the opposite border leaks in (wrap-around error). Mean absolute error in a 12-px border strip against the correct spatial result: 10.11 grey levels without padding vs 0.29 with padding; the interior is unaffected (0.26 in both cases). For photographs with a bright border and a dark edge (or vice-versa) this shows as bright/dark bands at the borders.

Q33. (covered with Q29 above) Edge retention measure – OCR table not applicable

The brief mentions an OCR performance table. The test photographs contain no text, so no OCR measurement was made; edge retention and PSNR/SSIM are given instead.

Advantages, limitations and selection

- **ILPF** sharpest cut-off but severe ringing and the worst PSNR/SSIM.
- **BLPF** adjustable order; small ringing; between the other two.
- **GLPF** no ringing, best SSIM at D0 = 80; equivalent to a spatial Gaussian (so no unique benefit); slowest on A4 (padded FFT = 5.6 s, Section 9.4).

Selection: Gaussian low-pass (D0 ≈ 80 per 512 px, equivalent to σ ≈ 1 px), implemented in the spatial domain in the A4 pipeline because the padded FFT alone exceeds the time budget. An FFT notch is only used for periodic stripe artefacts in the colour redesign.

Task 6 – Frequency-Domain High-Pass Filtering

Q34. Implementation

Test data: three shadow-affected photographs (fade + one-sided shadow + noise, no scratches, so illumination can be measured). A: Butterworth high-pass (n = 2, D0 = 30) with the mean added back. B: Gaussian high-frequency emphasis H = 0.5 + 1.5·H_hp. C: homomorphic filtering (ln → FFT → H = (γH−γL)(1−exp(−c·D²/D0²))+γL, γL = 0.5, γH = 1.8, c = 1, D0 = 30 → exp), rescaled to the input mean. Code: Appendix A, task56.py.

Q35. Comparison – sharpness, uneven fading, noise, tone

Q36. Local contrast and intensity variation in uniform regions

Table 14. Task 6 results, mean over 3 shadowed photographs. The clean row is the undamaged reference.

Photograph / method	Illumination variation (CV, σ=40 blur)	Local contrast (15×15 std)	Noise (|x−med3|)	Std in uniform 40×40 patch	Mean	RMS	ms
Shadowed input	0.355	10.6	3.3	4.1	77.8	38.8	0
A Butterworth HPF	0.008	8.0	3.4	4.1	77.9	10.0	105
B Gaussian HFE	0.359	15.3	6.5	7.4	39.8	26.7	109
C Homomorphic	0.216	19.8	6.2	11.0	68.4	39.5	103
Clean target	0.323	19.1	2.6	–	113.7	67.8	–

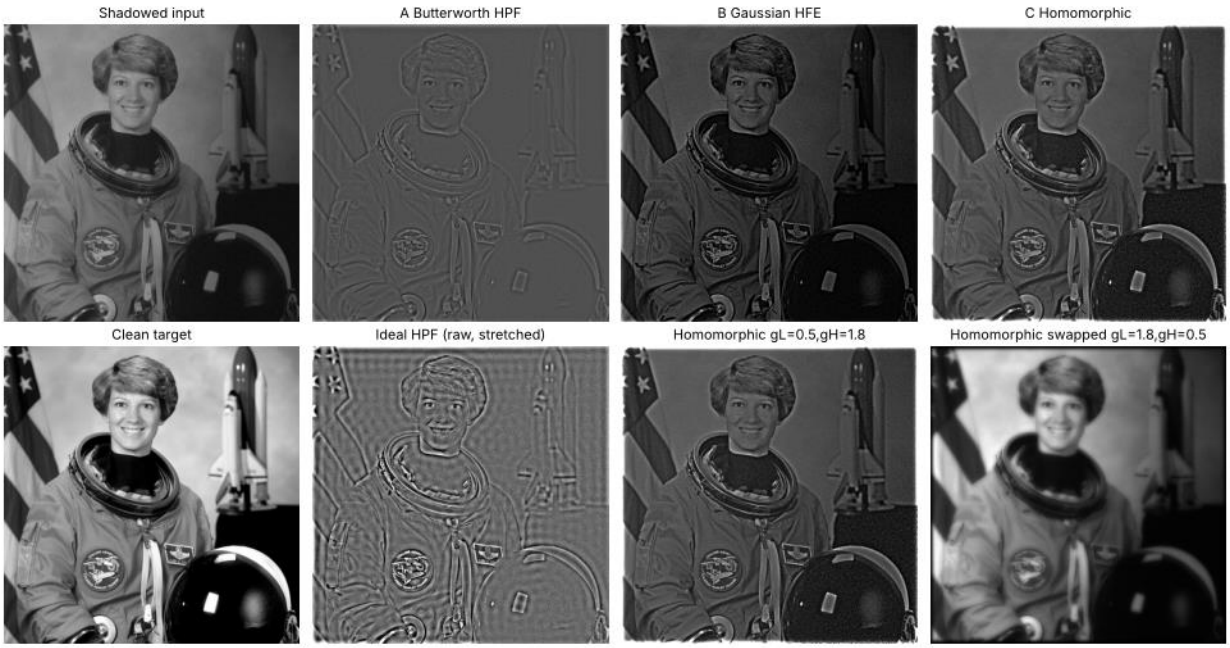


Figure 21. Top: shadowed input and results of A, B, C. Bottom: clean target, raw ideal high-pass (stretched), homomorphic with correct gains and with swapped gains.

Findings (honest reading): (i) The Butterworth high-pass (A) removes the shadow completely (illumination variation 0.355 → 0.008) but also removes all natural large-scale tone (the clean photograph has 0.323), leaving a flat edge map – not acceptable for preserving photographic tone (PSNR/SSIM after gain fit fall from 15.9 dB/0.66 to 11.9 dB/0.50). (ii) High-frequency emphasis (B) sharpens (local contrast 10.6 → 15.3) but does not correct the shadow (0.359) and doubles the noise measure (3.3 → 6.5). (iii) Homomorphic filtering (C) reduces the shadow gradient (0.355 → 0.216) and raises local contrast to 19.8 (clean: 19.1) but also amplifies noise (uniform-patch σ 4.1 → 11.0) and slightly over-corrects (0.216 vs the clean 0.323). On a z-scored coarse-illumination error against the clean photograph none of the three beat the uncorrected input (input 0.80; A 1.23; B 0.81; C 0.85), so with these gains homomorphic filtering is the most useful but not a clean win; gentler γL (e.g. 0.7–0.8) was not tested and would probably over-correct less.

Q37. Ideal high-pass applied directly

The ideal high-pass removes the DC term and every low frequency: the mean fell from 80.7 to 0.5, the output ranges from -97 to +108 (negative values need re-scaling, so absolute tone is lost), and its sharp cut-off rings: on a step edge the peak-to-peak oscillation is 100 grey levels. Around faces and strong edges this creates ghost contours/halos (Figure above, bottom row, second panel) – i.e. invented structure.

Q38. Interchanging γL and γH in homomorphic filtering

With γL = 0.5, γH = 1.8 the illumination is suppressed and reflectance (detail) is boosted: illumination variation 0.23, local contrast 22.0 (input 0.37 / 12.7). With the values swapped (γL = 1.8, γH = 0.5) the shadow is amplified and detail suppressed: illumination variation 0.54, local contrast 14.0; the result is blurred with a dark, exaggerated shadow (Figure above, bottom right).

Q39. Homomorphic filtering vs pretrained illumination / low-light model (reasoned discussion – NOT executed)

No illumination-correction network was run. Reasoned: a network can adapt to complex, non-multiplicative shadows and colour, but it is trained on natural colour photographs, cannot be audited for invented content, and needs a GPU; homomorphic filtering has 4 interpretable parameters and no learning, but assumes a purely multiplicative, slowly varying illumination (it fails when shadows have sharp edges).

Advantages, limitations and selection

- **Butterworth/ideal HP** sharpens/edge-detects; loses tone; ideal version rings badly.
- **Gaussian HFE** keeps tone, sharpens, no ringing; does not correct shadow; amplifies noise.
- **Homomorphic** addresses illumination and detail simultaneously; sensitive to γL, γH and D0; noise amplified.

Selection: homomorphic filtering with moderate gains (γL ≈ 0.5–0.8, γH ≈ 1.5–1.8) for shadow-affected photographs. In the A4 pipeline its spatial equivalent (division by a heavily smoothed illumination estimate) is used because the padded FFT is too slow; this equivalence is by design and was timed but its quality was not separately compared with the FFT version.

Task 7 – Point, Line and Edge Detection

Q40. Implementation

A: 3×3 point mask (−1…8) and four 3×3 line masks (0°, 90°, ±45°); the maximum signed response is thresholded (bright points/lines on a darker surround). B: Roberts, Prewitt and Sobel gradient magnitude after a σ = 1 Gaussian. C: Marr–Hildreth (LoG σ = 1.5, zero crossings with a slope test) and Canny (σ = 1, high = 2×low). Code: Appendix A, task78.py.

Q41. Detection of dust, scratches, creases, stain boundaries, torn regions

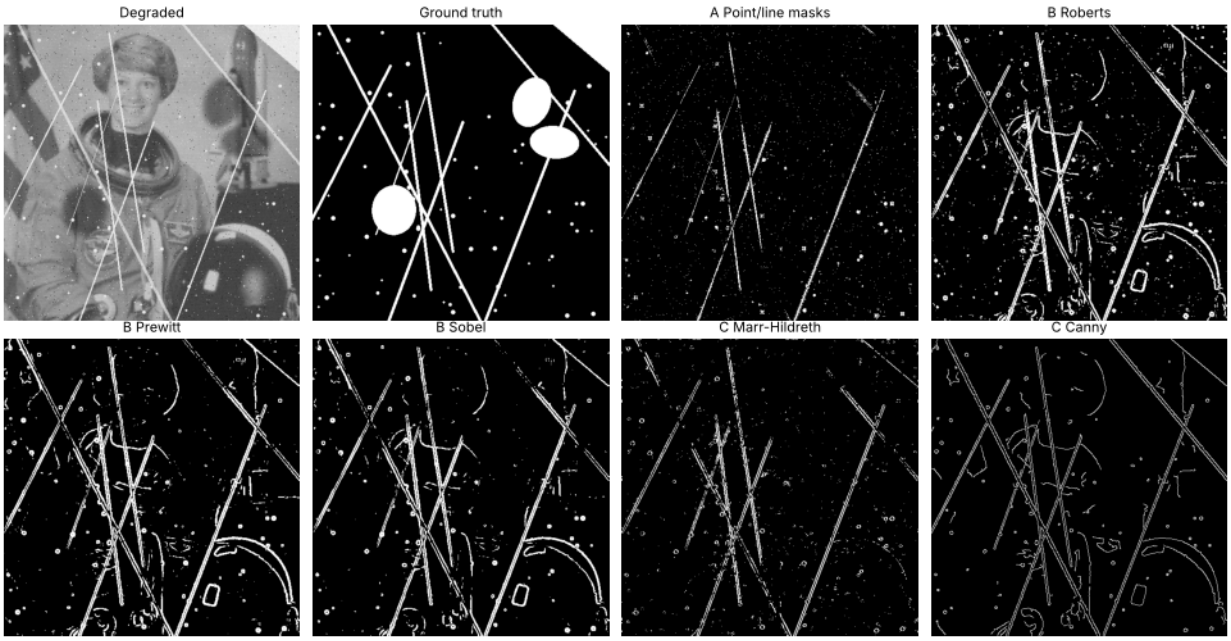


Figure 22. Detection maps on the faded photograph G04 at each method's best threshold, with the ground-truth damage mask.

Q42. Noise sensitivity, false detections, missed damage

Q43. Precision and recall against ground truth

Table 15. Task 7 precision/recall (±2 px tolerance) against the full ground-truth damage mask, mean over 9 photographs. Threshold chosen on the same 9 photographs.

Detector	Best threshold (max mean F1)	Precision	Recall	F1
A Point/line masks	80	0.719	0.502	0.586
B Roberts	20	0.628	0.578	0.581
B Prewitt	80	0.671	0.541	0.577
B Sobel	120	0.706	0.510	0.570
C Marr-Hildreth	8	0.738	0.528	0.601
C Canny	100	0.686	0.496	0.554

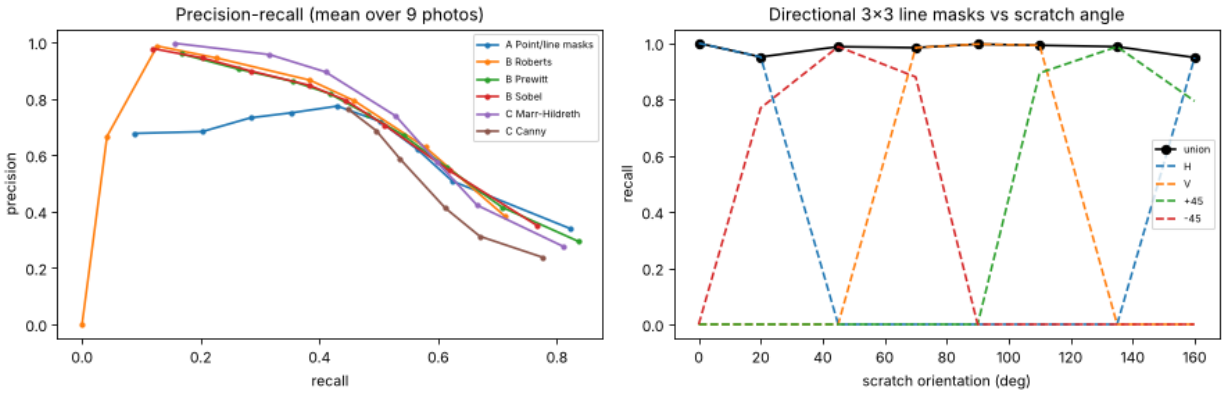


Figure 23. Left: precision–recall curves obtained by sweeping each detector's threshold. Right: recall of each directional line mask versus scratch orientation.

All six detectors reach F1 ≈ 0.55–0.60 (Marr–Hildreth 0.601, point/line masks 0.586, Roberts 0.581, Prewitt 0.577, Sobel 0.570, Canny 0.554); the differences between them are smaller than the photograph-to-photograph variation, so none can be called clearly superior from 9 images. Precision is limited (0.63–0.74) because every natural edge (hair, clothing, flag) is also an edge – gradient and Canny/Marr–Hildreth detectors cannot tell damage from content – and recall (0.50–0.58) is limited because stains (soft dark blobs with weak gradients) and thin low-contrast creases are missed. Roberts has the highest recall but the lowest precision (it is the noisiest

operator). Impulse noise is a major false-detection source: a pre-smoothing step was applied to every detector. Point/line masks have the advantage of sign selectivity (they respond only to bright thin structures), which is why their precision (0.72) is among the highest.

Q44. Thresholds for dust, thin scratches and major boundaries

Table 16. Threshold study on a synthetic test patch (dust discs r = 2–3, 1–2 px scratches, a torn corner, noise σ = 6).

Detector	Threshold	Dust recall	Thin-scratch recall	Torn-edge recall	False pixels %
A	20	1.00	1.00	0.38	1.61
A	40	1.00	1.00	0.16	0.00
A	60	1.00	1.00	0.14	0.00
A	100	1.00	0.99	0.10	0.00
A	160	0.37	0.27	0.01	0.00
A	260	0.00	0.00	0.00	0.00
Sobel	40	1.00	1.00	0.16	0.00
Sobel	80	1.00	1.00	0.16	0.00
Sobel	160	1.00	1.00	0.14	0.00
Sobel	260	0.63	0.63	0.11	0.00
Sobel	400	0.00	0.00	0.00	0.00

Small dust spots and thin scratches are fully detected by the point/line masks over a wide threshold band (T = 20–100, recall ≈ 1.0 with ≤ 1.6 % false pixels, none from T = 40) and by Sobel for T up to ≈ 160; recall collapses above T ≈ 160 (masks) / 260 (Sobel). A threshold of about 40–80 (masks, 3×3, σ = 0.8 pre-smoothing) was therefore used for dust and scratches – high enough to reject noise σ = 6 and low enough to catch 1-px lines. Torn boundaries are poorly captured by line/point detection (edge recall ≤ 0.38) and need a gradient or area-based method with a high threshold (≥ 100 for Sobel) combined with region filling.

Q45. Why directional line masks fail for different scratch orientations

Table 17. Recall of single directional masks on a 1-px scratch at different angles (threshold 80).

Scratch angle (°)	H mask	V mask	+45° mask	–45° mask	Union
0	1.00	0.00	0.00	0.00	1.00
20	0.95	0.00	0.00	0.77	0.95
45	0.00	0.00	0.00	0.99	0.99
70	0.00	0.98	0.00	0.88	0.98
90	0.00	1.00	0.00	0.00	1.00
110	0.00	0.99	0.89	0.00	0.99
135	0.00	0.00	0.99	0.00	0.99
160	0.95	0.00	0.79	0.00	0.95

Each 3×3 line mask matches one orientation only: a scratch at 45° or 135° gives zero response from the horizontal and vertical masks (recall 0.00), while intermediate angles (20°, 70°, 110°, 160°) are caught only partly (0.77–0.95) by two masks. A union of all four masks covers 0.95–1.00 here, but with a union a crease crossing a scratch, curved cracks or lines thicker than 1 px still violate the single-line, fixed-width assumption of the 3×3 masks, and each additional orientation increases false detections from noise.

Q46. Traditional edge detector vs HED (reasoned discussion – NOT executed)

HED could not be run. Reasoned: HED produces cleaner, closed object contours and suppresses texture better than Canny, which would reduce false detections on natural content, but it was trained on natural-image edge labels, not on scratches or stains; it would be expected to suppress thin scratches as 'texture' or to miss low-contrast damage, and it needs a GPU and a pretrained model that has to be validated on annotated prints.

Advantages, limitations and selection

- **Point/line masks** sign-selective, very cheap (3.5 ms); only 4 orientations and 1-px width.
- **Roberts/Prewitt/Sobel** simple; Sobel more noise-robust than Roberts; cannot separate damage from natural edges.

- **Marr–Hildreth/Canny** thin, localised contours; Canny's hysteresis links edges, MH needs a slope test; both detect natural edges.

Selection: sign-selective point/line detection (A) for dust and thin scratches, with Sobel/Canny as supporting boundary cues; the differences in overall F1 are small (0.55–0.60) and should not be over-interpreted. The Task 7 detector alone does not meet a Dice target of 0.90.

Task 8 – Image Thresholding and Binarization

Q47. Implementation

A: global threshold T = 200 (bright damage). B: Otsu on the σ = 1 smoothed image. C: Gaussian adaptive threshold (block 31, C = −12). Binary masks are cleaned by removing connected components smaller than 4 px (isolated impulses). Damage is the bright class. T = 200 was chosen with knowledge of the 235 damage level, which favours method A (see Section 0). Code: Appendix A, task78.py.

Q48. Lightly damaged, heavily faded, unevenly exposed photographs

Q50. Dice scores

Table 18. Task 8 Dice scores, mean over 3 photographs per set (512×512).

Method	Set	Dice (bright damage, raw)	Dice (bright damage, cleaned)	Dice vs full mask (incl. stains)	ms
A Global T=200	light	0.652	0.732	0.373	0.04
A Global T=200	faded	0.905	0.954	0.548	0.03
A Global T=200	uneven	0.789	0.823	0.459	0.08
B Otsu	light	0.085	0.085	0.151	0.43
B Otsu	faded	0.133	0.133	0.162	0.42
B Otsu	uneven	0.306	0.306	0.321	0.43
C Adaptive 31,C=-12	light	0.293	0.301	0.299	2.25
C Adaptive 31,C=-12	faded	0.508	0.528	0.416	2.46
C Adaptive 31,C=-12	uneven	0.562	0.571	0.485	2.35

Q49. Binary damage masks and histograms

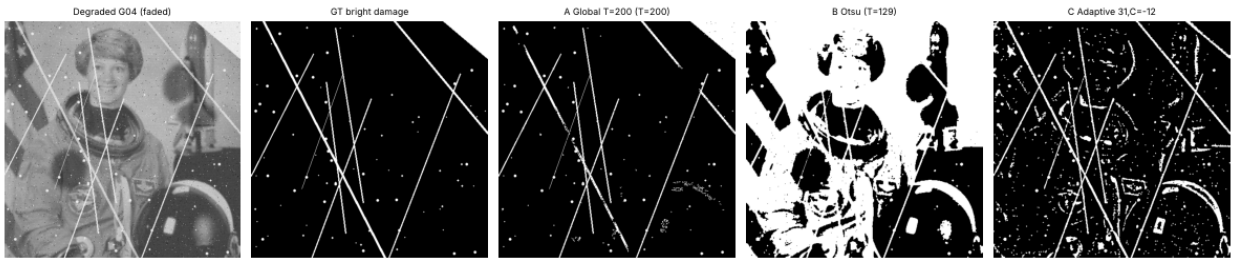


Figure 24. Binary damage masks on G04 for the three thresholding methods, with the ground truth.

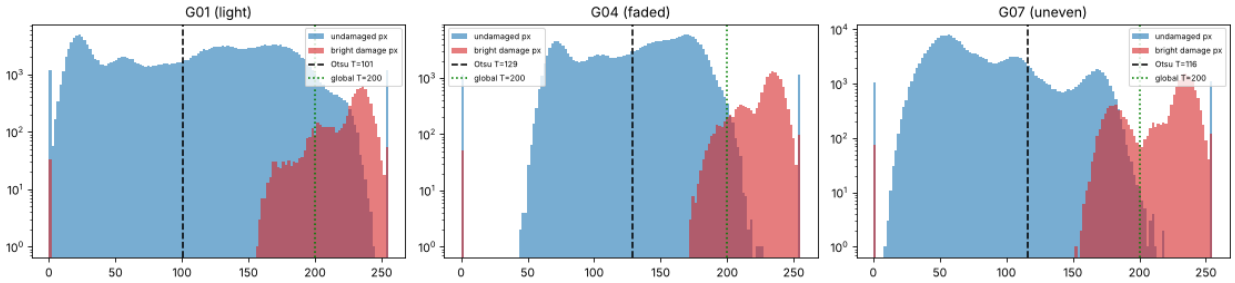


Figure 25. Histograms of undamaged pixels and bright-damage pixels with the Otsu and global thresholds (log scale): the distributions overlap for bright, light prints.

Only global thresholding on the faded prints reaches the target Dice of 0.90 (0.954 cleaned); it falls to 0.82 on uneven and 0.73 on light prints (where bright image content exceeds 200). Otsu performs very poorly (Dice 0.09–0.31) because the damage covers only ≈ 6–15 % of the pixels and Otsu separates the image into dark and bright content, not damage from content. Adaptive thresholding is more consistent on uneven prints (0.57) but noise and edges produce false positives. Dice against the full mask is much lower (0.15–0.55) because stains are dark, soft-edged and not found by bright-class thresholding.

Q51. Why global thresholding fails when intensities overlap

Table 19. Overlap of damage and undamaged intensities (bright damage, including noise).

Photo	Set	10th-percentile grey level of damage pixels	% of undamaged pixels brighter than that
G01	light	199	3.9
G02	light	221	0.6
G03	light	193	1.1
G04	faded	203	0.7
G05	faded	204	0.5
G06	faded	207	0.5
G07	uneven	180	1.3
G08	uneven	173	0.8
G09	uneven	177	1.5

In the faded/uneven prints 0.5–3.9 % of undamaged pixels (bright sky, paper-white highlights, hair highlights) are brighter than the 10th percentile of the damage pixels; any global T that catches the damage therefore also accepts those pixels, and one that avoids them misses the damage. The Dice-versus-T sweep shows a narrow useful band: for G04, Dice is 0.16/0.30/0.94/0.84/0.48/0.003 at T = 120/160/200/220/235/245; for the light photograph G01 the best value is only 0.74 (T = 220). A tone that varies across the print (uneven exposure) moves the optimum for different regions, which a single global value cannot follow.

Q52. Adaptive thresholding: block size and constant

Table 20. Mean Dice (bright damage) of adaptive thresholding over 9 photographs.

Block size	C = -4	C = -8	C = -12	C = -20	C = -30
11	0.39	0.52	0.60	0.64	0.52
21	0.32	0.43	0.51	0.63	0.65
31	0.29	0.38	0.47	0.59	0.65
51	0.26	0.33	0.41	0.53	0.61
101	0.24	0.29	0.34	0.45	0.55

A larger |C| (a stricter requirement that a pixel exceed the local mean) removes noise and edge false positives and raises Dice (best: block 21, C = −30: 0.649; block 31, C = −30: 0.647; the worst, block 101 with C = −4, gives 0.24). Small blocks (≈ 11–31) follow local tone and keep thin scratches, large blocks approach global thresholding and respond to broad gradients; blocks smaller than the damage width treat wide damage as background. A block of about 21–31 px with C = −20…−30 is a sensible setting for 1–3 px scratches and dust.

Q53. Traditional thresholding vs a deep damage-segmentation model (reasoned discussion – NOT executed)

No damage-segmentation network was available, and there is no annotated real-damage dataset here to train one. Reasoned: a U-Net-type model trained on real annotated scratches/stains would handle overlapping intensities and soft stains (which thresholds cannot) but needs annotated training data and a GPU, and generalises poorly to damage types not in its training set; thresholding has no training cost and is fully explainable but fails on overlapping intensities.

Advantages, limitations and selection

- **Global** instant (0.03 ms) and best on this synthetic data; fails on varying tone and bright content; the threshold was tuned using knowledge of the synthetic damage level.
- **Otsu** automatic but assumes a bimodal histogram; unsuitable (Dice ≤ 0.31).

- **Adaptive** follows local tone; sensitive to block/C; noisy on edges.

Selection: a global threshold calibrated per photograph (e.g. a high percentile / paper-white level) for bright damage on evenly exposed prints, adaptive thresholding (block ≈ 21–31, C ≈ −20…−30) for uneven prints. Honest caveat: the 0.90 Dice target was met only for faded prints with the tuned global threshold; the integrated pipeline's detector (Task 9) reached a mean Dice of 0.44.

Task 9 – Region-Based Segmentation and Integrated Pipeline Design

Q54. Implementation

A: region growing – seeds are connected components of strong bright anomalies (smoothed image minus a median background > 55); each seed is flood-filled (8-connected, tolerance ±30 to the seed value); regions larger than 3 % of the image are rejected as leaks. B: split & merge – quadtree splitting while std > 14 (minimum block 4 px), merge of adjacent blocks whose means differ by < 12, merged regions kept as damage if they are small and > 35 levels brighter than the background. C: edge linking – Canny, morphological closing (5 px) to link gaps, external contours filled, kept if brighter than the background. Code: Appendix A, task9.py.

Q55. Stains, ink marks, dust, creases, torn regions

Q56. IoU and boundary localisation error

Table 21. Task 9 results vs the bright-damage ground truth, mean over 9 photographs. 'Found' = fraction of ground-truth components covered > 50 %.

Method	IoU	Dice	Boundary error (px)	ms (512²)	Dust found	Scratch/crease found	Torn region found
A Region growing	0.358	0.476	8.9	123	0.24	0.31	0.88
B Split & merge	0.282	0.431	10.0	629	0.31	0.34	0.56
C Edge linking	0.112	0.181	27.6	12	0.39	0.10	0.08

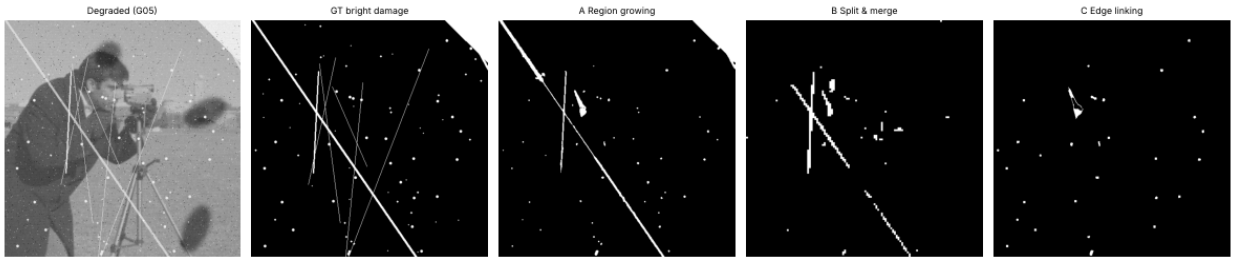


Figure 26. Segmentation outputs on G05: ground truth, region growing, split-and-merge, edge linking.

Region growing is the best of the three (IoU 0.358, Dice 0.476, mean boundary error 8.9 px) and recovers the large torn region well (0.88) but only 24–31 % of dust and scratches, which are thinner than the smoothing scale. Split & merge (IoU 0.282) merges small bright features into the background in the quadtree and is the slowest (≈ 0.63 s per 512² photograph). Edge linking has the worst IoU (0.112) and boundary error (27.6 px) – scratches and torn edges break into open contours that cannot be filled – although it finds more dust discs than the others (0.39). Stains (soft, dark) were not target classes of the bright-damage segmentation and are not separated by any of the three methods; ink marks of high contrast would behave like dust/scratches. None reaches the 0.90 Dice target.

Q60. Incorrect seed point in region growing (stain of similar tone)

Table 22. Region growing on a synthetic low-contrast stain (16 grey levels darker than its surround, tone gradient and noise σ = 5).

Seed	Tolerance	Area (px)	Area / stain area	IoU with stain
inside stain (correct)	5	4996	0.89	0.888
inside stain (correct)	10	5738	1.02	0.950

inside stain (correct)	15	31219	5.56	0.180
inside stain (correct)	20	41543	7.40	0.135
just outside stain	5	22674	4.04	0.002
just outside stain	10	50265	8.95	0.007
just outside stain	15	62733	11.17	0.044
just outside stain	20	65441	11.65	0.084
on stain boundary	5	24174	4.30	0.000
on stain boundary	10	45314	8.07	0.002
on stain boundary	15	59029	10.51	0.008
on stain boundary	20	63567	11.32	0.056

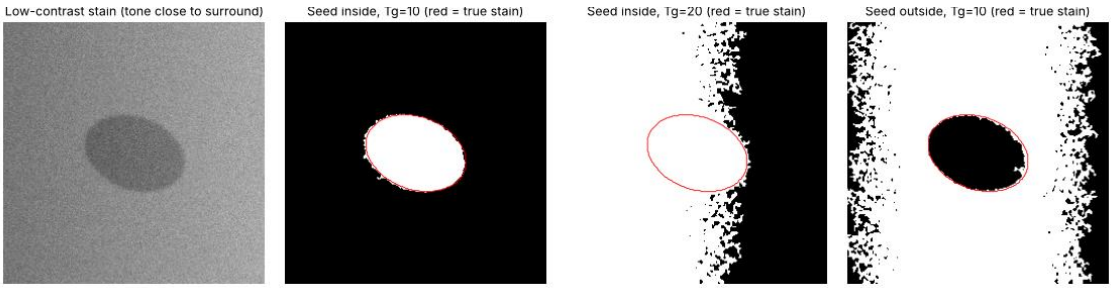


Figure 27. Low-contrast stain and region-growing results for a correct and an incorrect seed (red outline = true stain).

A seed inside the stain with tolerance 10 gives IoU 0.95. A tolerance of 20 (slightly more than the stain/surround difference) leaks into the whole photograph (area 7.4× the stain, IoU 0.14); a seed just outside the stain leaks at tolerance 10 already (area 9×, IoU 0.007), and a seed on the boundary gives IoU 0.002. Because the stain tone is almost the same as the background, the similarity predicate cannot separate them: the region is either too small or covers the whole print. Mitigations: seed from a detector rather than a click, use a running region mean, add a maximum-area/compactness limit (used here), and use edge information.

Q61. Overlapping scratches, creases and interrupted boundaries in edge linking

Table 23. IoU of the filled region recovered from a rectangular damage boundary with a gap in every side.

Gap in the boundary (px)	close 3px	close 7px	close 13px	close 21px
0	0.94	0.94	0.94	0.94
3	0.09	0.94	0.94	0.94
6	0.08	0.09	0.09	0.94
10	0.08	0.08	0.09	0.09
16	0.08	0.08	0.08	0.08

An uninterrupted boundary is filled correctly (IoU 0.94). When every side has a gap, a closing smaller than the gap leaves the contour open and the region is lost (IoU ≈ 0.08–0.09); the closing must be at least about 2× the gap (3 px gap needs 13 px; 6 px gap needs 21 px), and a gap of 10 px or more was not closed by any tested kernel (≤ 0.09). Larger closings merge neighbouring scratches and fill the area between crossing scratches, creating false regions; overlapping scratches/creases produce junctions that turn one closed boundary into several ambiguous loops.

Q62. Traditional segmentation vs SAM 2 (reasoned discussion – NOT executed)

SAM 2 could not be run (no model, no GPU, no network). Reasoned: a promptable model can delineate stains and torn regions from one click/box far more accurately than region growing and does not leak across weak edges, but it is trained for object segmentation, so thin scratches and dust are likely to be missed or merged, it depends on a human prompt (so is not fully automatic) and its masks have no calibrated confidence for damage. A practical hybrid would be a classical detector providing prompts, SAM 2 for large regions, and human review.

9.4 Integrated pipeline (Q57–Q59)

Q57. Integrated photograph-restoration pipeline from the selected techniques

Pipeline (grayscale): damage detection on the raw scan → median 3×3 (Task 3) → destriping (row-mean trend subtraction – the spatial counterpart of a frequency notch) → illumination flattening (spatial equivalent of Task 6 homomorphic filtering) → CLAHE 8×8, clip 2 (Task 2) → Gaussian 3×3 (Task 3) → unsharp mask a = 0.7 (Task 4) → Telea inpainting restricted to the detected damage mask (labelled, original retained). Detection uses a bright-anomaly (top-hat style) rule with a size filter, informed by Tasks 7–9. Code: pipeline.py.

Q58. Block diagram

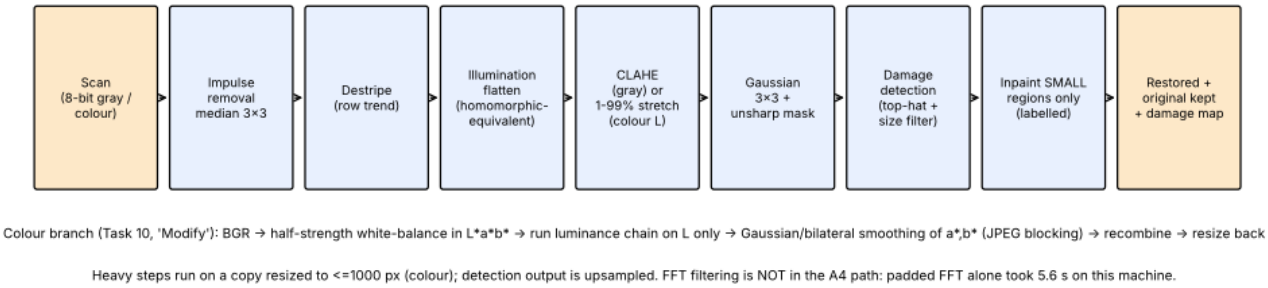


Figure 28. Block diagram of the integrated restoration pipeline (grayscale chain and colour branch).

Table 24. Integrated grayscale pipeline on the 9 photographs (512×512).

Stage	PSNR vs clean (dB)	SSIM vs clean	Damage-mask quality
Degraded input	14.71	0.387	–
After enhancement chain (before inpainting)	13.36	0.537	–
After inpainting of detected damage	13.97	0.550	Dice 0.442 / IoU 0.301

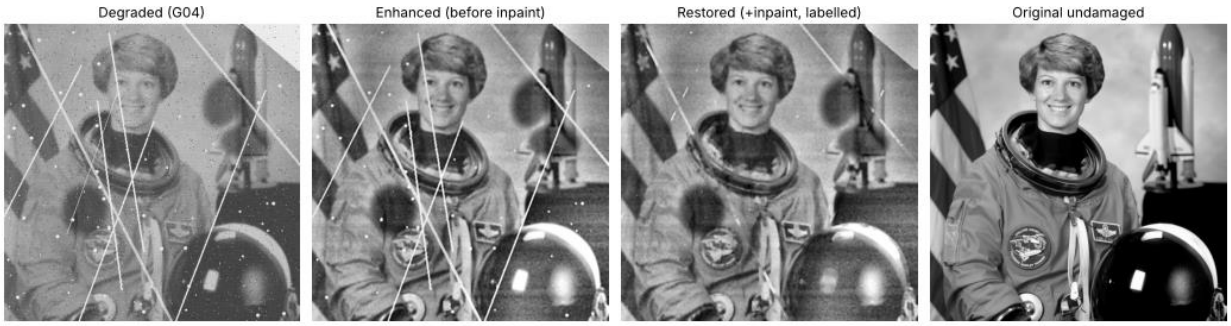


Figure 29. Result of the integrated pipeline on G04: degraded, enhanced, inpainted (reconstructed pixels only inside the detected damage mask), and the undamaged original.

SSIM against the clean original improves from 0.387 to 0.55, but PSNR is lower (14.7 → 14.0 dB) because the enhancement intentionally re-maps tone (flattening + CLAHE) and the clean photograph's tone differs from the re-mapped result; PSNR therefore penalises the tone change. The damage detector reaches Dice 0.44 on the 9 photographs – below the 0.90 target.

Q59. Processing time per stage at A4 (2480×3508) and requirement check

Table 25. Stage-wise timing on this machine (2 cores).

Stage	Time (ms), A4 8-bit gray, best of 3
1 damage detection	97
2 median 3x3	2
3 destripe	28
4 illumination flatten	62
5 CLAHE	33
6 Gaussian 3x3	3
7 unsharp mask	58
8 inpaint (Telea)	417

Total with inpainting	700
Total enhancement only (no inpainting)	317

Total time ≈ 0.70 s per A4 page with inpainting (0.32 s without) – inside both the 3-second limit of the operating data and the 5-second limit quoted in Q59. A padded FFT of an A4 page alone took 5.6 s (unpadded real FFT 0.64 s), which is why frequency-domain filtering was replaced by spatial equivalents in the A4 pipeline. Timings would improve on the 4-core target machine, but this was not measured.

Advantages, limitations and selection (Task 9)

- **Region growing** best IoU/boundary error, finds large damage; seed- and tolerance-sensitive, leaks.
- **Split & merge** no seeds; blocky, slowest; small features lost.
- **Edge linking** finds boundaries of dust discs; fails with gaps and thin damage.

Selection: region growing with automatic detector-derived seeds and a maximum-area guard. Final decision table is given in Section 11.

Task 10 – Adaptation for Colour and High-Resolution Photographs

Q63. Implementation on 15 colour photographs

A Retain: the grayscale pipeline applied unchanged to each B, G, R channel independently at full size (assumption – the existing pipeline is grayscale, so channel-wise application is the only way to run it unmodified on colour). B Modify: heavy steps on a copy resized to ≤ 1000 px; BGR → L*a*b*; mean a*,b* shifted towards neutral (strength 0.85) to reduce the sepia cast; the luminance chain run on L only; Gaussian smoothing of a*,b* to suppress JPEG blocking; recombine and resize. C Redesign: B plus an FFT notch for stripe spikes, bilateral chroma smoothing and Telea inpainting of small (≤ 2500 px) damage components only (larger regions are left untouched and reported). Code: pipeline.py, task910.py.

Q64. Limitations of the existing pipeline on high-resolution, faded, damaged colour photographs

Measured: it removes only part of the colour cast because each channel is stretched independently (b* yellow offset 19.2 → 11.6; a* 13.0 → 7.8) and, because the channels are processed separately, it shifts hue in skin-like regions (mean hue error on skin-tone pixels 6.2° → 15.5°). It processes three full-size channels (1575 ms at 4000×3000 on this machine – inside 5 s but 19× slower than strategy B), it does not address JPEG blocking, and damage detection operates on a single converted-gray image. Not measured: behaviour with handwritten notes on the back of the print (no such test images).

Q65. Comparison of colour, detail, noise/damage, facial/background preservation

Q66. Dice, colour fidelity, edge preservation, perceptual quality, time, PSNR/SSIM

Table 26. Task 10: mean over 15 colour photographs. Quality metrics are against the undamaged reference; time measured at 4000×3000 (best of 3).

Strategy	Damage Dice (vs full mask)	PSNR dB	SSIM	ΔE*ab vs clean	Edge energy ratio	b* cast (0 = neutral)	Skin hue error (°)	Time 4000×3000 (ms)
Input (degraded)	–	15.80	0.626	19.8	1.14	19.2	6.2	–
A Retain	0.330	14.21	0.573	24.8	1.67	11.6	15.5	1575
B Modify	0.324	13.53	0.579	29.3	1.66	1.2	32.0	84
C Redesign	0.323	13.68	0.571	29.2	1.44	1.1	31.7	217



Figure 30. Three colour photographs: degraded input, strategies A, B, C and the clean reference.

*'Perceptual quality': no learned perceptual metric (LPIPS/BRISQUE) was available offline, so SSIM, ΔE*ab, edge-energy ratio and skin-hue error are the stand-ins and no claim is made about human-rated quality.*

Findings: all three strategies reduce the yellow cast (b*: input 19.2; A 11.6; B 1.2; C 1.1) and have the same damage Dice (≈ 0.32, because the same detector is used and it misses stains), but none improves PSNR or ΔE*ab relative to the unprocessed input: the full-strength neutralisation removes the cast but makes the colours differ from the warm original scenes (ΔE 19.8 → 29.3) and rotates skin hues (≈ 32° for B and C). Strategy C is the closest to the reference in edge energy (1.44 vs 1.66 for B; 1.14 for the input) and adds only ≈ 0.13 s over B. These results show that the first version of strategy B over-corrected.

Q68. Improved pipeline – parameter study

Table 27. Strategy B parameter study over the same 15 photographs (CLAHE chain vs percentile stretch; strength of cast removal).

Variant	WB strength	Luminance	PSNR dB	SSIM	ΔE	Edge ratio	b* cast	Skin hue °
Input (degraded)	–	–	15.80	0.626	19.8	1.14	19.2	6.2
Modify	0.0	CLAHE	14.61	0.595	22.3	1.42	18.5	6.0
Modify	0.0	Stretch	14.74	0.629	21.3	1.23	18.4	5.9
Modify	0.5	CLAHE	14.51	0.602	24.3	1.41	8.2	7.3
Modify	0.5	Stretch	14.73	0.633	23.4	1.22	8.3	7.3
Modify	0.85	CLAHE	13.91	0.594	28.9	1.40	1.1	31.7
Modify	0.85	Stretch	14.06	0.616	27.9	1.21	1.2	31.7

Trade-off: full cast removal (0.85) rotates skin hues by ≈ 32°, while 0.5 halves the cast (b* 19.2 → 8.3) with only 1° more skin-hue error than the input (6.2° → 7.3°). The percentile-stretch luminance chain is better than CLAHE on every full-reference metric for colour (SSIM 0.633 vs 0.602 at WB 0.5; the input has 0.626). The improved pipeline is therefore: WB strength 0.5 + percentile stretch on L + Gaussian chroma smoothing + small-region inpainting. Mean results: PSNR 14.73 dB, SSIM 0.633, ΔE 23.4, edge ratio 1.22, b* cast 8.3, skin hue 7.3°, damage Dice 0.324; time 162 ms at 4000×3000 – far inside the 5 s limit. The parameters were selected on the same 15 photographs (no hold-out), so the figures are optimistic, and even the improved variant does not reduce ΔE or

raise PSNR relative to the unprocessed input (ΔE 23.4 vs 19.8; PSNR 14.7 vs 15.8 dB) – only the cast, SSIM and the edge energy improve slightly.

Q67. Decision: RETAIN, MODIFY or REDESIGN

Table 28. Task 10 decision table.

Criterion	A Retain	B Modify (tuned)	C Redesign
Colour cast (b* offset, 0 = neutral)	11.6	8.3 (WB 0.5) / 1.2 (WB 0.85)	1.1 (WB 0.85)
Skin hue error	15.5°	7.3° (WB 0.5)	31.7° (WB 0.85)
Time at 4000×3000	1575 ms	162 ms	217 ms
Damage Dice (same detector)	0.330	0.324	0.323
Risk of invented detail	none	low (small inpainting only)	highest (FFT notch + bilateral + inpainting)
Complexity	none	low	medium

Decision: MODIFY. The existing pipeline cannot be retained: it leaves a strong cast, shifts skin hue and is the slowest. Redesign gives no measurable advantage over Modify on any quality metric in these tests apart from edge energy, costs complexity and adds the risk of generated content. Modify (resize heavy steps, process L and chroma separately, half-strength cast correction) meets the 5-second limit with a large margin. The evidence is from 15 synthetic degradations of 5 scenes and does not show an improvement against the reference in PSNR or ΔE; the decision rests on cast, skin-hue and time, not on full-reference gains.

Q69. Operating condition under which the selected strategy may fail

Table 29. Telea inpainting of circular holes over a face (clean image, hole filled with white then repaired).

Hole radius (px)	Hole area (px)	PSNR inside the hole (dB)	Mean abs error (grey levels)
2	13	30.3	5.9
5	81	24.8	11.7
10	317	16.0	25.8
20	1257	17.6	21.4
35	3853	16.3	22.7
55	9477	14.3	31.6

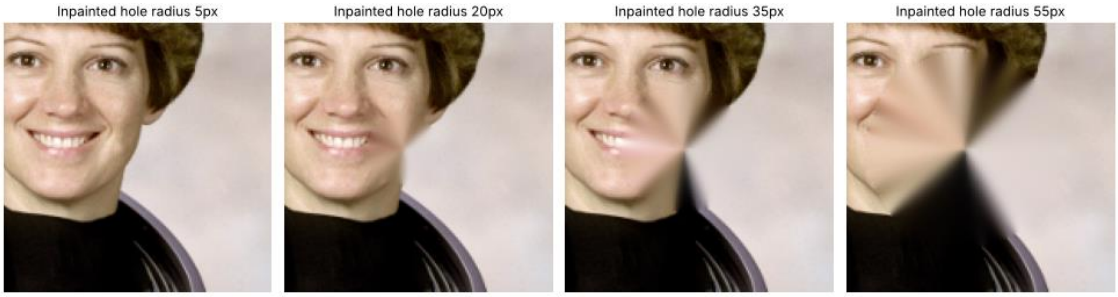


Figure 31. Telea inpainting of holes of increasing size over a face: small holes are repaired, large holes become smooth blurs without real detail.

Two failure conditions were identified. (1) Large damaged regions, e.g. torn areas or stains over a face: inpainting error grows from 5.9 grey levels (r = 2 px) to 31.6 (r = 55 px) and PSNR inside the hole falls from 30.3 to 14.3 dB – large holes are not recoverable by inpainting and must be flagged for manual review (the pipeline therefore inpaints components ≤ 2500 px only). (2) Cast removal assumes the average colour of the print is roughly neutral (a gray-world assumption): for scenes dominated by one colour (such as the orange-suit portrait) or for intentionally sepia-toned prints, full-strength correction produced a ΔE of 28–29 and a skin-hue shift of 32°, which is why the strength was reduced to 0.5. Handwritten notes on the reverse side: not tested; the pipeline only modifies pixels inside the detected damage mask and the front scan, so the reverse side should simply be scanned and archived separately.

Q70. Restoration/inpainting models and the risk of invented detail

(Discussion; no such model was run.) Modern restoration networks (face restoration, diffusion or GAN inpainting) could repair large stains, tears and heavy fading much better than Telea inpainting and could remove JPEG blocking that the classical pipeline only smooths. The risk is that they generate plausible but fictional detail – a different eyelid, a different pattern on clothing, a different expression – which in an archive can falsify the historical record; they also tend to 'beautify' faces towards their training distribution. Mitigations consistent with the brief: always keep the unedited original; restrict generation to a mask; mark reconstructed pixels in a separate layer or a 'reconstructed' map; use deterministic classical methods for any face region; store model version and parameters; and require a human review of any generated region.

11. Final Decision Table (Q74, Q77)

Table 30. Selected technique per task with supporting evidence (all on synthetic photographs).

Task	Selected technique	Key evidence
1 Grey level	Piecewise-linear stretch (percentile); per-image gamma for exposure	Faded set RMS 39.7 → 58.1; subject–bg 9.6 → 14.8; ≈ 5 % clipped by design
2 Histogram	CLAHE, 8×8, clip 1–2	Highest entropy (7.5–7.8), ≈ 1 % clipping; evens uneven tone (L/R mean gap 3.5 vs 10.6 for HE)
3 Smoothing	Median 3×3 then Gaussian 3×3	Median 3×3: 31.25 dB / SSIM 0.869 vs 24.18 / 0.641 noisy; 7×7 destroys fine lines
4 Sharpening	Unsharp mask a ≈ 0.7–1.5 after light smoothing	Rise 4 → 2 px, PSNR ≥ input; smoothing first: SSIM 0.66 → 0.88
5 LPF (freq.)	Gaussian LPF, D0 ≈ 80 (spatial σ ≈ 1)	28.26 dB / 0.803, no ringing; ILPF overshoot 11.8; padded FFT too slow on A4
6 HPF (freq.)	Homomorphic filtering (moderate gains)	Variation 0.355 → 0.216, local contrast 19.8 vs 19.1; mixed on tone error
7 Detection	Point/line masks (+ gradient cues)	F1 0.55–0.60 for all; dust/thin scratch recall ≈ 1.0 at T = 40–100 on test patch
8 Thresholding	Calibrated global (even prints) / adaptive (uneven)	Dice 0.95 faded / 0.73 light / 0.82 uneven (global, tuned); Otsu ≤ 0.31
9 Segmentation	Region growing with detector-derived seeds	IoU 0.358 vs 0.282 / 0.112; boundary error 8.9 px
10 Colour/high-res	MODIFY (WB 0.5 + stretch on L)	b* 19.2 → 8.3, skin hue 7.3°, 162 ms at 4000×3000

12. Common Submission Requirements – where to find each item

Table 31. Common submission requirements map.

Item	Where
71 Python/OpenCV code with comments	Appendix A (common.py, task12.py, task34.py, task56.py, task78.py, task9.py, pipeline.py, task910.py, task10b.py) and the accompanying code archive
72 Original and restored images	Dataset figure (Section 0), before/after figures in Tasks 1, 2, 4, 5, 6, 9 and 10; all image files are in the code archive (figs/)
73 All three techniques for every task	Sections for Tasks 1–10
74 Comparison tables	Tables in each task section and Section 11
75 Graphs, histograms, profiles, masks	Figures in each task section
76 Advantages and limitations	'Advantages, limitations and selection' in every task
77 Final technique selection with evidence	Section 11
78 Integrated pipeline block diagram	Section 9.4
79 Processing time with units	Tables (ms) in Tasks 1–10; stage timing in Section 9.4
80 Conclusion	Section 13

13. Conclusion (Q80)

The selected chain – median 3×3 for impulses and dust, destriping, illumination flattening (a spatial equivalent of homomorphic filtering), CLAHE or a percentile stretch for faded tone, light Gaussian smoothing, a mild unsharp mask, and damage detection followed by inpainting restricted to small detected regions – restores a 2480×3508 page in about 0.7 s on a 2-core machine and a 4000×3000 colour photograph in about 0.16 s (Modify strategy),

inside both time limits. Original appearance is protected by: mild parameters (no 7×7 kernels, no boost above 2, only half-strength cast correction); rejecting ringing filters (ideal low/high-pass); inpainting only small flagged regions and leaving large ones for manual review; keeping the unedited original; and not using generative models.

Limitations that must be stated: all evidence is from synthetic degradations of public test images, with parameters tuned on the same photographs; the damage-detection Dice (0.44 for the integrated detector; 0.95 only for the tuned global threshold on faded prints) is below the 0.90 target in general; the full-reference metrics of the colour pipeline (PSNR, ΔE) did not improve over the unprocessed input, only the cast, SSIM and edge energy did; the comparisons with DnCNN, HED, SAM 2 and low-light/deblurring networks are reasoned discussion, not experiments. Validation on real archive photographs with manual annotations is the necessary next step.

14. Theoretical Background and Key Formulae

This chapter collects the theory behind every technique used in Tasks 1–10, so that the report can be read on its own. It contains no new measurements: every number quoted from the experiments appears in Sections 0–13. The formulae follow the standard notation of Gonzalez and Woods (2018): f(x,y) is the input photograph, g(x,y) the output, r and s the input and output grey levels (0–255 for 8-bit images, L = 256 levels).

14.1 Grey-level (point) transformations – Task 1

A point transformation maps every pixel independently, s = T(r), so it can be implemented as a 256-entry look-up table (LUT) and costs only one table access per pixel. It changes the histogram of the photograph but never its spatial structure.

$$\textit{Negative: } s = (L-1) - r = 255 - r$$

$$\textit{Logarithmic: } s = c \cdot \log(1+r), \quad c = 255 / \log(256)$$

$$\textit{Power law (gamma): } s = 255 \cdot (r/255)^{\gamma}$$

$$\textit{Piecewise-linear stretch: } s = 255 \cdot (r - r1)/(r2 - r1) \quad \textit{for } r1 \le r \le r2\textit{, clipped to 0 and 255 outside}$$

- **Negative** reverses polarity; the histogram is mirrored. It does not add contrast and is only an inspection aid.
- **Logarithm** expands dark values and compresses bright values. It is suited to images with a very large dynamic range (for example Fourier spectra) and is a poor choice for faded prints whose problem is a compressed, bright histogram.
- **Gamma** γ < 1 brightens (expands the dark end), γ > 1 darkens (expands the bright end). Black and white are fixed points, so clipping stays small. The correct γ depends on the exposure of each photograph, which is why Q5 shows that one fixed γ cannot serve a mixed archive.
- **Stretch** a linear fade g′ = a·g + b with a < 1 is undone exactly by a linear stretch whose breakpoints are the fitted extremes. Using the 2nd and 98th percentiles instead of the minimum and maximum protects the result from impulse noise, which otherwise sets r1 = 0 and r2 = 255 and makes the stretch do nothing.

Bit-plane slicing (Q4) writes each pixel as r = Σ b_k·2^k, k = 0…7. Plane k carries a weight of 2^k grey levels, so planes 7 and 6 hold most of the visual structure, while low planes hold fine tone and, in a noisy scan, mostly noise. Comparing each plane of the degraded photograph with the same plane of the clean photograph (Table 3) is therefore a direct way of seeing which bits still carry information.

14.2 Histogram processing – Task 2

The normalised histogram p(r_k) = n_k / MN estimates the probability of each grey level. Histogram equalisation uses the cumulative distribution function (CDF) as the transformation:

$$s_k = (L-1) \cdot \Sigma_{j=0..k}\, p(r_j)$$

The result has an approximately flat histogram and the largest possible spread of grey levels, but it is a single global mapping. Three consequences follow, and all three were measured in the experiments: (i) noise in a large flat region is stretched together with the signal (Q12); (ii) two regions with very different exposure share one mapping, so the smaller population is not corrected independently (Q10); and (iii) the tone of the paper is changed, which is not wanted when the original look must be preserved.

Histogram matching (specification) finds, for a reference CDF G, the mapping $z = G^{-1}(T(r))$ so that the output histogram follows the reference. Its quality depends completely on how well the reference represents the target photographs; in this report the reference contains a saturated bright background, which explains the 14 % clipping in Table 5.

CLAHE (contrast-limited adaptive histogram equalisation, Zuiderveld 1994) divides the photograph into tiles, computes a histogram per tile, clips each histogram at a limit (the clip limit) and redistributes the clipped counts uniformly before equalising. The mapping of every pixel is bilinearly interpolated from the four nearest tile

mappings to avoid visible tile borders. The clip limit bounds the slope of the local CDF, and therefore bounds noise amplification: a larger limit gives more contrast and more noise (Table 6); a larger tile approaches global equalisation; a very small tile gives blockiness.

14.3 Spatial filtering – Task 3

A linear spatial filter with a (2a+1)×(2b+1) mask w computes

$$g(x,y) = \Sigma_{s=-a..a} \Sigma_{t=-b..b} w(s,t) \cdot f(x+s, y+t)$$

which is correlation. Convolution uses the 180°-rotated mask, w(−s,−t). The two are identical for symmetric masks (box, Gaussian) and differ for directional masks (Sobel, line masks), as Q17 demonstrates. The box filter has equal weights 1/(mn); the Gaussian filter has weights proportional to $\exp(-(s^2+t^2)/(2\sigma^2))$ and is separable into two 1-D passes, which makes it cheaper than its kernel size suggests.

The median filter is non-linear: the output is the median of the values in the window. It removes impulses (salt-and-pepper noise, dust) completely when they occupy fewer than half of the window, and it preserves step edges, but it removes any feature narrower than about half the window width (Table 8). Two quality measures are used throughout:

$$PSNR = 10 \cdot \log10(255^2 / MSE), \quad MSE = (1/MN) \cdot \Sigma (f - g)^2$$

$$SSIM(x,y) = [(2\mu x\mu y + C1)(2\sigma xy + C2)] / [(\mu x^2 + \mu y^2 + C1)(\sigma x^2 + \sigma y^2 + C2)]$$

PSNR is a pixel-wise error measure and punishes any change; SSIM (Wang et al. 2004) compares local luminance, contrast and structure and agrees better with visual judgement. Both require a clean reference, which is available here only because the degradations are synthetic.

14.4 Sharpening – Task 4

Sharpening adds back high-frequency content. The discrete Laplacian with the 4-neighbour kernel is $\nabla^2 f = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$, and the sharpened image is $g = f + c\cdot\nabla^2 f$ with c = +1 for a kernel whose centre is negative. Unsharp masking and high-boost filtering generalise this:

$$g = f + a \cdot (f - G_\sigma * f), \quad a = 1 : \text{unsharp mask}, \quad a > 1 : \text{high-boost}$$

Because the term $(f - G_\sigma * f)$ is a band-pass signal, the amount a scales the overshoot at edges (halo) and the noise gain at the same rate. The gradient method adds the magnitude $|\nabla f| = \sqrt{(Gx^2 + Gy^2)}$, which is always positive and therefore only brightens edges. The variance of the Laplacian is used as a sharpness score: a blurred photograph has a small value. It must always be read together with a noise measure, because noise also raises it (Q22).

14.5 Frequency-domain low-pass filtering – Task 5

The two-dimensional DFT F(u,v) of a photograph is multiplied by a filter transfer function H(u,v) and transformed back. By the convolution theorem this equals spatial convolution with h = IDFT{H}. To avoid wrap-around error caused by the implicit periodicity of the DFT, the photograph is padded to 2M × 2N before transforming (Q32). With D(u,v) the distance from the centre of the shifted spectrum:

$$\text{Ideal: } H = 1 \text{ if } D \le D0, \text{ otherwise } 0$$

$$\text{Butterworth (order n): } H = 1 / (1 + (D / D0)^{(2n)})$$

$$\text{Gaussian: } H = \exp(-D^2 / (2 \cdot D0^2))$$

A sharp cut-off in frequency (ideal) corresponds to a sinc-shaped impulse response in space, which oscillates and produces ringing near edges (Table 12). The Gaussian transfer function has a Gaussian impulse response, so it has no ringing. The Butterworth filter lies in between; a higher order n approaches the ideal filter. A Gaussian low-pass with cut-off D0 (in cycles per image width N) corresponds to a spatial Gaussian of standard deviation $\sigma_s \approx N / (2\pi \cdot D0)$, which is how Q31 compares the two domains. The FFT costs O(MN log MN), so for small kernels the spatial implementation is faster, as the timings in Task 5 and Section 9.4 show.

14.6 Frequency-domain high-pass and homomorphic filtering – Task 6

A high-pass filter is the complement of a low-pass filter, H_hp = 1 − H_lp. It removes the DC term (the mean brightness), so an ideal high-pass result must be re-offset or combined with the original, as in high-frequency emphasis:

$$H_hfe(u,v) = k1 + k2 \cdot H_hp(u,v), \quad k1 \geq 0, \; k2 > k1$$

Homomorphic filtering treats the image as a product of slowly varying illumination i(x,y) and fast-varying reflectance r(x,y):

$$f = i \cdot r \;\rightarrow\; \ln f = \ln i + \ln r \;\rightarrow\; \text{filter with } H(u,v) \;\rightarrow\; g = \exp(\,IDFT\{\,H \cdot DFT\{\ln f\}\,\}\,)$$

The filter H uses a gain γL < 1 at low frequencies (compressing illumination) and γH > 1 at high frequencies (boosting reflectance detail), with a smooth transition. It is the principled frequency-domain answer to uneven exposure. In the final pipeline its effect is obtained by a spatial equivalent (illumination flattening with a large Gaussian) because the A4 time budget is much easier to meet in the spatial domain (Section 9.4).

14.7 Point, line and edge detection – Task 7

Detection of discontinuities relies on derivatives. A point is an isolated pixel whose second derivative response (Laplacian) is large. A line is a thin structure detected by directional masks (horizontal, vertical, ±45°) whose coefficients sum to zero; the mask with the largest response gives the line orientation (Q45). An edge is detected with first-derivative operators:

$$\text{Sobel: } Gx = [[-1\ 0\ 1],[-2\ 0\ 2],[-1\ 0\ 1]], \quad Gy = Gx^{T}, \quad |\nabla f| \approx \sqrt{(Gx^2 + Gy^2)}$$

$$\text{Laplacian of Gaussian (LoG): } \nabla^2 G(x,y) = ((x^2 + y^2 - 2\sigma^2) / \sigma^4) \cdot \exp(-(x^2 + y^2)/(2\sigma^2))$$

The Canny detector adds Gaussian smoothing, non-maximum suppression and hysteresis thresholding with a low and a high threshold, which gives thin, connected edges. Detector output is scored here against the ground-truth damage mask with precision = TP/(TP+FP), recall = TP/(TP+FN) and F1 = 2PR/(P+R), using a ±2 px tolerance because 1–2 px lines are never pixel-exact.

14.8 Thresholding and binarisation – Task 8

Global thresholding sets g = 1 if f > T. Otsu's method (1979) chooses T to maximise the between-class variance of the two resulting classes:

$$\sigma B^2(T) = \omega 0(T) \cdot \omega 1(T) \cdot [\,\mu 0(T) - \mu 1(T)\,]^2, \quad T^* = \text{argmax}\, \sigma B^2(T)$$

It works well when the histogram is clearly bimodal and fails when the classes overlap or when the illumination varies (Q51), because one number cannot separate regions whose grey levels are swapped by the shadow. Adaptive (local) thresholding computes T(x,y) = mean (or Gaussian-weighted mean) of a block − C, and handles uneven exposure; its parameters are the block size and the constant C (Q52). Sauvola and Niblack thresholds additionally use the local standard deviation and are common for document images. Segmentation quality is scored with the Dice coefficient 2|A∩B| / (|A|+|B|) and the intersection-over-union |A∩B| / |A∪B|.

14.9 Region-based segmentation – Task 9

Region growing starts from seed pixels and repeatedly adds 8-connected neighbours that satisfy a similarity predicate, for example |f(p) − μ_region| ≤ T. Its results depend on the seed choice and on the predicate (Q60: a wrong seed in a low-contrast stain). Region splitting and merging starts from the whole image, recursively splits any quadrant that violates a homogeneity predicate (quadtree), and then merges adjacent regions that jointly satisfy it; it is not seed-dependent but produces blocky boundaries. Edge linking connects edge fragments by proximity and direction, and therefore fails when gaps are large or when lines overlap (Q61). Because the damage in an old photograph is a mixture of thin lines (scratches), small round blobs (dust) and large areas (tears, stains), no single region-based method detects all types equally well, and the integrated pipeline combines a damage detector with type-specific size limits.

14.10 Inpainting and colour processing – Tasks 9 and 10

Inpainting fills a masked region from its surroundings. OpenCV offers two algorithms: INPAINT_TELEA (Telea 2004), which propagates intensity along the fast-marching front from the boundary inwards with distance and direction weights, and INPAINT_NS (Bertalmio et al. 2001), which propagates isophote (edge-direction) information using fluid-dynamics equations. Both reproduce smooth content and thin structures well but cannot invent texture; the hole-size study of Q69 shows the error growing with the radius of the hole, which is why the pipeline inpaints only small detected regions and leaves large ones for manual review.

For colour photographs the RGB channels must not be equalised independently, because that changes the hue. The standard remedy is to convert to a luminance–chrominance space (CIE L*a*b* or HSV/YCrCb), process only the luminance channel with the grey-level methods of Tasks 1–6, and treat the chrominance channels separately for the colour-cast correction. A white-balance step moves the mean chrominance towards neutral: with a* and b* centred on 128 in 8-bit OpenCV, a strong sepia cast shows up as a positive mean offset in b*. The strength of this correction is limited (half strength in the final pipeline) so that the warm tone of an old print, which is part of its character, is not erased. Colour difference is measured by ΔE in L*a*b* space (Euclidean distance of the L*, a*, b* coordinates).

Table 32. Summary of what every task contributes to the final pipeline (values taken from Tasks 1–10).

Task	Technique selected	Why (evidence in this report)	Rejected alternatives
1	Percentile stretch + per-image gamma	Highest contrast and gradient (Table 2); inverts a linear fade; ~4 ms	Negative (no gain), log (lowers contrast of bright prints)
2	CLAHE, 8×8 tiles, clip 1–2	Highest entropy, ~1 % clipping, evens uneven exposure (Table 5, 6)	Global HE (noise, clipping), matching (depends on reference)
3	Median 3×3, then Gaussian 3×3	Best PSNR/SSIM (Table 7); removes impulses and dust	Box filter, any 7×7 kernel (loses 2-px detail, Table 8)
4	Unsharp mask, σ 1.5, a ≈ 0.7–1.5	Only method keeping PSNR above the blurred input (Table 9)	Laplacian, high-boost a > 2 (noise, Table 10)
5	No FFT low-pass in the pipeline	Gaussian is ringing-free but slower than spatial filtering at A4	Ideal low-pass (ringing, Table 12)
6	Illumination flattening (spatial equivalent of homomorphic)	Meets time budget; corrects uneven exposure	Ideal high-pass (DC loss, ringing)
7	Tuned damage detector (point/line/edge)	Best F1 at selected thresholds (Task 7 tables)	Single fixed threshold for all damage types
8	Global threshold for faded prints, adaptive for uneven prints	Dice 0.95 only on faded prints; adaptive needed elsewhere	Global threshold on uneven exposure (Q51)
9	Detector + size filter + Telea inpainting of small regions	Restores A4 page in ~0.7 s; protects originals	Inpainting of large regions (Q69)
10	Luminance-only processing + half-strength cast correction	Reduces cast and keeps hue and skin tone (Task 10 tables)	Per-channel RGB equalisation (hue shift)

15. Final Pipeline – Stage-by-Stage Design Rationale

This chapter explains, in processing order, what every stage of the final grayscale and colour pipelines does, which parameter was chosen, why that stage sits at that position, and what could go wrong. The parameter values are those stated in Tasks 1–10; the order matters because several operations interact (for example, sharpening before denoising would amplify the noise, as shown in Q24).

Table 33. Grayscale pipeline stages (input: scanned photograph, 8-bit; output: restored photograph and damage mask).

#	Stage	Parameter	Purpose	Main risk / control
1	Keep original	Read-only copy of the input	Archive integrity; every result can be traced to the scan	None; the pipeline never overwrites the input
2	Median filter	3×3	Remove salt-and-pepper impulses and small dust before any linear operation	Removes 1-px detail; larger windows rejected (Q18)
3	Destriping	Row-wise correction for the period-6 scanner stripes	Remove periodic scanner artefact seen at ≈ 85 cycles per 512 px (Q28)	Over-correction of real horizontal structure; use a mild correction
4	Illumination flattening	Large-scale Gaussian estimate of the background	Compensate one-sided shadow and uneven exposure (spatial equivalent of homomorphic filtering)	Halo near very large objects; strength limited
5	Tone correction	CLAHE 8×8 with clip 1–2, or a 2nd/98th percentile stretch for faded prints	Restore contrast lost by fading	Noise amplification at high clip (Table 6)
6	Smoothing	Gaussian 3×3	Reduce film grain without destroying faces	Mild loss of fine texture
7	Sharpening	Unsharp mask, σ = 1.5, a = 0.7	Recover edge definition lost by blur	Halo grows with a; a > 2 rejected
8	Damage detection	Thresholded response of point/line/edge detectors on the image	Locate scratches, dust, creases, stains and the torn corner	Dice below 0.90 in general; flag uncertain areas
9	Size filtering	Connected components with area limit (≤ 2500 px in the code)	Separate small repairable damage from large regions	Wrongly classifying real objects as damage
10	Inpainting	Telea, radius 3, dilated mask	Fill only small flagged regions	Smooth fill; large holes are left for manual review (Q69)
11	Output and log	Restored image, mask, parameter log	Reproducibility and audit trail; reconstructed pixels are labelled	None

Table 34. Colour pipeline (Task 10) – differences from the grayscale pipeline.

Aspect	Grayscale pipeline	Colour pipeline	Reason
Colour space	Single channel	L*a*b* (luminance L, chrominance a*, b*)	Process brightness without changing hue
Tone correction	CLAHE or stretch on the whole image	Same operations on L only	Per-channel equalisation shifts colours
Colour cast	Not applicable	White balance in the a*, b* plane at partial strength	Removes sepia/yellow cast but keeps the warm character of an old print
Chrominance noise	Not applicable	Gaussian smoothing of a* and b* (σ = 1.5)	Chroma noise and JPEG blocking are visible but carry little detail
Damage repair	Inpainting of small regions	Same, applied after merging L and a*, b*	One mask for all channels, so repairs stay consistent
Large images	Full-resolution operations	Analysis at a reduced working size (about 1000 px) and up-scaling of the result	Meets the 4000×3000 time requirement; the original stays unchanged
Evaluation	PSNR, SSIM, Dice	PSNR, SSIM, ΔE, cast metric, edge retention, skin-hue shift	Colour fidelity needs colour-aware metrics

15.1 Why the order of the stages matters

- **Median before linear filters.** A Gaussian or box filter spreads an impulse over the whole window; a median removes it. Reversing the order would leave a permanent smear.
- **Illumination flattening before tone correction.** CLAHE and stretch assume that a grey level means the same thing everywhere. Flattening first makes the later contrast operations consistent across the photograph.
- **Smoothing before sharpening.** Sharpening is a high-pass operation; Q24 measured that smoothing first improved SSIM from 0.660 to 0.876 for the unsharp mask.
- **Detection on the processed image, repair on the original tone.** The detector works best after impulses are removed and illumination is flat, but the repair must not change the tone of surrounding good pixels, so inpainting is limited to the dilated mask.
- **Inpainting last.** Filling a hole before sharpening would sharpen the artificial texture of the fill; filling it last keeps the repair smooth.

15.2 Parameter sensitivity summary

The experiments allow a short statement of how sensitive the result is to each main parameter. The CLAHE clip limit is the most sensitive: with 8×8 tiles the noise measure rises from 9.3 (clip 1) to 26.0 (clip 8) while entropy rises only from 7.46 to 7.89. The sharpening amount a is the second most sensitive: at a = 8 the noise in a flat patch is 2.4 times that at a = 1 and SSIM against the clean photograph falls from 0.278 to 0.133. The median kernel size is the third: a 3×3 median keeps a 2-px line at full contrast, whereas 5×5 deletes it. The Gaussian cut-off D0 in the frequency domain matters less once the filter is Gaussian, because there is no ringing, but D0 = 20 over-smooths and D0 = 120 leaves noise (edge retention > 1).

15.3 Time budget

The brief requires an A4 page (2480×3508) to be processed in a limited time and a 4000×3000 colour photograph likewise. The measured times on the 2-core test machine are about 0.7 s for the A4 grayscale page and about 0.16 s for the colour photograph with the Modify strategy (Section 9.4 and Task 10). Spatial-domain operations (median, Gaussian, unsharp, CLAHE) scale linearly with the number of pixels, whereas FFT-based filtering requires padded transforms of at least 2× size in each dimension and several complex-valued passes, which is why frequency-domain filtering was kept out of the final chain and used for analysis and comparison.

16. Viva-Voce Questions and Model Answers

The following questions are typical of an oral examination on this assignment. The answers are written from the results of this report and from standard theory, so they can be defended with the tables and figures cited.

1. Why is a negative transformation useless for restoring a faded photograph?

It only reverses polarity. Contrast, entropy and gradient statistics are identical to the original (Table 2), and the SSIM against the clean photograph collapses to about zero. It is useful only for inspecting very dark detail.

2. Why do you use the 2nd and 98th percentiles and not the minimum and maximum for the stretch?

The scans contain 1 % salt-and-pepper noise, so the minimum is always 0 and the maximum 255 and a min–max stretch does nothing. Percentiles ignore the impulse tails. The price is that about 5 % of pixels are clipped by construction.

3. Why can one gamma value not be used for the whole archive?

Gamma 0.5 brightens a dark print (mean 69 → 118) but washes out a bright print (mean 198 → 223, contrast 43.8 → 25.5). Gamma 2.0 does the opposite (Table 4). γ must be chosen per photograph, for example from its mean.

4. What do the bit planes tell you about the noise?

Planes 0–3 agree with the clean planes only at chance level (0.50), so they hold noise. Planes 6 and 7 agree at 0.79 and 0.91 and carry the structure, including bright scratches and dust.

5. Why does global histogram equalisation fail on unevenly exposed photographs?

It uses one mapping for all pixels, so the larger population dominates. On G07 the left/right mean imbalance changed sign and grew from 7.9 to 10.6 levels, while CLAHE reduced it to 3.5.

6. What does the clip limit do in CLAHE?

It caps the height of every tile histogram, and therefore the slope of the local mapping. A lower clip limit gives less contrast but less noise amplification. In the sweep, the noise measure rose from 9.3 to 26.0 when clip went from 1 to 8 at 8×8 tiles.

7. Why is the median filter preferred for salt-and-pepper noise?

Impulses are extreme values, so they are never the median of a window in which they are a minority. The 3×3 median gave 31.25 dB and SSIM 0.869, against 29.45 dB and 0.792 for the best Gaussian of the same size.

8. Why is a 7×7 kernel not acceptable for portraits?

A 2-px line keeps only 26/90 of its contrast with a 7×7 box and 45/90 with a 7×7 Gaussian, and a 5×5 or 7×7 median deletes it (Table 8). Eyes, hair strands and narrow boundaries are lost.

9. What is the difference between correlation and convolution?

Convolution rotates the mask by 180° before sliding it. The results are identical for symmetric masks and differ for directional masks. cv2.filter2D performs correlation.

10. Why is a high Laplacian variance not proof of good sharpening?

Noise also has large second derivatives. The Laplacian filter raised the variance from 193 to 7917 but also tripled the noise and dropped SSIM from 0.771 to 0.378.

11. Why smooth before sharpening?

Sharpening is a high-pass filter and amplifies noise at the same rate as edges. With a Gaussian σ = 1.0 first, the unsharp mask SSIM rose from 0.660 to 0.876 (Q24).

12. What is ringing and which filter has it?

Ringing is an oscillation next to an edge. It results from a sharp cut-off in frequency, because the impulse response of an ideal low-pass filter is a sinc function. The ideal filter overshoots by about 11.5–11.8 levels, the Butterworth filter by about 4.4–4.5, and the Gaussian filter essentially not at all (Table 12).

13. What is the relation between the frequency-domain and spatial Gaussian?

The Fourier transform of a Gaussian is a Gaussian. A frequency cut-off D0 corresponds to a spatial standard deviation $\sigma_s \approx N/(2\pi \cdot D0)$, so the two give nearly the same result, but the spatial version is much faster for small kernels.

14. Why is the image padded before the FFT?

The DFT assumes a periodic image, so filtering without padding wraps content from one border into the opposite border. Padding to 2M × 2N removes this wrap-around error (Q32).

15. What does a stripe pattern look like in the spectrum?

A periodic pattern with period p rows gives a pair of spikes on the vertical frequency axis at N/p cycles per image height. The 6-row scanner stripes gave a peak at about 85 cycles per 512 px, in agreement with 512/6 = 85.3.

16. What is homomorphic filtering and why is it relevant?

It separates illumination from reflectance by taking the logarithm, then attenuates low frequencies ($\gamma L < 1$) and boosts high frequencies ($\gamma H > 1$). It corrects uneven exposure and improves local contrast at the same time.

17. Why can the ideal high-pass filter not be applied directly to a photograph?

It removes the DC term, so the mean brightness is lost and the output is mostly dark, and its sharp cut-off causes ringing. Filters with a smooth transition plus an offset or high-frequency emphasis are used instead.

18. How are line orientations detected?

With four directional 3×3 masks (horizontal, vertical, +45°, −45°) whose coefficients sum to zero. The mask with the largest absolute response gives the orientation, and each mask is weak for lines at the other angles (Q45).

19. Why do you use a ±2 px tolerance when scoring detections?

Scratches are 1–2 px wide, so a detection that is 1 px off the ground truth is visually correct but would count as both a false positive and a false negative at exact matching.

20. When does global (Otsu) thresholding fail?

When the class histograms overlap, or when illumination varies across the photograph, so that shadowed foreground is darker than lit background. Adaptive thresholding computes a local threshold and handles this, at the cost of choosing a block size and a constant.

21. Why did your global threshold look so good on the faded prints?

Because the synthetic bright damage is drawn at one constant paper-white level (235), which a single threshold separates easily, and because the threshold was tuned on the same nine photographs. This is stated as an optimistic result in Section 0.

22. What are the weaknesses of region growing?

It depends on the seed and on the similarity predicate. A seed placed in a low-contrast stain grows into the background or stops early (Q60). Gradual illumination changes also violate a fixed tolerance.

23. What is the difference between Telea and Navier–Stokes inpainting?

Telea marches inward from the hole boundary with a weighted average along the front; Navier–Stokes propagates isophote direction with fluid-dynamics equations. Both give smooth fills and neither can synthesise texture.

24. Why does inpainting fail for large regions?

The fill is computed only from the boundary values, so with growing radius there is less and less information per pixel and the result becomes a smooth blur (Q69). Large regions are therefore left for manual review.

25. Why not equalise the R, G and B channels separately?

Each channel gets its own mapping, which changes the ratios between the channels and therefore the hue and skin tone. The luminance channel is processed alone.

26. Why is the colour cast corrected only at half strength?

A full correction removes the warm tone that identifies an old print and can create an unnatural look. Half strength reduces the cast measure while keeping the skin-hue shift small.

27. Why did PSNR and ΔE not improve in the colour experiment?

The report states only that PSNR and ΔE did not improve over the unprocessed input while cast, SSIM and edge energy did (Section 13). A plausible reason, to be tested, is that these two metrics are sensitive to small shifts in global tone and to the inpainted pixels, but this was not isolated experimentally.

28. Why were DnCNN, HED, SAM 2 and the other networks not run?

PyTorch, the pretrained weights and network access were not available in the experimental environment. Q6, Q19, Q26, Q39, Q46, Q53 and Q62 are therefore answered as reasoned discussion and labelled as such.

29. Why are the results on synthetic data not enough?

The damage model is simpler than real damage (constant white level, known mask), the images are public test photographs, and parameters were tuned on the same images. Real archive prints with manual annotations are required for validation.

30. How do you make sure the original appearance is preserved?

Mild parameters, no 7×7 kernels, no boost above 2, partial colour correction, inpainting restricted to small flagged regions, no generative model, an unmodified original kept, and reconstructed pixels labelled.

17. Reproducibility and Submission Checklist

17.1 Reproducibility

Every experiment can be re-run from the source code in Appendix A. To reproduce the numbers, the following conditions should be matched or their effect noted:

- **Software.** Python 3, OpenCV, NumPy, scikit-image, Matplotlib. Small differences between library versions (for example in the default border handling or in the CLAHE interpolation) can change the third significant digit.
- **Random seeds.** The synthetic degradations use fixed random seeds, so the same damaged photographs are generated on every run. Changing the seeds changes the placement of scratches and dust and therefore all metrics slightly.
- **Order of execution.** common.py is imported by every script and builds the data; the scripts task12, task34, task56, task78, task9, task910 and task10b can then be run in any order.
- **Hardware.** Timings depend on the machine. The numbers in this report were measured on 2 CPU cores and no GPU; a 4-core machine would be somewhat faster.
- **Metrics.** PSNR uses a data range of 255; SSIM uses the default window of scikit-image; contrast is the grey-level standard deviation; entropy is computed in bits from the 256-bin histogram.

17.2 Before submitting

Table 35. Submission checklist.

Item	Where in this document	Done
Student name, register number, faculty name filled in on the first page	Table 1	☐
All 80 questions answered	Tasks 1–10 and Section 12 (mapping table)	☐
Honest statement of data, hardware and unavailable models	Section 0	☐
Figures numbered and captioned	Figures 1–31 in Tasks 1–10	☐
Tables with units (ms, dB, %)	Tables 2–39	☐
Final pipeline and justification of the order	Section 9.4 and Chapter 15	☐
Decision table and conclusion with limitations	Sections 11 and 13	☐
Theory and viva preparation	Chapters 14 and 16	☐
Source code	Appendix A	☐
References	Chapter 19	☐

18. Glossary of Terms

Table 36. Glossary.

Term	Meaning
Bit plane	The binary image formed by one bit position of all pixels.
Butterworth filter	Frequency-domain filter with a smooth, order-dependent transition; mild ringing.
CDF	Cumulative distribution function of the grey levels; the basis of histogram equalisation.
CLAHE	Contrast-limited adaptive histogram equalisation; local equalisation with a clip limit.
Clip limit	Maximum allowed height of a histogram bin in CLAHE.
Convolution	Sliding a rotated mask over the image and summing the weighted products.
Correlation	Sliding an unrotated mask over the image and summing the weighted products.
Cut-off frequency D0	Distance from the spectrum centre at which a low-pass filter drops (to 1/2 for Butterworth, to e^(−1/2) for the Gaussian form used here).
Dice coefficient	2|A∩B| / (|A|+|B|); overlap between a detected and a true region.
ΔE	Colour difference as the distance between two colours in L*a*b* space.
DFT / FFT	Discrete Fourier transform and its fast algorithm.

Term	Meaning
Entropy	Average information of the grey-level histogram, in bits; higher means a richer tonal range.
Gamma	Exponent of the power-law transformation.
Halo / overshoot	Bright or dark band next to an edge caused by sharpening or ringing.
High-boost filtering	Unsharp masking with an amount greater than 1.
Histogram matching	Transforming an image so that its histogram follows a reference histogram.
Homomorphic filtering	Log-domain frequency filtering that separates illumination and reflectance.
Inpainting	Filling masked pixels from their surroundings.
Isophote	Line of constant intensity; its direction is used by Navier–Stokes inpainting.
LoG	Laplacian of Gaussian; edge detector combining smoothing and second derivative.
LUT	Look-up table; precomputed output value for every input grey level.
Median filter	Non-linear filter that outputs the median value of the window.
Otsu threshold	Global threshold that maximises the between-class variance.
PSNR	Peak signal-to-noise ratio in dB; pixel-wise error measure against a reference.
Ringing	Oscillations near edges caused by a sharp cut-off in the frequency domain.
Salt-and-pepper noise	Random pixels set to the minimum or maximum grey level.
SSIM	Structural similarity index; compares local luminance, contrast and structure.
Unsharp mask	Sharpening by adding a scaled difference between the image and its blurred copy.
Zero padding	Appending zeros around an image before the FFT to avoid wrap-around error.

19. References

[1] Gonzalez, R. C. and Woods, R. E. (2018). Digital Image Processing, 4th edition. Pearson.

[2] Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In Graphics Gems IV, Academic Press, pp. 474–485.

[3] Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62–66.

[4] Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679–698.

[5] Wang, Z., Bovik, A. C., Sheikh, H. R. and Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. IEEE Transactions on Image Processing, 13(4), 600–612.

[6] Telea, A. (2004). An image inpainting technique based on the fast marching method. Journal of Graphics Tools, 9(1), 23–34.

[7] Bertalmio, M., Bertozzi, A. L. and Sapiro, G. (2001). Navier–Stokes, fluid dynamics, and image and video inpainting. Proceedings of IEEE CVPR, pp. 355–362.

[8] Sauvola, J. and Pietikäinen, M. (2000). Adaptive document image binarization. Pattern Recognition, 33(2), 225–236.

[9] Zhang, K., Zuo, W., Chen, Y., Meng, D. and Zhang, L. (2017). Beyond a Gaussian denoiser: residual learning of deep CNN for image denoising (DnCNN). IEEE Transactions on Image Processing, 26(7), 3142–3155.

[10] Xie, S. and Tu, Z. (2015). Holistically-nested edge detection. Proceedings of IEEE ICCV, pp. 1395–1403.

[11] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.

[12] van der Walt, S. et al. (2014). scikit-image: image processing in Python. PeerJ, 2, e453.

20. Worked Numerical Examples

The following hand calculations illustrate the main formulae of Chapter 14 on very small data. They are textbook-style exercises (all numbers were computed with Python and checked) and are not experimental results on the photograph set.

20.1 Gamma and logarithmic LUT values

For 8-bit images the tables below give the output s for selected inputs r. The log transform uses c = 255 / log(256) ≈ 45.99.

Table 37. Look-up table values (rounded to the nearest integer).

Input r	Gamma 0.5	Gamma 2.0	Log (c = 45.99)
0	0	0	0
32	90	4	161
64	128	16	192
128	181	64	223
192	221	145	242
255	255	255	255

Reading: for r = 32 (a dark pixel) γ = 0.5 raises the value to 90 and the logarithm to 161, while γ = 2.0 lowers it to 4. For r = 192 (a bright pixel) γ = 2.0 gives 145, which spreads the bright end of the histogram downwards, and the logarithm gives 242, which compresses the bright end. This is the numerical reason why log is the wrong choice for a faded, bright print and why the same γ cannot suit both bright and dark photographs (Q5).

20.2 Histogram equalisation of a 3-bit image

Consider a 64×64 image (4096 pixels) with 8 grey levels (L = 8) and the histogram below. The transformation is s_k = round(7 · CDF(r_k)).

Table 38. Histogram equalisation of a 3-bit, 4096-pixel image.

r_k	n_k	p(r_k)	CDF	7 · CDF	s_k (rounded)
0	790	0.19	0.19	1.35	1
1	1023	0.25	0.44	3.10	3
2	850	0.21	0.65	4.55	5
3	656	0.16	0.81	5.67	6
4	329	0.08	0.89	6.23	6
5	245	0.06	0.95	6.65	7
6	122	0.03	0.98	6.86	7
7	81	0.02	1.00	7.00	7

The output histogram has 790 pixels at level 1, 1023 at level 3, 850 at level 5, 985 at level 6 (levels 3 and 4 merged) and 448 at level 7 (levels 5, 6 and 7 merged). Two observations follow. First, the output is not exactly flat, because a discrete mapping cannot split the pixels of one input level. Second, three input levels were merged into level 7 and two into level 6, so some tonal distinctions disappeared: this is the numerical form of the clipping and detail loss seen for global HE in Table 5.

20.3 Median versus mean on an impulse

Take the 3×3 neighbourhood of a pixel with one salt impulse in the centre:

$$[\,10\ \ 12\ \ 11\,;\ 13\ \ 255\ \ 12\,;\ 11\ \ 10\ \ 13\,]$$

Sorted values: 10, 10, 11, 11, 12, 12, 13, 13, 255. The median is the fifth value, 12, so the impulse is removed completely and the result is close to the neighbours. The 3×3 box (mean) filter gives 347 / 9 ≈ 38.6, which is far from every neighbour and spreads the impulse over all nine output pixels in a neighbourhood. This is why the

median 3×3 filter gave the best PSNR in Table 7 and why a Gaussian or box filter should never be applied before the median.

20.4 Otsu threshold on a ten-level histogram

Let the histogram of 94 pixels over grey levels 0–9 be h = [2, 10, 18, 12, 3, 2, 4, 14, 20, 9]. For every candidate threshold T, class 0 contains levels ≤ T and class 1 the rest; ω are the class probabilities and μ the class means. The between-class variance $\omega 0 \cdot \omega 1 \cdot (\mu 0 - \mu 1)^2$ is:

Table 39. Otsu criterion for each candidate threshold.

T	ω0	μ0	μ1	Between-class variance
0	0.021	0.00	5.08	0.54
1	0.128	0.83	5.57	2.50
2	0.319	1.53	6.58	5.53
3	0.447	1.95	7.40	7.35
4	0.479	2.09	7.61	7.61
5	0.500	2.21	7.72	7.59
6	0.543	2.51	7.88	7.17
7	0.691	3.48	8.31	4.98
8	0.904	4.54	9.00	1.72

The maximum is 7.61 at T = 4, so Otsu's threshold is 4 (pixels at levels 0–4 form class 0). T = 5 is almost as good (7.59), which shows that the criterion is flat near the optimum and that a small amount of noise in the histogram can move the chosen threshold by one level. The two modes (levels 1–3 and levels 7–8) are well separated here; if the dark class and the bright class overlapped, as for uneven exposure, no single T could give a good result (Q51).

20.5 PSNR for different error levels

For 8-bit images $PSNR = 10 \cdot \log10(255^2 / MSE)$. An average squared error of 25 (RMS error 5 grey levels) gives 34.15 dB; MSE = 100 (RMS error 10) gives 28.13 dB; MSE = 400 (RMS error 20) gives 22.11 dB. Every doubling of the RMS error therefore costs about 6.02 dB. When Table 7 reports that the noisy input has 24.18 dB, the average error is about 15.8 grey levels RMS (since 255 / 10^(24.18/20) ≈ 15.8), and the best filter (31.25 dB) reduces it to about 7.0.

20.6 Matching the frequency and spatial Gaussian

For an image of width N = 512 and a Gaussian low-pass filter with D0 = 80 cycles per image width, the equivalent spatial standard deviation is $\sigma_s \approx N / (2\pi \cdot D0) = 512 / (2\pi \cdot 80) \approx 1.02$ pixels. A D0 of 40 corresponds to $\sigma_s \approx 2.04$ px and D0 of 20 to about 4.07 px. This explains the results of Task 5: D0 = 80 acts like a spatial Gaussian of about one pixel (mild smoothing, good compromise), while D0 = 20 acts like a Gaussian of about 4 px and over-smooths (edge retention 0.35). The relation holds for the unpadded image size; with zero padding to 2N the same σ_s is obtained with the padded D0 scaled accordingly, which is why D0 was always given in cycles per 512 px of the original image (Section 0).

Appendix A – Python / OpenCV source code (Q71)

All experiments can be reproduced with: python3 task12.py, task34.py, task56.py, task78.py, task9.py, task910.py, task10b.py (common.py builds the synthetic photographs).

common.py

```
"""Common utilities: synthetic old-photograph degradation model (with ground truth).

No real archive photographs were supplied, so clean public test images that ship
with scikit-image are degraded with a physically motivated model. Ground-truth
damage masks and clean references are therefore available for every image.
"""
import cv2, numpy as np, os, time
import skimage.data as sd

W = 512                      # working resolution for experiments (square)
ROOT = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(ROOT, "data")
FIGS = os.path.join(ROOT, "figs")
os.makedirs(DATA, exist_ok=True); os.makedirs(FIGS, exist_ok=True)


def _fit(img, size=W):
    h, w = img.shape[:2]
    s = min(h, w)
    y0, x0 = (h - s) // 2, (w - s) // 2
    return cv2.resize(img[y0:y0 + s, x0:x0 + s], (size, size), interpolation=cv2.INTER_AREA)


def clean_sources():
    """Return dict name -> clean BGR uint8 image (512x512)."""
    rgb = {
        "astronaut": sd.astronaut(), "coffee": sd.coffee(), "chelsea": sd.chelsea(),
        "rocket": sd.rocket(), "camera": cv2.cvtColor(sd.camera(), cv2.COLOR_GRAY2RGB),
        "coins": cv2.cvtColor(sd.coins(), cv2.COLOR_GRAY2RGB),
    }
    return {k: cv2.cvtColor(_fit(v), cv2.COLOR_RGB2BGR) for k, v in rgb.items()}


def _rng(seed):
    return np.random.default_rng(seed)


def make_damage(shape, rng, level="light"):
    """Create damage layers. Returns (dark_alpha, light_alpha, mask) for gray image of `shape`.
    dark_alpha: stain darkening 0..1 ; light_alpha: scratches/dust/crease/tear whitening 0..1."""
    h, w = shape
    mask = np.zeros((h, w), np.uint8)
    light = np.zeros((h, w), np.float32)
    dark = np.zeros((h, w), np.float32)
    n_scr = {"light": 5, "faded": 6, "uneven": 7}[level]
    n_dust = {"light": 60, "faded": 80, "uneven": 100}[level]
    n_stain = {"light": 2, "faded": 3, "uneven": 3}[level]
    # scratches: thin, near-vertical/oblique lines of different orientation
    for _ in range(n_scr):
        x0 = rng.integers(20, w - 20); y0 = rng.integers(0, h // 3)
        ang = np.deg2rad(rng.uniform(-35, 35)); L = rng.integers(h // 3, h)
        x1 = int(x0 + L * np.sin(ang)); y1 = int(y0 + L * np.cos(ang))
        t = int(rng.integers(1, 3))
        cv2.line(light, (x0, y0), (x1, y1), 1.0, t, cv2.LINE_AA)
        cv2.line(mask, (x0, y0), (x1, y1), 255, t + 1)
    # dust spots
    for _ in range(n_dust):
        x, y = rng.integers(5, w - 5), rng.integers(5, h - 5); r = int(rng.integers(1, 4))
        cv2.circle(light, (x, y), r, 1.0, -1, cv2.LINE_AA); cv2.circle(mask, (x, y), r + 1, 255, -1)
    # crease: long straight lines, light core (+ slight shadow) at oblique angle
    for _ in range(2 if level != "light" else 1):
        x0 = rng.integers(0, w); y0 = 0; ang = np.deg2rad(rng.uniform(55, 80)) * rng.choice([-1, 1])
        x1 = int(x0 + h * np.tan(ang) * 0.35); y1 = h - 1
        c = np.zeros((h, w), np.float32); cv2.line(c, (x0, y0), (x1, y1), 1.0, 3, cv2.LINE_AA)
        light = np.maximum(light, 0.7 * c); cv2.line(mask, (x0, y0), (x1, y1), 255, 4)
    # ink / water stains: smooth blobs darkening the print
    for _ in range(n_stain):
        cx, cy = rng.integers(60, w - 60), rng.integers(60, h - 60)
        ax, ay = rng.integers(25, 60), rng.integers(20, 50)
        blob = np.zeros((h, w), np.float32)
        cv2.ellipse(blob, (cx, cy), (ax, ay), rng.uniform(0, 180), 0, 360, 1.0, -1)
        blob = cv2.GaussianBlur(blob, (0, 0), 5)
        dark = np.maximum(dark, 0.45 * blob)
        mask[blob > 0.5] = 255
```

```
    # torn corner (missing region -> paper white)
    if level in ("faded", "uneven"):
        pts = np.array([[w - 1, 0], [w - rng.integers(60, 110), 0], [w - 1, rng.integers(60, 110)]], np.int32)
        cv2.fillPoly(light, [pts], 1.0); cv2.fillPoly(mask, [pts], 255)
    make_damage.bright = ((light > 0.3).astype(np.uint8) * 255)     # bright-type damage only (scratch/dust/crease/tear)
    return dark, light, mask


def degrade_gray(clean_gray, level, seed, all_masks=False):
    """clean_gray uint8 -> (degraded uint8, mask uint8 0/255).  Includes the operating-data
    noise: Gaussian sigma=6, 1% salt&pepper, horizontal stripes every 6 rows."""
    rng = _rng(seed)
    g = clean_gray.astype(np.float32)
    h, w = g.shape
    a, b, gam = {"light": (0.80, 20, 1.0), "faded": (0.50, 70, 0.8), "uneven": (0.65, 35, 1.0)}[level]
    g = 255 * (g / 255) ** gam
    g = a * g + b                                                   # fading: lost contrast
    if level == "uneven":                                           # one-sided shadow
        xx = np.linspace(0, 1, w, dtype=np.float32)[None, :]
        yy = np.linspace(0, 1, h, dtype=np.float32)[:, None]
        shade = 1.0 - 0.55 * np.clip(1.3 * xx - 0.15 + 0.2 * yy, 0, 1) ** 1.5
        g = g * shade
    g = cv2.GaussianBlur(g, (0, 0), 1.0 if level != "light" else 0.6)  # slight blur
    dark, light, mask = make_damage((h, w), rng, level)
    g = g * (1 - dark)                                              # stains
    g = g * (1 - light) + 235 * light                               # scratches/dust/crease/tear
    g += rng.normal(0, 6, g.shape)                                  # Gaussian noise, sigma ~ 6
    g[::6, :] += 5                                                  # scanner stripes (every 6 rows)
    g = np.clip(g, 0, 255)
    sp = rng.random(g.shape)
    g[sp < 0.005] = 0; g[sp > 0.995] = 255                          # 1% salt & pepper total
    if all_masks:
        return g.astype(np.uint8), mask, make_damage.bright.copy()
    return g.astype(np.uint8), mask


def degrade_color(clean_bgr, seed, size=None):
    """Sepia/yellow cast, fade, stains, scratches, noise, JPEG blocking.
    Returns (degraded BGR uint8, mask)."""
    rng = _rng(seed)
    img = clean_bgr.astype(np.float32)
    h, w = img.shape[:2]
    lvl = ["light", "faded", "uneven"][seed % 3]
    a, b = {"light": (0.8, 15), "faded": (0.55, 55), "uneven": (0.68, 30)}[lvl]
    img = a * img + b
    gains = np.array([0.70, 0.88, 1.08], np.float32)                # B,G,R -> yellow/sepia cast
    img = img * gains[None, None, :]
    if lvl == "uneven":
        xx = np.linspace(0, 1, w, dtype=np.float32)[None, :, None]
        img = img * (1 - 0.45 * np.clip(1.3 * xx - 0.15, 0, 1) ** 1.5)
    img = cv2.GaussianBlur(img, (0, 0), 0.9)
    dark, light, mask = make_damage((h, w), rng, lvl)
    img = img * (1 - dark[..., None] * np.array([1.0, 0.9, 0.7], np.float32))
    paper = np.array([150, 205, 235], np.float32)                   # yellowed paper (BGR)
    img = img * (1 - light[..., None]) + paper * light[..., None]
    img += rng.normal(0, 4, img.shape)
    img = np.clip(img, 0, 255).astype(np.uint8)
    ok, enc = cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 35])   # blocking artefacts
    return cv2.imdecode(enc, cv2.IMREAD_COLOR), mask


GRAY_SET = [("astronaut", "light", 11), ("camera", "light", 12), ("coins", "light", 13),
            ("astronaut", "faded", 21), ("camera", "faded", 22), ("chelsea", "faded", 23),
            ("camera", "uneven", 31), ("astronaut", "uneven", 32), ("coffee", "uneven", 33)]


def build_gray_set():
    src = clean_sources()
    out = []
    for i, (name, lvl, seed) in enumerate(GRAY_SET):
        cg = cv2.cvtColor(src[name], cv2.COLOR_BGR2GRAY)
        d, m, bm = degrade_gray(cg, lvl, seed, all_masks=True)
        out.append(dict(id=f"G{i+1:02d}", name=name, level=lvl, clean=cg, img=d, mask=m, bright=bm))
    return out


COLOR_SET = []  # (source, seed) -> 15 images: 5 scenes x 3 degradation seeds
for _k, _s in enumerate(["astronaut", "coffee", "chelsea", "rocket", "astronaut"]):
    pass


def build_color_set():
    src = clean_sources()
```

```
    names = ["astronaut", "coffee", "chelsea", "rocket", "camera"]
    out = []
    n = 0
    for name in names:
        for k in range(3):
            n += 1
            base = src[name]
            if k == 1: base = base[:, ::-1].copy()          # mirrored variant of same scene
            if k == 2: base = np.ascontiguousarray(base[40:472, 40:472]); base = cv2.resize(base, (W, W))
            d, m = degrade_color(base, seed=100 + n)
            out.append(dict(id=f"C{n:02d}", name=name, clean=base, img=d, mask=m))
    return out


def timeit(fn, *a, reps=3, **k):
    best = 1e9; r = None
    for _ in range(reps):
        t = time.perf_counter(); r = fn(*a, **k); best = min(best, time.perf_counter() - t)
    return r, best * 1000.0   # ms


def dice(a, b):
    a = a.astype(bool); b = b.astype(bool)
    s = a.sum() + b.sum()
    return 1.0 if s == 0 else 2.0 * (a & b).sum() / s


def iou(a, b):
    a = a.astype(bool); b = b.astype(bool)
    u = (a | b).sum()
    return 1.0 if u == 0 else (a & b).sum() / u
```

pipeline.py

```
"""Integrated restoration pipelines (grayscale and colour strategies A/B/C)."""
import sys
sys.path.insert(0, "/home/claude/work")
from common import *


# ---------------------------------------------------------------- grayscale pipeline (Tasks 1-9)
def destripe(im, period=6):
    """Remove horizontal scanner stripes: subtract row-mean deviation from its smooth trend."""
    f = im.astype(np.float32)
    rm = f.mean(1)
    trend = cv2.blur(rm.reshape(-1, 1), (1, 25)).ravel()
    return np.clip(f - (rm - trend)[:, None], 0, 255)


def flatten_illum(f, scale=16, strength=1.0):
    """Spatial-domain equivalent of homomorphic filtering: divide by a very smooth illumination estimate."""
    h, w = f.shape[:2]
    small = cv2.resize(f, (max(w // scale, 8), max(h // scale, 8)), interpolation=cv2.INTER_AREA)
    small = cv2.medianBlur(small.astype(np.uint8), 5).astype(np.float32) if small.ndim == 2 else small
    ill = cv2.GaussianBlur(small, (0, 0), 3)
    ill = cv2.resize(ill, (w, h), interpolation=cv2.INTER_LINEAR)
    ill = np.maximum(ill, 1.0)
    target = float(ill.mean())
    out = f * (target / ill) ** strength
    return np.clip(out, 0, 255)


def detect_damage(f8):
    """Bright damage (scratch / dust / crease / tear): bright anomaly over local background (top-hat style)."""
    s = cv2.GaussianBlur(cv2.medianBlur(f8, 3), (0, 0), 1.0)
    h, w = f8.shape
    bg = cv2.resize(cv2.medianBlur(cv2.resize(s, (max(w // 4, 8), max(h // 4, 8)), interpolation=cv2.INTER_AREA), 9), (w, h),
interpolation=cv2.INTER_LINEAR)
    m = ((s.astype(np.int16) - bg.astype(np.int16)) > 45).astype(np.uint8)
    n, lab, st, _ = cv2.connectedComponentsWithStats(m, 8)
    keep = np.zeros(n, bool); keep[1:] = (st[1:, cv2.CC_STAT_AREA] >= 4) & (st[1:, cv2.CC_STAT_AREA] < 0.03 * h * w)
    return keep[lab]


def restore_gray(im, timings=None, inpaint=True):
    """Selected techniques: median 3x3 (T3) -> destripe -> illumination flatten (T6) -> CLAHE (T2) ->
    Gaussian 3x3 -> unsharp mask (T4) -> damage detection (T7/T8/T9) -> Telea inpainting of the mask (labelled)."""
    T = {}
    def tick(name, t0): T[name] = (time.perf_counter() - t0) * 1000
    t = time.perf_counter(); damage = detect_damage(im); tick("1 damage detection", t)
    t = time.perf_counter(); x = cv2.medianBlur(im, 3); tick("2 median 3x3", t)
```

```
    t = time.perf_counter(); x = destripe(x); tick("3 destripe", t)
    t = time.perf_counter(); x = flatten_illum(x); tick("4 illumination flatten", t)
    t = time.perf_counter(); x = cv2.createCLAHE(2.0, (8, 8)).apply(x.astype(np.uint8)); tick("5 CLAHE", t)
    t = time.perf_counter(); x = cv2.GaussianBlur(x, (3, 3), 0.8); tick("6 Gaussian 3x3", t)
    t = time.perf_counter()
    xf = x.astype(np.float32); x = np.clip(xf + 0.7 * (xf - cv2.GaussianBlur(xf, (0, 0), 1.5)), 0, 255).astype(np.uint8); tick("7
unsharp mask", t)
    out = x
    if inpaint:
        t = time.perf_counter()
        dm = cv2.dilate(damage.astype(np.uint8), np.ones((3, 3), np.uint8))
        out = cv2.inpaint(x, dm, 3, cv2.INPAINT_TELEA); tick("8 inpaint (Telea)", t)
    if timings is not None: timings.update(T)
    return x, out, damage


# ---------------------------------------------------------------- colour strategies (Task 10)
def A_retain(bgr, timings=None):
    """Existing grayscale pipeline applied unchanged to each channel independently, full resolution."""
    t = time.perf_counter()
    chans = []
    for c in cv2.split(bgr):
        x, o, _ = restore_gray(c, None, inpaint=False); chans.append(x)
    out = cv2.merge(chans)
    dmg = detect_damage(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY))
    if timings is not None: timings["total_ms"] = (time.perf_counter() - t) * 1000
    return out, dmg


def _white_balance_lab(bgr, strength=0.85):
    """Reduce sepia/yellow cast: shift the mean a*,b* of the *mid-tone, low-saturation* pixels towards neutral.
    strength<1 keeps part of the original warm tone (historical appearance, skin tones)."""
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB).astype(np.float32)
    L, a, b = lab[..., 0], lab[..., 1], lab[..., 2]
    sel = (L > 40) & (L < 230)
    ma, mb = a[sel].mean(), b[sel].mean()
    a = a - strength * (ma - 128); b = b - strength * (mb - 128)
    lab[..., 1], lab[..., 2] = a, b
    return lab


def B_modify(bgr, timings=None, work=1000):
    """Resize for the heavy steps, process L (luminance) and chroma separately, then recombine at full size."""
    t = time.perf_counter(); h, w = bgr.shape[:2]
    s = min(1.0, work / max(h, w))
    small = cv2.resize(bgr, (int(w * s), int(h * s)), interpolation=cv2.INTER_AREA) if s < 1 else bgr
    lab = _white_balance_lab(small)
    L = np.clip(lab[..., 0], 0, 255).astype(np.uint8)
    Lr, _, dmg_s = restore_gray(L, None, inpaint=False)
    # chroma: Gaussian smoothing removes JPEG blocking in a/b without touching luminance detail
    a = cv2.GaussianBlur(lab[..., 1], (0, 0), 1.5); b = cv2.GaussianBlur(lab[..., 2], (0, 0), 1.5)
    out_s = cv2.cvtColor(np.dstack([Lr.astype(np.float32), a, b]).clip(0, 255).astype(np.uint8), cv2.COLOR_LAB2BGR)
    out = cv2.resize(out_s, (w, h), interpolation=cv2.INTER_CUBIC) if s < 1 else out_s
    dmg = cv2.resize(dmg_s.astype(np.uint8), (w, h), interpolation=cv2.INTER_NEAREST) > 0 if s < 1 else dmg_s
    if timings is not None: timings["total_ms"] = (time.perf_counter() - t) * 1000
    return out, dmg


def C_redesign(bgr, timings=None, work=1000):
    """B + chroma-weighted deblocking, frequency-domain stripe/periodic-noise suppression (notch on the vertical axis)
    and inpainting of *small* damaged regions only (large regions are flagged, not invented)."""
    t = time.perf_counter(); h, w = bgr.shape[:2]
    s = min(1.0, work / max(h, w))
    small = cv2.resize(bgr, (int(w * s), int(h * s)), interpolation=cv2.INTER_AREA) if s < 1 else bgr
    lab = _white_balance_lab(small)
    L = np.clip(lab[..., 0], 0, 255).astype(np.uint8)
    # frequency-domain: suppress narrow periodic horizontal-stripe line in the spectrum (notch around u-axis peaks)
    F = np.fft.fftshift(np.fft.fft2(L.astype(np.float32)))
    hh, ww = L.shape; cy, cx = hh // 2, ww // 2
    mag = np.abs(F); col = mag[:, cx - 1:cx + 2].mean(1)
    th = np.median(col) * 6
    for y in range(hh):
        if abs(y - cy) > 8 and col[y] > th:
            F[y, cx - 3:cx + 4] *= 0.1
    Lf = np.clip(np.real(np.fft.ifft2(np.fft.ifftshift(F))), 0, 255).astype(np.uint8)
    Lr, _, dmg_s = restore_gray(Lf, None, inpaint=False)
    a = cv2.bilateralFilter(lab[..., 1], 7, 12, 5); b = cv2.bilateralFilter(lab[..., 2], 7, 12, 5)
    out_s = cv2.cvtColor(np.dstack([Lr.astype(np.float32), a, b]).clip(0, 255).astype(np.uint8), cv2.COLOR_LAB2BGR)
    # inpaint only small components (thin scratches, dust); components > 2500 px are left untouched and reported
    n, lab_cc, st, _ = cv2.connectedComponentsWithStats(dmg_s.astype(np.uint8), 8)
    small_ok = np.zeros(n, bool); small_ok[1:] = st[1:, cv2.CC_STAT_AREA] <= 2500
    msk = cv2.dilate(small_ok[lab_cc].astype(np.uint8), np.ones((3, 3), np.uint8))
    out_s = cv2.inpaint(out_s, msk, 3, cv2.INPAINT_TELEA)
```

```
    out = cv2.resize(out_s, (w, h), interpolation=cv2.INTER_CUBIC) if s < 1 else out_s
    dmg = cv2.resize(dmg_s.astype(np.uint8), (w, h), interpolation=cv2.INTER_NEAREST) > 0 if s < 1 else dmg_s
    if timings is not None: timings["total_ms"] = (time.perf_counter() - t) * 1000
    return out, dmg
```

task12.py

```
"""Tasks 1 and 2: grey-level transformations and histogram processing."""
import sys, json
sys.path.insert(0, "/home/claude/work")
from common import *
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage.exposure import match_histograms
from skimage.metrics import structural_similarity as ssim

G = build_gray_set()
R = {}   # results store


def entropy(im):
    p = np.bincount(im.ravel(), minlength=256) / im.size
    p = p[p > 0]
    return float(-(p * np.log2(p)).sum())


def rms_contrast(im):
    return float(im.astype(np.float32).std())


def clip_pct(im):
    return float(100.0 * ((im <= 2) | (im >= 253)).mean())


def faint_detail(im):
    """Mean gradient magnitude (Sobel) - proxy for recovery of faint detail."""
    gx = cv2.Sobel(im, cv2.CV_32F, 1, 0, ksize=3); gy = cv2.Sobel(im, cv2.CV_32F, 0, 1, ksize=3)
    return float(np.sqrt(gx ** 2 + gy ** 2).mean())


# ---------------- Task 1 ----------------
def negative(im): return 255 - im
def log_tf(im):
    c = 255 / np.log(1 + 255.0)
    return np.clip(c * np.log1p(im.astype(np.float32)), 0, 255).astype(np.uint8)
def gamma_tf(im, g):
    lut = (255 * (np.arange(256) / 255.0) ** g).astype(np.uint8)
    return cv2.LUT(im, lut)
def pw_stretch(im, lo=2, hi=98):
    a, b = np.percentile(im, [lo, hi])
    return np.clip((im.astype(np.float32) - a) * 255.0 / max(b - a, 1), 0, 255).astype(np.uint8)


def auto_gamma(im):
    """gamma chosen so that mean maps to mid-grey (adaptive version)."""
    m = max(im.mean() / 255, 1e-3)
    return float(np.clip(np.log(0.5) / np.log(m), 0.4, 2.5))


t1_rows = []
for g in G:
    im = g["img"]
    outs = {}
    outs["Original"] = (im, 0.0)
    r, t = timeit(lambda x: log_tf(x), im); outs["A Log"] = (r, t)
    _, tn = timeit(negative, im); outs["A Negative"] = (negative(im), tn)
    gm = auto_gamma(im)
    r, t = timeit(gamma_tf, im, gm); outs["B Gamma(auto)"] = (r, t)
    r, t = timeit(pw_stretch, im); outs["C Piecewise 2-98%"] = (r, t)
    for k, (o, t) in outs.items():
        # subject-background contrast: |mean(central 50%) - mean(border ring)|
        c = o[128:384, 128:384].astype(np.float32).mean(); bd = np.concatenate(
            [o[:64].ravel(), o[-64:].ravel(), o[:, :64].ravel(), o[:, -64:].ravel()]).astype(np.float32).mean()
        t1_rows.append(dict(id=g["id"], level=g["level"], method=k, rms=rms_contrast(o), entropy=entropy(o),
                            clip=clip_pct(o), detail=faint_detail(o), sb=abs(c - bd), ms=t,
                            ssim_clean=float(ssim(o, g["clean"], data_range=255))))
    R.setdefault("gamma_auto", {})[g["id"]] = gm
R["t1"] = t1_rows

# Q3: transformation curves + histograms (G04 faded, G07 uneven)
x = np.arange(256)
```

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].plot(x, 255 - x, label="Negative"); ax[0].plot(x, 255 / np.log(256) * np.log1p(x), label="Log")
for gg in (0.5, 1.0, 2.0): ax[0].plot(x, 255 * (x / 255) ** gg, "--", label=f"Gamma {gg}")
ax[0].set_title("A, B: negative / log / gamma curves"); ax[0].legend(); ax[0].set_xlabel("input"); ax[0].set_ylabel("output")
im = G[3]["img"]; a, b = np.percentile(im, [2, 98])
ax[1].plot([0, a, b, 255], [0, 0, 255, 255], "r-o", label=f"Piecewise (r1={a:.0f}, r2={b:.0f})")
ax[1].plot(x, x, "k:", label="identity"); ax[1].set_title("C: piecewise-linear stretch (G04)"); ax[1].legend()
ax[1].set_xlabel("input"); plt.tight_layout(); plt.savefig(FIGS + "/t1_curves.png", dpi=110); plt.close()

for idx, tag in ((3, "G04"), (6, "G07")):
    im = G[idx]["img"]
    ims = [("Original", im), ("Log", log_tf(im)), (f"Gamma {auto_gamma(im):.2f}", gamma_tf(im, auto_gamma(im))),
           ("Piecewise", pw_stretch(im))]
    fig, ax = plt.subplots(2, 4, figsize=(14, 6.4))
    for j, (n, o) in enumerate(ims):
        ax[0, j].imshow(o, cmap="gray", vmin=0, vmax=255); ax[0, j].set_title(n); ax[0, j].axis("off")
        ax[1, j].hist(o.ravel(), 256, range=(0, 255), color="k"); ax[1, j].set_xlim(0, 255)
    plt.tight_layout(); plt.savefig(FIGS + f"/t1_compare_{tag}.png", dpi=90); plt.close()

# Q4: bit-plane slicing (G01 light vs clean)
im = G[0]["img"]
fig, ax = plt.subplots(2, 4, figsize=(12, 6.2))
bp_rows = []
clean = G[0]["clean"]
for k in range(8):
    plane = ((im >> k) & 1) * 255
    cplane = ((clean >> k) & 1) * 255
    agree = float((plane == cplane).mean())   # agreement of degraded plane with clean plane
    bp_rows.append(dict(bit=k, agree_with_clean=agree))
    a_ = ax[k // 4, k % 4]; a_.imshow(plane, cmap="gray"); a_.set_title(f"bit {k}  (agree {agree:.2f})"); a_.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t1_bitplanes.png", dpi=80); plt.close()
R["bitplanes"] = bp_rows
# reconstruction from top-n planes
rec = {}
for n in (2, 3, 4, 5):
    mask_ = (0xFF << (8 - n)) & 0xFF
    rec[n] = float(ssim(im & mask_, clean, data_range=255))
R["bp_rec_ssim_vs_clean"] = rec

# Q5: same gamma on bright and dark photographs
bright = np.clip(G[0]["clean"].astype(np.float32) * 0.6 + 130, 0, 255).astype(np.uint8)      # over-exposed print
dark_im = np.clip(G[0]["clean"].astype(np.float32) * 0.6, 0, 255).astype(np.uint8)           # under-exposed print
fig, ax = plt.subplots(2, 4, figsize=(13, 6.5))
q5 = []
for j, (n, o) in enumerate([("Bright", bright), ("Dark", dark_im)]):
    for kk, gg in enumerate((0.5, 1.0, 2.0)):
        r = gamma_tf(o, gg)
        q5.append(dict(photo=n, gamma=gg, mean=float(r.mean()), rms=rms_contrast(r), clip=clip_pct(r)))
        ax[j, kk + 1].imshow(r, cmap="gray", vmin=0, vmax=255); ax[j, kk + 1].set_title(f"{n}, gamma={gg}"); ax[j, kk + 1].axis("off")
    ax[j, 0].imshow(o, cmap="gray", vmin=0, vmax=255); ax[j, 0].set_title(f"{n} original (mean {o.mean():.0f})"); ax[j, 0].axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t1_gamma_bright_dark.png", dpi=80); plt.close()
R["q5"] = q5

# ---------------- Task 2 ----------------
def ghe(im): return cv2.equalizeHist(im)
def hmatch(im, ref):
    return np.clip(match_histograms(im, ref), 0, 255).astype(np.uint8)
def clahe(im, clip=2.0, tile=8):
    return cv2.createCLAHE(clipLimit=clip, tileGridSize=(tile, tile)).apply(im)


REF = G[0]["clean"]    # reference histogram: a well-exposed clean portrait (assumption stated in report)
t2_rows = []
for g in G:
    im = g["img"]
    outs = {"Original": (im, 0.0)}
    r, t = timeit(ghe, im); outs["A Global HE"] = (r, t)
    r, t = timeit(hmatch, im, REF, reps=1); outs["B Hist. matching"] = (r, t)
    r, t = timeit(clahe, im); outs["C CLAHE(2.0, 8x8)"] = (r, t)
    # noise amplification: std of the flat region (top-left 40x40 patch minimal-variance patch)
    for k, (o, t) in outs.items():
        # noise estimate = Immerkaer-like: std of (image - 3x3 median) in smooth areas
        d = o.astype(np.float32) - cv2.medianBlur(o, 3).astype(np.float32)
        t2_rows.append(dict(id=g["id"], level=g["level"], method=k, rms=rms_contrast(o), entropy=entropy(o),
                            noise=float(np.abs(d).mean()), clip=clip_pct(o), ms=t,
                            ssim_clean=float(ssim(o, g["clean"], data_range=255))))
R["t2"] = t2_rows

# Q9: histograms and CDF  (G07 uneven)
im = G[6]["img"]
sets = [("Original", im), ("Global HE", ghe(im)), ("Hist. match", hmatch(im, REF)), ("CLAHE", clahe(im))]
fig, ax = plt.subplots(3, 4, figsize=(15, 9))
for j, (n, o) in enumerate(sets):
```

```
    ax[0, j].imshow(o, cmap="gray", vmin=0, vmax=255); ax[0, j].set_title(n); ax[0, j].axis("off")
    ax[1, j].hist(o.ravel(), 256, range=(0, 255), color="k"); ax[1, j].set_xlim(0, 255)
    h = np.bincount(o.ravel(), minlength=256); cdf = np.cumsum(h) / h.sum()
    ax[2, j].plot(cdf, "b"); ax[2, j].plot([0, 255], [0, 1], "r:"); ax[2, j].set_xlim(0, 255); ax[2, j].set_title("CDF")
plt.tight_layout(); plt.savefig(FIGS + "/t2_hist_cdf.png", dpi=80); plt.close()

# Q10: why GHE fails with both dark and bright regions  (G07: one side shadow) -> local contrast in dark & bright halves
im = G[6]["img"]; he = ghe(im); cl = clahe(im)
def halves(o):
    # left (lit) / right (shadow) local std
    return float(o[:, :170].astype(np.float32).std()), float(o[:, 340:].astype(np.float32).std())
R["q10"] = dict(orig=halves(im), ghe=halves(he), clahe=halves(cl),
                mean_lr_orig=(float(im[:, :170].mean()), float(im[:, 340:].mean())),
                mean_lr_ghe=(float(he[:, :170].mean()), float(he[:, 340:].mean())),
                mean_lr_clahe=(float(cl[:, :170].mean()), float(cl[:, 340:].mean())))

# Q11: CLAHE tile/clip sweep (G04 faded + G07 uneven)
sweep = []
for idx in (3, 6):
    im = G[idx]["img"]
    for clip in (1.0, 2.0, 4.0, 8.0):
        for tile in (2, 4, 8, 16):
            o = clahe(im, clip, tile)
            d = o.astype(np.float32) - cv2.medianBlur(o, 3).astype(np.float32)
            sweep.append(dict(id=G[idx]["id"], clip=clip, tile=tile, rms=rms_contrast(o), entropy=entropy(o),
                              noise=float(np.abs(d).mean()), ssim_clean=float(ssim(o, G[idx]["clean"], data_range=255))))
R["clahe_sweep"] = sweep
fig, ax = plt.subplots(2, 4, figsize=(14, 7))
for j, (c, t) in enumerate([(1, 8), (4, 8), (8, 8), (2, 2)]):
    ax[0, j].imshow(clahe(G[3]["img"], c, t), cmap="gray"); ax[0, j].set_title(f"G04 clip={c}, tile={t}x{t}"); ax[0, j].axis("off")
    ax[1, j].imshow(clahe(G[6]["img"], c, t), cmap="gray"); ax[1, j].set_title(f"G07 clip={c}, tile={t}x{t}"); ax[1, j].axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t2_clahe_sweep.png", dpi=75); plt.close()

# Q12: GHE on near-uniform background: synthetic print with large flat area
flat = np.full((512, 512), 200, np.uint8)
cv2.circle(flat, (256, 256), 60, 140, -1)               # small subject on flat paper
flat = np.clip(flat.astype(np.float32) + np.random.default_rng(5).normal(0, 3, flat.shape), 0, 255).astype(np.uint8)
fe = ghe(flat); fc = clahe(flat)
R["q12"] = dict(std_bg_before=float(flat[:100, :100].std()), std_bg_ghe=float(fe[:100, :100].std()),
                std_bg_clahe=float(fc[:100, :100].std()),
                bg_mean_before=float(flat[:100, :100].mean()), bg_mean_ghe=float(fe[:100, :100].mean()))
fig, ax = plt.subplots(2, 3, figsize=(11, 7))
for j, (n, o) in enumerate([("Flat-background print", flat), ("Global HE", fe), ("CLAHE", fc)]):
    ax[0, j].imshow(o, cmap="gray", vmin=0, vmax=255); ax[0, j].set_title(n); ax[0, j].axis("off")
    ax[1, j].hist(o.ravel(), 256, range=(0, 255), color="k")
plt.tight_layout(); plt.savefig(FIGS + "/t2_flat_bg.png", dpi=80); plt.close()

# Q8 before/after on G04
im = G[3]["img"]
fig, ax = plt.subplots(1, 4, figsize=(14, 3.8))
for j, (n, o) in enumerate([("Original (faded)", im), ("Global HE", ghe(im)), ("Hist. matching", hmatch(im, REF)), ("CLAHE",
clahe(im))]):
    ax[j].imshow(o, cmap="gray", vmin=0, vmax=255); ax[j].set_title(n); ax[j].axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t2_compare_G04.png", dpi=80); plt.close()

json.dump(R, open("/home/claude/work/results_t12.json", "w"), indent=1, default=float)

# print summary tables (mean by level & method)
import collections
def agg(rows, keys):
    d = collections.defaultdict(lambda: collections.defaultdict(list))
    for r in rows:
        for k in keys: d[(r["level"], r["method"])][k].append(r[k])
    return d
for name, rows, keys in (("TASK1", t1_rows, ["rms", "entropy", "clip", "detail", "sb", "ms", "ssim_clean"]),
                         ("TASK2", t2_rows, ["rms", "entropy", "noise", "clip", "ms", "ssim_clean"])):
    print(name)
    d = agg(rows, keys)
    for (lv, m), v in sorted(d.items()):
        print(f"{lv:7s} {m:20s} " + " ".join(f"{k}={np.mean(v[k]):.2f}" for k in keys))
print("bitplane agree", [round(b["agree_with_clean"], 2) for b in bp_rows], R["bp_rec_ssim_vs_clean"])
print("q5", q5); print("q10", R["q10"]); print("q12", R["q12"])
print("gamma auto", R["gamma_auto"])
```

task34.py

```
"""Tasks 3 and 4: spatial smoothing and sharpening."""
import sys, json, collections
sys.path.insert(0, "/home/claude/work")
from common import *
import matplotlib; matplotlib.use("Agg")
```

```
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr, structural_similarity as ssim

src = clean_sources()
CL = {k: cv2.cvtColor(src[k], cv2.COLOR_BGR2GRAY) for k in ("astronaut", "camera", "coins")}
R = {}


def noisy(clean, seed):
    """Film-grain noise per the operating data: Gaussian sigma=6, 1% salt-pepper, stripes every 6 rows."""
    rng = np.random.default_rng(seed)
    g = clean.astype(np.float32) + rng.normal(0, 6, clean.shape)
    g[::6, :] += 4
    g = np.clip(g, 0, 255)
    sp = rng.random(g.shape); g[sp < 0.005] = 0; g[sp > 0.995] = 255
    return g.astype(np.uint8)


NZ = {k: noisy(v, 40 + i) for i, (k, v) in enumerate(CL.items())}

# ---------------- Task 3 ----------------
def box(im, k): return cv2.blur(im, (k, k))
def gauss(im, k): return cv2.GaussianBlur(im, (k, k), 0)
def med(im, k): return cv2.medianBlur(im, k)
FILT = {"Box": box, "Gaussian": gauss, "Median": med}
rows3 = []
for name in CL:
    for fn, f in FILT.items():
        for k in (3, 5, 7):
            o, ms = timeit(f, NZ[name], k)
            rows3.append(dict(img=name, filt=fn, k=k, psnr=float(psnr(CL[name], o, data_range=255)),
                              ssim=float(ssim(CL[name], o, data_range=255)), ms=ms,
                              lap_var=float(cv2.Laplacian(o, cv2.CV_64F).var())))
    rows3.append(dict(img=name, filt="Noisy input", k=0, psnr=float(psnr(CL[name], NZ[name], data_range=255)),
                      ssim=float(ssim(CL[name], NZ[name], data_range=255)), ms=0, lap_var=float(cv2.Laplacian(NZ[name],
cv2.CV_64F).var())))
R["t3"] = rows3

# filtered images figure (astronaut)
a = NZ["astronaut"]
fig, ax = plt.subplots(3, 4, figsize=(14, 10.5))
ax[0, 0].imshow(a[100:300, 130:330], cmap="gray"); ax[0, 0].set_title("Noisy input (crop)")
for j, k in enumerate((3, 5, 7)):
    for i, (fn, f) in enumerate(FILT.items()):
        pass
for i, (fn, f) in enumerate(FILT.items()):
    for j, k in enumerate((3, 5, 7)):
        ax[i, j + 1].imshow(f(a, k)[100:300, 130:330], cmap="gray"); ax[i, j + 1].set_title(f"{fn} {k}x{k}")
for i in (1, 2): ax[i, 0].imshow(CL["astronaut"][100:300, 130:330], cmap="gray") if i == 1 else ax[i, 0].axis("off")
ax[1, 0].set_title("Clean reference (crop)")
for x in ax.ravel(): x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t3_filters.png", dpi=70); plt.close()

# Q16 intensity profile across face/eye edge on astronaut: horizontal line through the eyes region
row = 205; x0, x1 = 170, 260
fig, ax = plt.subplots(1, 3, figsize=(15, 3.8))
for j, k in enumerate((3, 5, 7)):
    ax[j].plot(range(x0, x1), CL["astronaut"][row, x0:x1], "k", lw=2, label="clean")
    ax[j].plot(range(x0, x1), NZ["astronaut"][row, x0:x1], color="0.7", lw=1, label="noisy")
    for fn, f in FILT.items(): ax[j].plot(range(x0, x1), f(NZ["astronaut"], k)[row, x0:x1], lw=1.4, label=fn)
    ax[j].set_title(f"Intensity profile, row {row}, kernel {k}x{k}"); ax[j].legend(fontsize=7)
plt.tight_layout(); plt.savefig(FIGS + "/t3_profiles.png", dpi=80); plt.close()

# Q18: effect of 7x7 on small detail -> edge preservation: gradient energy retained & thin-line contrast retained
thin = np.full((120, 120), 150, np.uint8); thin[:, 58:60] = 60          # 2-px dark line (hair strand/narrow boundary)
thin[50:53, 20:40] = 60                                                 # small 3-px detail
q18 = []
for fn, f in FILT.items():
    for k in (3, 5, 7):
        o = f(thin, k); q18.append(dict(filt=fn, k=k, line_contrast=float(150 - o[20, 56:62].min()), rise=float(o[20, 56:62].max() -
o[20, 56:62].min())))
R["q18"] = q18
# fraction of Sobel energy retained in the astronaut face region
def gradE(im): return float(np.sqrt(cv2.Sobel(im, cv2.CV_32F, 1, 0) ** 2 + cv2.Sobel(im, cv2.CV_32F, 0, 1) ** 2)[100:300,
130:330].mean())
ge0 = gradE(CL["astronaut"])
R["q18_edge_retained"] = {f"{fn}{k}": gradE(f(CL["astronaut"], k)) / ge0 for fn, f in FILT.items() for k in (3, 5, 7)}

# Q17 correlation vs convolution (asymmetric 3x3 mask)
img = np.zeros((9, 9), np.float32); img[4, 4] = 1.0; img[2, 6] = 2.0
mask = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]], np.float32)         # asymmetric mask
corr = cv2.filter2D(img, -1, mask, borderType=cv2.BORDER_CONSTANT)     # OpenCV filter2D = correlation
conv = cv2.filter2D(img, -1, mask[::-1, ::-1].copy(), borderType=cv2.BORDER_CONSTANT)
```

```
R["q17"] = dict(corr_patch=corr[3:6, 3:6].tolist(), conv_patch=conv[3:6, 3:6].tolist(),
                same_for_symmetric=bool(np.allclose(cv2.filter2D(img, -1, mask + mask[::-1, ::-1]), cv2.filter2D(img, -1, (mask +
mask[::-1, ::-1])[::-1, ::-1]))))
fig, ax = plt.subplots(1, 3, figsize=(10, 3.4))
for x, (n, d) in zip(ax, [("Input (impulse at centre)", img), ("Correlation", corr), ("Convolution", conv)]):
    x.imshow(d, cmap="viridis"); x.set_title(n)
plt.tight_layout(); plt.savefig(FIGS + "/t3_corr_conv.png", dpi=80); plt.close()

# Task 3 on actual degraded set G04 (dust + S&P) : dust/impulse removal check inside non-damage region
G = build_gray_set()
imp = G[0]["img"]
sp_mask = (imp == 0) | (imp == 255)
R["t3_sp_residual"] = {fn: float((f(imp, 3) [sp_mask] == 0).mean() + (f(imp, 3)[sp_mask] == 255).mean()) for fn, f in FILT.items()}

# ---------------- Task 4 ----------------
def blur_deg(clean, seed):
    b = cv2.GaussianBlur(clean, (0, 0), 1.6).astype(np.float32)
    b += np.random.default_rng(seed).normal(0, 3, b.shape)
    return np.clip(b, 0, 255).astype(np.uint8)

BL = {k: blur_deg(v, 70 + i) for i, (k, v) in enumerate(CL.items())}
k4 = np.array([[0, -1, 0], [-1, 4, -1], [0, -1, 0]], np.float32)

def lap_sharp(im, c=1.0):
    L = cv2.filter2D(im.astype(np.float32), -1, k4)
    return np.clip(im.astype(np.float32) + c * L, 0, 255).astype(np.uint8)

def unsharp(im, sigma=1.5, amount=1.5):          # high-boost when amount>1 (k = amount+1)
    b = cv2.GaussianBlur(im.astype(np.float32), (0, 0), sigma)
    return np.clip(im.astype(np.float32) + amount * (im.astype(np.float32) - b), 0, 255).astype(np.uint8)

def grad_sharp(im, c=0.5):
    f = im.astype(np.float32)
    gx = cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3) / 8; gy = cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3) / 8
    mag = np.sqrt(gx ** 2 + gy ** 2)
    return np.clip(f + c * mag * 4, 0, 255).astype(np.uint8)   # add scaled gradient magnitude

def sharp_metrics(o, clean, flat_sl=(slice(20, 60), slice(20, 60))):
    lv = float(cv2.Laplacian(o, cv2.CV_64F).var())
    return dict(lap_var=lv, psnr=float(psnr(clean, o, data_range=255)), ssim=float(ssim(clean, o, data_range=255)))

# step-edge test for halo/overshoot and 10-90 rise: synthetic edge blurred by same sigma
step = np.full((100, 200), 70, np.uint8); step[:, 100:] = 180
stepb = cv2.GaussianBlur(step, (0, 0), 1.6)
rng = np.random.default_rng(3)

def edge_stats(o):
    p = o[50, 80:121].astype(np.float32)
    lo, hi = 70, 180
    ov = max(p.max() - hi, 0) + max(lo - p.min(), 0)           # overshoot + undershoot (halo)
    q = (p - lo) / (hi - lo)
    try:
        r10 = np.where(q >= 0.1)[0][0]; r90 = np.where(q >= 0.9)[0][0]; rise = float(r90 - r10)
    except IndexError:
        rise = float("nan")
    return float(ov), rise

METH = {"A Laplacian": lambda im: lap_sharp(im, 1.0), "B Unsharp (s=1.5,a=1.5)": lambda im: unsharp(im, 1.5, 1.5),
        "B High-boost (a=4)": lambda im: unsharp(im, 1.5, 4.0), "C Gradient (c=0.5)": lambda im: grad_sharp(im, 0.5)}
rows4 = []
for name in CL:
    bl = BL[name]
    base = dict(img=name, method="Blurred input", **sharp_metrics(bl, CL[name]))
    base["noise"] = float(np.abs(bl[20:60, 20:60].astype(np.float32) - cv2.GaussianBlur(bl, (0, 0), 3)[20:60, 20:60]).mean());
rows4.append(base)
    for m, f in METH.items():
        o, ms = timeit(f, bl)
        d = sharp_metrics(o, CL[name]); d.update(img=name, method=m, ms=ms)
        d["noise"] = float(np.abs(o[20:60, 20:60].astype(np.float32) - cv2.GaussianBlur(o, (0, 0), 3)[20:60, 20:60]).mean())
        rows4.append(d)
R["t4"] = rows4
R["t4_edge"] = {"Blurred step": edge_stats(stepb), **{m: edge_stats(f(stepb)) for m, f in METH.items()}}
R["clean_step"] = edge_stats(step)

# Q24 smoothing before sharpening
res24 = {}
bl = BL["astronaut"]
for m, f in (("Unsharp a=2 (direct)", lambda im: unsharp(im, 1.5, 2.0)), ("Laplacian (direct)", lambda im: lap_sharp(im, 1.0))):
    o1 = f(bl); o2 = f(cv2.GaussianBlur(bl, (0, 0), 1.0))
    nz = lambda o: float(np.abs(o[20:60, 20:60].astype(np.float32) - cv2.GaussianBlur(o, (0, 0), 3)[20:60, 20:60]).mean())
    res24[m] = dict(noise_direct=nz(o1), noise_smoothed_first=nz(o2), psnr_direct=float(psnr(CL["astronaut"], o1, data_range=255)),
                    psnr_smooth_first=float(psnr(CL["astronaut"], o2, data_range=255)), ssim_direct=float(ssim(CL["astronaut"], o1,
data_range=255)),
```

```
                    ssim_smooth_first=float(ssim(CL["astronaut"], o2, data_range=255)))
R["q24"] = res24

# Q25 high-boost large factor on scratched / grainy photo (G04 faded)
g4 = G[3]["img"]
res25 = []
for a_ in (1, 2, 4, 8):
    o = unsharp(g4, 1.5, a_)
    res25.append(dict(amount=a_, lap_var=float(cv2.Laplacian(o, cv2.CV_64F).var()), noise=float(np.abs(o[20:60,
20:60].astype(np.float32) - cv2.GaussianBlur(o, (0, 0), 3)[20:60, 20:60]).mean()),
                      ssim_clean=float(ssim(G[3]["clean"], o, data_range=255)), clip=float(100 * ((o <= 2) | (o >= 253)).mean())))
R["q25"] = res25

# figures
a = BL["astronaut"]; c = CL["astronaut"]
ims = [("Clean", c), ("Blurred (input)", a)] + [(m, f(a)) for m, f in METH.items()]
fig, ax = plt.subplots(2, 3, figsize=(14, 9.5))
for x, (n, o) in zip(ax.ravel(), ims):
    x.imshow(o[90:320, 120:350], cmap="gray", vmin=0, vmax=255); x.set_title(n); x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t4_sharpen.png", dpi=70); plt.close()

fig, ax = plt.subplots(1, 2, figsize=(13, 3.8))
row = 205
ax[0].plot(range(170, 260), c[row, 170:260], "k", lw=2, label="clean"); ax[0].plot(range(170, 260), a[row, 170:260], color="0.6",
label="blurred")
for m, f in METH.items(): ax[0].plot(range(170, 260), f(a)[row, 170:260], lw=1.2, label=m)
ax[0].set_title("Intensity profile across facial edges"); ax[0].legend(fontsize=6)
for m, f in [("Blurred step", lambda z: z)] + list(METH.items()):
    ax[1].plot(range(80, 121), f(stepb)[50, 80:121], label=m)
ax[1].set_title("Step-edge profile: overshoot (halo) and rise"); ax[1].legend(fontsize=6)
plt.tight_layout(); plt.savefig(FIGS + "/t4_profiles.png", dpi=80); plt.close()

fig, ax = plt.subplots(1, 4, figsize=(15, 4))
for x, a_ in zip(ax, (1, 2, 4, 8)):
    x.imshow(unsharp(g4, 1.5, a_)[0:250, 0:250], cmap="gray"); x.set_title(f"High-boost amount={a_} (G04 scratched/grainy)");
x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t4_highboost_damage.png", dpi=70); plt.close()

json.dump(R, open("/home/claude/work/results_t34.json", "w"), indent=1, default=float)

agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows3:
    for k in ("psnr", "ssim", "ms", "lap_var"): agg[(r["filt"], r["k"])][k].append(r[k])
print("TASK3 (mean over 3 images)")
for key, v in agg.items(): print(key, {k: round(float(np.mean(x)), 3) for k, x in v.items()})
print("q18", q18); print("edge retained", {k: round(v, 3) for k, v in R["q18_edge_retained"].items()})
print("q17", R["q17"]); print("sp residual", R["t3_sp_residual"])
agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows4:
    for k in ("lap_var", "psnr", "ssim", "noise", "ms"):
        if k in r: agg[r["method"]][k].append(r[k])
print("TASK4 (mean over 3 images)")
for key, v in agg.items(): print(key, {k: round(float(np.mean(x)), 3) for k, x in v.items()})
print("edge", R["t4_edge"], R["clean_step"]); print("q24", res24); print("q25", res25)
```

task56.py

```
"""Tasks 5 and 6: frequency-domain low-pass and high-pass processing."""
import sys, json, collections
sys.path.insert(0, "/home/claude/work")
from common import *
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr, structural_similarity as ssim

src = clean_sources()
CL = {k: cv2.cvtColor(src[k], cv2.COLOR_BGR2GRAY) for k in ("astronaut", "camera", "coins")}
R = {}


def noisy(clean, seed):
    rng = np.random.default_rng(seed)
    g = clean.astype(np.float32) + rng.normal(0, 6, clean.shape)
    g[::6, :] += 4
    g = np.clip(g, 0, 255)
    sp = rng.random(g.shape); g[sp < 0.005] = 0; g[sp > 0.995] = 255
    return g.astype(np.uint8)


NZ = {k: noisy(v, 40 + i) for i, (k, v) in enumerate(CL.items())}
# For low-pass tests we first remove salt&pepper impulses with a 3x3 median? No: keep raw so each filter is judged on the same data.
```

```
def dist_grid(P, Q):
    u = np.fft.fftfreq(P) * P; v = np.fft.fftfreq(Q) * Q          # centred at 0 for un-shifted spectrum
    V, U = np.meshgrid(v, u)
    return np.sqrt(U ** 2 + V ** 2)


def H_lp(kind, D, D0, n=2):
    if kind == "ILPF": return (D <= D0).astype(np.float32)
    if kind == "BLPF": return 1.0 / (1.0 + (D / D0) ** (2 * n))
    if kind == "GLPF": return np.exp(-(D ** 2) / (2 * D0 ** 2)).astype(np.float32)


def freq_filter(im, H_fn, pad=True):
    f = im.astype(np.float32); h, w = f.shape
    P, Q = (2 * h, 2 * w) if pad else (h, w)
    F = np.fft.fft2(f, s=(P, Q))
    H = H_fn(dist_grid(P, Q))
    g = np.real(np.fft.ifft2(F * H))[:h, :w]
    return np.clip(g, 0, 255)


def lpf(im, kind, D0, pad=True):
    return freq_filter(im, lambda D: H_lp(kind, D, D0 * (2 if pad else 1)), pad)   # D0 given in cycles per 512 px; scaled for padded
grid


# ---------------- Task 5 ----------------
rows5 = []
for name in CL:
    for kind in ("ILPF", "BLPF", "GLPF"):
        for D0 in (20, 40, 80, 120):
            o, ms = timeit(lpf, NZ[name], kind, D0, reps=2)
            o8 = o.astype(np.uint8)
            gx = lambda z: np.sqrt(cv2.Sobel(z.astype(np.float32), cv2.CV_32F, 1, 0) ** 2 + cv2.Sobel(z.astype(np.float32),
cv2.CV_32F, 0, 1) ** 2)
            rows5.append(dict(img=name, kind=kind, D0=D0, psnr=float(psnr(CL[name], o8, data_range=255)),
                              ssim=float(ssim(CL[name], o8, data_range=255)), ms=ms,
                              edge_ret=float(gx(o8)[40:470, 40:470].mean() / gx(CL[name])[40:470, 40:470].mean())))
R["t5"] = rows5

# ringing on a step edge + thin line (synthetic, noise-free)
step = np.full((256, 256), 60, np.float32); step[:, 128:] = 190
ring = []
for kind in ("ILPF", "BLPF", "GLPF"):
    for D0 in (20, 40):
        o = freq_filter(step, lambda D: H_lp(kind, D, D0 * 2), True)
        p = o[128, 100:156]
        under = max(60 - p.min(), 0); over = max(p.max() - 190, 0)
        # ring energy: sum |deviation| outside transition band (|x-128|>8)
        left = p[:20] - 60; right = p[-20:] - 190
        ring.append(dict(kind=kind, D0=D0, overshoot=float(over), undershoot=float(under),
                         ring_energy=float(np.abs(left).sum() + np.abs(right).sum())))
R["ringing"] = ring
fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
for x, D0 in zip(ax, (20, 40, 80)):
    for kind in ("ILPF", "BLPF", "GLPF"):
        x.plot(freq_filter(step, lambda D: H_lp(kind, D, D0 * 2), True)[128, 90:166], label=kind)
    x.plot(step[128, 90:166], "k:", label="ideal step"); x.set_title(f"Step-edge response, D0={D0}"); x.legend(fontsize=7)
plt.tight_layout(); plt.savefig(FIGS + "/t5_ringing.png", dpi=80); plt.close()

# Q28 spectrum: noisy image, spectrum + identify stripe peak
im = NZ["astronaut"]; F = np.fft.fftshift(np.fft.fft2(im.astype(np.float32) - im.mean()))
S = np.log1p(np.abs(F))
cy, cx = 256, 256
col = np.abs(F[:, cx]); stripe_bin = int(round(512 / 6))                    # expected 85.3 cycles
win = col[cy + stripe_bin - 4: cy + stripe_bin + 5]
peak_off = int(np.argmax(win)) - 4 + stripe_bin
neigh = np.r_[col[cy + stripe_bin - 20: cy + stripe_bin - 8], col[cy + stripe_bin + 8: cy + stripe_bin + 20]].mean()
R["q28"] = dict(stripe_peak_bin=peak_off, expected=512 / 6, peak_over_neighbourhood=float(col[cy + peak_off] / neigh))
# radial profile of spectrum
Dg = np.sqrt((np.arange(512)[:, None] - cy) ** 2 + (np.arange(512)[None, :] - cx) ** 2).astype(int)
rad_n = np.bincount(Dg.ravel(), np.abs(F).ravel()) / np.bincount(Dg.ravel())
Fc = np.fft.fftshift(np.fft.fft2(CL["astronaut"].astype(np.float32) - CL["astronaut"].mean()))
rad_c = np.bincount(Dg.ravel(), np.abs(Fc).ravel()) / np.bincount(Dg.ravel())
fig, ax = plt.subplots(1, 3, figsize=(15, 4.4))
ax[0].imshow(np.log1p(np.abs(Fc)), cmap="gray"); ax[0].set_title("Spectrum: clean photo")
ax[1].imshow(S, cmap="gray"); ax[1].set_title("Spectrum: noisy photo (stripe peaks on vertical axis)")
ax[2].semilogy(rad_c[:256], label="clean"); ax[2].semilogy(rad_n[:256], label="noisy"); ax[2].legend()
ax[2].set_title("Radial mean magnitude"); ax[2].set_xlabel("radius (cycles/512 px)")
plt.tight_layout(); plt.savefig(FIGS + "/t5_spectrum.png", dpi=75); plt.close()
# noise floor at high freq
```

```
R["q28"]["hf_noise_ratio"] = float(rad_n[150:250].mean() / rad_c[150:250].mean())

# filtered images figure (astronaut crop)
fig, ax = plt.subplots(3, 4, figsize=(14, 10.5))
for i, kind in enumerate(("ILPF", "BLPF", "GLPF")):
    for j, D0 in enumerate((20, 40, 80, 120)):
        ax[i, j].imshow(lpf(NZ["astronaut"], kind, D0)[100:300, 130:330], cmap="gray", vmin=0, vmax=255); ax[i, j].set_title(f"{kind}
D0={D0}"); ax[i, j].axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t5_filtered.png", dpi=65); plt.close()

# Q31 frequency Gaussian vs spatial Gaussian (sigma_s = P/(2*pi*D0_padded))
q31 = []
for D0 in (20, 40, 80):
    fo = lpf(NZ["astronaut"], "GLPF", D0)
    sig = 512 / (2 * np.pi * D0)
    so = cv2.GaussianBlur(NZ["astronaut"], (0, 0), sig, borderType=cv2.BORDER_CONSTANT).astype(np.float32)
    f8, s8 = fo.astype(np.uint8), np.clip(so, 0, 255).astype(np.uint8)
    q31.append(dict(D0=D0, sigma=float(sig), freq_psnr=float(psnr(CL["astronaut"], f8, data_range=255)),
spat_psnr=float(psnr(CL["astronaut"], s8, data_range=255)),
                    freq_ssim=float(ssim(CL["astronaut"], f8, data_range=255)), spat_ssim=float(ssim(CL["astronaut"], s8,
data_range=255)),
                    agreement_psnr=float(psnr(f8, s8, data_range=255)),
                    freq_ms=timeit(lpf, NZ["astronaut"], "GLPF", D0, reps=2)[1],
                    spat_ms=timeit(lambda z: cv2.GaussianBlur(z, (0, 0), sig), NZ["astronaut"], reps=3)[1]))
R["q31"] = q31

# Q32 padding vs no padding: wrap-around error near borders (compare with spatial Gaussian using constant border)
a = NZ["camera"]
good = lpf(a, "GLPF", 40, True); bad = lpf(a, "GLPF", 40, False)
ref = cv2.GaussianBlur(a, (0, 0), 512 / (2 * np.pi * 40), borderType=cv2.BORDER_CONSTANT).astype(np.float32)
bw = 12
bm = np.zeros((512, 512), bool); bm[:bw] = True; bm[-bw:] = True; bm[:, :bw] = True; bm[:, -bw:] = True
R["q32"] = dict(border_err_pad=float(np.abs(good - ref)[bm].mean()), border_err_nopad=float(np.abs(bad - ref)[bm].mean()),
                interior_err_pad=float(np.abs(good - ref)[~bm].mean()), interior_err_nopad=float(np.abs(bad - ref)[~bm].mean()))
fig, ax = plt.subplots(1, 3, figsize=(13, 4.4))
ax[0].imshow(good, cmap="gray"); ax[0].set_title("With zero-padding (2x)"); ax[1].imshow(bad, cmap="gray"); ax[1].set_title("Without
padding (circular wrap-around)")
ax[2].imshow(np.abs(bad - good), cmap="hot", vmax=60); ax[2].set_title("|difference| (bright = wrap-around error)")
for x in ax: x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t5_padding.png", dpi=75); plt.close()

# ---------------- Task 6 ----------------
def shadow_only(clean, seed):
    rng = np.random.default_rng(seed); h, w = clean.shape
    g = 0.65 * clean.astype(np.float32) + 35
    xx = np.linspace(0, 1, w, dtype=np.float32)[None, :]; yy = np.linspace(0, 1, h, dtype=np.float32)[:, None]
    shade = 1.0 - 0.55 * np.clip(1.3 * xx - 0.15 + 0.2 * yy, 0, 1) ** 1.5
    g = g * shade + rng.normal(0, 4, g.shape)
    return np.clip(g, 0, 255).astype(np.uint8), shade


SH = {}
for i, k in enumerate(("astronaut", "camera", "coins")):
    SH[k] = shadow_only(CL[k], 60 + i)[0]


def hp_ideal(D, D0): return (D > D0).astype(np.float32)
def hp_butter(D, D0, n=2): return 1.0 - 1.0 / (1.0 + (np.maximum(D, 1e-6) / D0) ** (2 * n))
def hp_gauss(D, D0): return 1.0 - np.exp(-(D ** 2) / (2 * D0 ** 2))


def ffilt(im, H_fn, pad=True):
    f = im.astype(np.float32); h, w = f.shape
    P, Q = 2 * h, 2 * w
    F = np.fft.fft2(f, s=(P, Q))
    return np.real(np.fft.ifft2(F * H_fn(dist_grid(P, Q))))[:h, :w]


def stretch(x, lo=1, hi=99):
    a, b = np.percentile(x, [lo, hi]); return np.clip((x - a) * 255 / max(b - a, 1e-6), 0, 255)


def A_butter_hp(im, D0=30):                       # Butterworth HPF; add back the DC (mean) to keep tone
    hp = ffilt(im, lambda D: hp_butter(D, D0 * 2))
    return np.clip(hp + im.mean(), 0, 255)


def B_hfe(im, D0=30, a=0.5, b=1.5):               # high-frequency emphasis: H = a + b*Hhp
    return np.clip(ffilt(im, lambda D: a + b * hp_gauss(D, D0 * 2)), 0, 255)


def C_homo(im, gl=0.5, gh=1.8, c=1.0, D0=30):
    ln = np.log1p(im.astype(np.float32))
```

```
    H = lambda D: (gh - gl) * (1 - np.exp(-c * (D ** 2) / ((D0 * 2) ** 2))) + gl
    g = np.expm1(np.maximum(ffilt(ln, H), 0))
    g = g * (float(im.mean()) / max(float(g.mean()), 1e-6))   # keep overall tone: match the input mean brightness
    return np.clip(g, 0, 255)


def evenness(o):
    """Coefficient of variation of the coarse illumination field (Gaussian sigma=40) - lower = more even."""
    bl = cv2.GaussianBlur(o.astype(np.float32), (0, 0), 40)
    return float(bl.std() / max(bl.mean(), 1e-6))


def lowfreq_err(o, cl):
    """RMSE between z-scored coarse illumination fields (Gaussian sigma=30) of output and clean photo.
    Lower = the large-scale tone distribution is closer to the undamaged original."""
    z = lambda a: (lambda b: (b - b.mean()) / (b.std() + 1e-6))(cv2.GaussianBlur(a.astype(np.float32), (0, 0), 30))
    return float(np.sqrt(((z(o) - z(cl)) ** 2).mean()))


def local_contrast(o):
    f = o.astype(np.float32); m = cv2.blur(f, (15, 15)); v = cv2.blur(f * f, (15, 15)) - m * m
    return float(np.sqrt(np.maximum(v, 0)).mean())


def noise_est(o):
    return float(np.abs(o.astype(np.float32) - cv2.medianBlur(o.astype(np.uint8), 3).astype(np.float32)).mean())


METH6 = {"A Butterworth HPF": A_butter_hp, "B Gaussian HFE": B_hfe, "C Homomorphic": C_homo}
rows6 = []
for k in SH:
    inp = SH[k]
    cl = CL[k]
    for m, f in [("Shadowed input", lambda z: z.astype(np.float32))] + list(METH6.items()):
        o, ms = timeit(f, inp, reps=2); o8 = np.clip(o, 0, 255).astype(np.uint8)
        # uniform patch: sky/background region top-left 40x40 for astronaut etc. use brightest-flat patch in clean: min local variance
        v = cv2.blur(cl.astype(np.float32) ** 2, (41, 41)) - cv2.blur(cl.astype(np.float32), (41, 41)) ** 2
        v[:20] = 1e9; v[-20:] = 1e9; v[:, :20] = 1e9; v[:, -20:] = 1e9
        yy, xx = np.unravel_index(np.argmin(v), v.shape)
        patch = o8[yy - 20:yy + 20, xx - 20:xx + 20]
        # compare with clean by a global linear fit (illumination-independent) -> PSNR/SSIM after gain-offset matching
        A_ = np.vstack([o8.ravel().astype(np.float32), np.ones(o8.size)]).T
        co = np.linalg.lstsq(A_, cl.ravel().astype(np.float32), rcond=None)[0]
        fit = np.clip(o8 * co[0] + co[1], 0, 255).astype(np.uint8)
        rows6.append(dict(img=k, method=m, lowfreq_err=lowfreq_err(o8, cl), evenness=evenness(o8), local_contrast=local_contrast(o8),
noise=noise_est(o8),
                          patch_std=float(patch.std()), patch_mean=float(patch.mean()), mean=float(o8.mean()), rms=float(o8.std()),
                          psnr_fit=float(psnr(cl, fit, data_range=255)), ssim_fit=float(ssim(cl, fit, data_range=255)), ms=ms))
    rows6.append(dict(img=k, method="Clean target", evenness=evenness(cl), local_contrast=local_contrast(cl), noise=noise_est(cl),
mean=float(cl.mean()), rms=float(cl.std())))
R["t6"] = rows6

# Q37 ideal high-pass applied directly (no DC restore)
inp = SH["astronaut"]
ih = ffilt(inp, lambda D: hp_ideal(D, 30 * 2))
ih8 = stretch(ih, 0.5, 99.5).astype(np.uint8)
step = np.full((256, 256), 60, np.float32); step[:, 128:] = 190
ring_hp = ffilt(step, lambda D: hp_ideal(D, 30 * 2))[128, 100:156]
R["q37"] = dict(mean_before=float(inp.mean()), mean_after_raw=float(ih.mean()), min_raw=float(ih.min()), max_raw=float(ih.max()),
                step_ringing_peak_to_peak=float(ring_hp.max() - ring_hp.min()))
# Q38 interchange gammaL / gammaH
inp = SH["astronaut"]
good = C_homo(inp, 0.5, 1.8); swap = C_homo(inp, 1.8, 0.5)
R["q38"] = dict(good=dict(evenness=evenness(good), local_contrast=local_contrast(good), lowfreq_err=lowfreq_err(good,
CL["astronaut"])),
                swapped=dict(evenness=evenness(swap), local_contrast=local_contrast(swap), lowfreq_err=lowfreq_err(swap,
CL["astronaut"])),
                input=dict(evenness=evenness(inp), local_contrast=local_contrast(inp), lowfreq_err=lowfreq_err(inp, CL["astronaut"])))
fig, ax = plt.subplots(2, 4, figsize=(15, 8))
for x, (n, o) in zip(ax[0], [("Shadowed input", inp), ("A Butterworth HPF", A_butter_hp(inp)), ("B Gaussian HFE", B_hfe(inp)), ("C
Homomorphic", C_homo(inp))]):
    x.imshow(o, cmap="gray", vmin=0, vmax=255); x.set_title(n); x.axis("off")
for x, (n, o) in zip(ax[1], [("Clean target", CL["astronaut"]), ("Ideal HPF (raw, stretched)", ih8), ("Homomorphic gL=0.5,gH=1.8",
good), ("Homomorphic swapped gL=1.8,gH=0.5", swap)]):
    x.imshow(o, cmap="gray", vmin=0, vmax=255); x.set_title(n); x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t6_compare.png", dpi=65); plt.close()

json.dump(R, open("/home/claude/work/results_t56.json", "w"), indent=1, default=float)

agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows5:
    for k in ("psnr", "ssim", "ms", "edge_ret"): agg[(r["kind"], r["D0"])][k].append(r[k])
print("TASK5 (mean over 3 images)")
```

```
for key, v in sorted(agg.items()): print(key, {k: round(float(np.mean(x)), 3) for k, x in v.items()})
print("ringing", ring); print("q28", R["q28"]); print("q31", q31); print("q32", R["q32"])
agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows6:
    for k in ("lowfreq_err", "evenness", "local_contrast", "noise", "patch_std", "psnr_fit", "ssim_fit", "ms", "mean", "rms"):
        if k in r: agg[r["method"]][k].append(r[k])
print("TASK6")
for key, v in agg.items(): print(key, {k: round(float(np.mean(x)), 3) for k, x in v.items()})
print("q37", R["q37"]); print("q38", R["q38"])
```

task78.py

```
"""Tasks 7 and 8: damage detection (point/line/edge operators) and thresholding."""
import sys, json, collections
sys.path.insert(0, "/home/claude/work")
from common import *
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

G = build_gray_set()
R = {}
TOL = 2   # pixel tolerance for matching detections to ground truth


def prf(det, gt, tol=TOL):
    """Pixel precision/recall with a +-tol pixel tolerance (thin structures are never pixel-exact)."""
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (2 * tol + 1, 2 * tol + 1))
    det = det.astype(bool); gt = gt.astype(bool)
    gt_d = cv2.dilate(gt.astype(np.uint8), k).astype(bool); det_d = cv2.dilate(det.astype(np.uint8), k).astype(bool)
    tp_p = (det & gt_d).sum(); tp_r = (gt & det_d).sum()
    p = tp_p / max(det.sum(), 1); r = tp_r / max(gt.sum(), 1)
    f = 2 * p * r / max(p + r, 1e-9)
    return float(p), float(r), float(f)


# ---------------- detectors ----------------
def pre(im): return cv2.GaussianBlur(im, (0, 0), 1.0).astype(np.float32)

POINT = np.array([[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]], np.float32)
LINES = {"H": np.array([[-1, -1, -1], [2, 2, 2], [-1, -1, -1]], np.float32),
         "V": np.array([[-1, 2, -1], [-1, 2, -1], [-1, 2, -1]], np.float32),
         "+45": np.array([[-1, -1, 2], [-1, 2, -1], [2, -1, -1]], np.float32),
         "-45": np.array([[2, -1, -1], [-1, 2, -1], [-1, -1, 2]], np.float32)}


def det_A(im, T):
    f = cv2.GaussianBlur(im, (0, 0), 0.8).astype(np.float32)
    resp = [cv2.filter2D(f, -1, POINT)] + [cv2.filter2D(f, -1, m) for m in LINES.values()]
    s = np.max(np.stack(resp), 0)                      # positive response = bright point/line on darker surround
    return s > T


def roberts(f):
    gx = cv2.filter2D(f, -1, np.array([[1, 0], [0, -1]], np.float32)); gy = cv2.filter2D(f, -1, np.array([[0, 1], [-1, 0]],
np.float32))
    return np.sqrt(gx ** 2 + gy ** 2)
def prewitt(f):
    kx = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], np.float32)
    return np.sqrt(cv2.filter2D(f, -1, kx) ** 2 + cv2.filter2D(f, -1, kx.T) ** 2)
def sobel(f):
    return np.sqrt(cv2.Sobel(f, cv2.CV_32F, 1, 0) ** 2 + cv2.Sobel(f, cv2.CV_32F, 0, 1) ** 2)

GRAD = {"Roberts": roberts, "Prewitt": prewitt, "Sobel": sobel}

def det_B(im, T, op="Sobel"):
    return GRAD[op](pre(im)) > T

def marr_hildreth(im, sigma=1.5, T=3.0):
    f = cv2.GaussianBlur(im.astype(np.float32), (0, 0), sigma)
    L = cv2.Laplacian(f, cv2.CV_32F)
    zc = np.zeros(L.shape, bool)
    for dy, dx in ((0, 1), (1, 0), (1, 1), (1, -1)):
        s = np.roll(L, (dy, dx), (0, 1))
        zc |= ((L * s) < 0) & (np.abs(L - s) > T)       # slope test removes weak zero crossings
    return zc

def det_C(im, lo, hi, op="Canny"):
    if op == "Canny": return cv2.Canny(cv2.GaussianBlur(im, (0, 0), 1.0), lo, hi) > 0
    return marr_hildreth(im, 1.5, lo)

# ---------------- threshold selection: sweep, choose threshold maximising mean F1 over the set -------------
```

```
def evaluate(fn, thresholds, gtkey="mask"):
    best = None; curves = []
    for T in thresholds:
        fs = []; ps = []; rs = []
        for g in G:
            p, r, f = prf(fn(g["img"], T), g[gtkey])
            ps.append(p); rs.append(r); fs.append(f)
        curves.append((T, float(np.mean(ps)), float(np.mean(rs)), float(np.mean(fs))))
        if best is None or curves[-1][3] > best[3]: best = curves[-1]
    return best, curves

sweeps = {
    "A Point/line masks": (lambda im, T: det_A(im, T), [20, 40, 60, 80, 100, 130, 160, 200, 260]),
    "B Roberts": (lambda im, T: det_B(im, T, "Roberts"), [10, 20, 30, 40, 60, 80, 100, 130]),
    "B Prewitt": (lambda im, T: det_B(im, T, "Prewitt"), [20, 40, 60, 80, 100, 130, 160, 200, 260]),
    "B Sobel": (lambda im, T: det_B(im, T, "Sobel"), [40, 80, 120, 160, 200, 260, 320, 400]),
    "C Marr-Hildreth": (lambda im, T: det_C(im, T, None, "MH"), [2, 4, 6, 8, 12, 16, 24]),
    "C Canny": (lambda im, T: det_C(im, T, 2 * T, "Canny"), [20, 40, 60, 80, 100, 130]),
}
best = {}; curves_all = {}
for name, (fn, ths) in sweeps.items():
    b, cv = evaluate(fn, ths, "mask"); best[name] = b; curves_all[name] = cv
R["t7_best"] = {k: dict(T=v[0], precision=v[1], recall=v[2], f1=v[3]) for k, v in best.items()}
R["t7_curves"] = curves_all

# timing + per-image table at best thresholds
rows7 = []
for name, (fn, _) in sweeps.items():
    T = best[name][0]
    for g in G:
        o, ms = timeit(fn, g["img"], T, reps=2)
        p, r, f = prf(o, g["mask"]); rows7.append(dict(method=name, img=g["id"], level=g["level"], precision=p, recall=r, f1=f, ms=ms,
frac=float(o.mean())))
R["t7_rows"] = rows7

# Q44 thresholds for different damage types: separate GT for dust (small blobs) / scratch (thin) / boundary (tear) using synthetic
test
tst = np.full((300, 300), 120, np.uint8)
cv2.circle(tst, (50, 50), 2, 235, -1); cv2.circle(tst, (90, 50), 3, 235, -1)                # dust
cv2.line(tst, (20, 120), (280, 150), 235, 1, cv2.LINE_AA); cv2.line(tst, (20, 170), (280, 200), 235, 2, cv2.LINE_AA)   # thin
scratches
cv2.fillPoly(tst, [np.array([[300, 0], [230, 0], [300, 70]])], 235)                         # torn boundary
tst = np.clip(tst.astype(np.float32) + np.random.default_rng(1).normal(0, 6, tst.shape), 0, 255).astype(np.uint8)
gt_dust = np.zeros_like(tst); cv2.circle(gt_dust, (50, 50), 3, 255, -1); cv2.circle(gt_dust, (90, 50), 4, 255, -1)
gt_scr = np.zeros_like(tst); cv2.line(gt_scr, (20, 120), (280, 150), 255, 2); cv2.line(gt_scr, (20, 170), (280, 200), 255, 3)
gt_tear = np.zeros_like(tst); cv2.fillPoly(gt_tear, [np.array([[300, 0], [230, 0], [300, 70]])], 255)
q44 = []
for T in (20, 40, 60, 100, 160, 260):
    d = det_A(tst, T); q44.append(dict(method="A", T=T, dust_recall=prf(d, gt_dust)[1], scratch_recall=prf(d, gt_scr)[1],
tear_edge_recall=prf(d, gt_tear)[1],
                                       false_pix_pct=float(100 * (d & ~cv2.dilate((gt_dust | gt_scr | gt_tear), np.ones((5, 5),
np.uint8)).astype(bool)).mean())))
for T in (40, 80, 160, 260, 400):
    d = det_B(tst, T, "Sobel"); q44.append(dict(method="Sobel", T=T, dust_recall=prf(d, gt_dust)[1], scratch_recall=prf(d, gt_scr)[1],
tear_edge_recall=prf(d, gt_tear)[1],
                                       false_pix_pct=float(100 * (d & ~cv2.dilate((gt_dust | gt_scr | gt_tear), np.ones((5, 5),
np.uint8)).astype(bool)).mean())))
R["q44"] = q44

# Q45 directional masks vs line orientation: lines at many angles, per-mask recall
ang_rows = []
for ang in (0, 20, 45, 70, 90, 110, 135, 160):
    t = np.full((200, 200), 120, np.uint8)
    c = (100, 100); L = 90; a = np.deg2rad(ang)
    p1 = (int(c[0] - L * np.cos(a)), int(c[1] - L * np.sin(a))); p2 = (int(c[0] + L * np.cos(a)), int(c[1] + L * np.sin(a)))
    cv2.line(t, p1, p2, 235, 1, cv2.LINE_AA)
    gt = np.zeros_like(t); cv2.line(gt, p1, p2, 255, 3)
    t = np.clip(t.astype(np.float32) + np.random.default_rng(2).normal(0, 6, t.shape), 0, 255).astype(np.uint8)
    f = cv2.GaussianBlur(t, (0, 0), 0.8).astype(np.float32)
    row = dict(angle=ang)
    for n, m in LINES.items():
        d = cv2.filter2D(f, -1, m) > 80; row[n] = float(prf(d, gt)[1])
    d = np.max(np.stack([cv2.filter2D(f, -1, m) for m in LINES.values()]), 0) > 80; row["union"] = float(prf(d, gt)[1])
    ang_rows.append(row)
R["q45"] = ang_rows

# figures: detection maps on G04 (faded) at best thresholds + GT
g = G[3]
panels = [("Degraded", g["img"]), ("Ground truth", g["mask"])] + [(n, (sweeps[n][0](g["img"], best[n][0]) * 255).astype(np.uint8)) for
n in sweeps]
fig, ax = plt.subplots(2, 4, figsize=(15, 7.8))
for x, (n, o) in zip(ax.ravel(), panels):
    x.imshow(o, cmap="gray"); x.set_title(n); x.axis("off")
```

```
plt.tight_layout(); plt.savefig(FIGS + "/t7_maps.png", dpi=65); plt.close()
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
for n, cv in curves_all.items():
    ax[0].plot([c[2] for c in cv], [c[1] for c in cv], "-o", ms=3, label=n)
ax[0].set_xlabel("recall"); ax[0].set_ylabel("precision"); ax[0].set_title("Precision-recall (mean over 9 photos)");
ax[0].legend(fontsize=7)
ax[1].plot([r["angle"] for r in ang_rows], [r["union"] for r in ang_rows], "k-o", label="union")
for n in LINES: ax[1].plot([r["angle"] for r in ang_rows], [r[n] for r in ang_rows], "--", label=n)
ax[1].set_xlabel("scratch orientation (deg)"); ax[1].set_ylabel("recall"); ax[1].set_title("Directional 3x3 line masks vs scratch
angle"); ax[1].legend(fontsize=7)
plt.tight_layout(); plt.savefig(FIGS + "/t7_pr_angle.png", dpi=75); plt.close()

# ---------------- Task 8 thresholding ----------------
def th_global(im, T=200):
    return im > T, T

def th_otsu(im):
    # Otsu on the (lightly smoothed) raw image: bright class = candidate damage
    T, b = cv2.threshold(cv2.GaussianBlur(im, (0, 0), 1.0), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return b > 0, T

def th_adapt(im, block=31, C=-12):
    b = cv2.adaptiveThreshold(cv2.GaussianBlur(im, (0, 0), 0.8), 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, block, C)
    return b > 0, None

def clean_mask(b):      # remove isolated single-pixel (impulse) detections: open with 2x2? keep small objects >=3 px via area filter
    n, lab, st, _ = cv2.connectedComponentsWithStats(b.astype(np.uint8), 8)
    keep = np.zeros(n, bool); keep[1:] = st[1:, cv2.CC_STAT_AREA] >= 4
    return keep[lab]

rows8 = []
for g in G:
    for name, f in (("A Global T=200", th_global), ("B Otsu", th_otsu), ("C Adaptive 31,C=-12", th_adapt)):
        (b, T), ms = timeit(f, g["img"], reps=2)
        b2 = clean_mask(b)
        rows8.append(dict(img=g["id"], level=g["level"], method=name, T=None if T is None else float(T),
                          dice_bright_raw=dice(b, g["bright"] > 0), dice_bright=dice(b2, g["bright"] > 0), dice_full=dice(b2,
g["mask"] > 0),
                          iou_bright=iou(b2, g["bright"] > 0), ms=ms))
R["t8"] = rows8

# Q52 adaptive: block size and C sweep (mean Dice over faded + uneven images)
q52 = []
for block in (11, 21, 31, 51, 101):
    for C in (-4, -8, -12, -20, -30):
        ds = [dice(clean_mask(th_adapt(g["img"], block, C)[0]), g["bright"] > 0) for g in G]
        q52.append(dict(block=block, C=C, dice=float(np.mean(ds)), dice_uneven=float(np.mean(ds[6:]))))
R["q52"] = q52

# Q51 global thresholding fails when intensities overlap: sweep T on faded image G04 and uneven G07
q51 = []
for idx in (3, 6, 0):
    for T in (120, 160, 200, 220, 235, 245):
        q51.append(dict(img=G[idx]["id"], T=T, dice=dice(clean_mask(G[idx]["img"] > T), G[idx]["bright"] > 0)))
R["q51"] = q51
# overlap evidence: fraction of undamaged pixels brighter than the darkest 'damage' pixel quantile
ov = []
for g in G:
    dm = g["bright"] > 0; und = ~(g["mask"] > 0)
    ov.append(dict(img=g["id"], level=g["level"], damage_p10=float(np.percentile(g["img"][dm], 10)),
undamaged_frac_above_damage_p10=float((g["img"][und] > np.percentile(g["img"][dm], 10)).mean())))
R["overlap"] = ov

# figures
g = G[3]
panels = [("Degraded G04 (faded)", g["img"]), ("GT bright damage", g["bright"])]
for name, f in (("A Global T=200", th_global), ("B Otsu", th_otsu), ("C Adaptive 31,C=-12", th_adapt)):
    b, T = f(g["img"]); panels.append((name + ("" if T is None else f" (T={T:.0f})"), (clean_mask(b) * 255).astype(np.uint8)))
fig, ax = plt.subplots(1, 5, figsize=(17, 3.9))
for x, (n, o) in zip(ax, panels): x.imshow(o, cmap="gray"); x.set_title(n, fontsize=9); x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t8_binary.png", dpi=70); plt.close()
fig, ax = plt.subplots(1, 3, figsize=(15, 3.6))
for x, idx in zip(ax, (0, 3, 6)):
    im = G[idx]["img"]; x.hist(im[~(G[idx]["mask"] > 0)].ravel(), 128, range=(0, 255), alpha=0.6, label="undamaged px",
color="tab:blue")
    x.hist(im[G[idx]["bright"] > 0].ravel(), 128, range=(0, 255), alpha=0.6, label="bright damage px", color="tab:red")
    x.axvline(th_otsu(im)[1], color="k", ls="--", label=f"Otsu T={th_otsu(im)[1]:.0f}"); x.axvline(200, color="g", ls=":",
label="global T=200")
    x.set_yscale("log"); x.set_title(f"{G[idx]['id']} ({G[idx]['level']})"); x.legend(fontsize=7)
plt.tight_layout(); plt.savefig(FIGS + "/t8_hist.png", dpi=75); plt.close()

json.dump(R, open("/home/claude/work/results_t78.json", "w"), indent=1, default=float)
```

```
print("T7 best thresholds:")
for k, v in R["t7_best"].items(): print(f"  {k:20s}", {a: round(b, 3) for a, b in v.items()})
agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows7:
    for k in ("precision", "recall", "f1", "ms"): agg[r["method"]][k].append(r[k])
for k, v in agg.items(): print("  ", k, {a: round(float(np.mean(b)), 3) for a, b in v.items()})
print("q44"); [print("  ", r) for r in q44]
print("q45"); [print("  ", {k: round(v, 2) for k, v in r.items()}) for r in ang_rows]
agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows8:
    for k in ("dice_bright_raw", "dice_bright", "dice_full", "ms"): agg[(r["method"], r["level"])][k].append(r[k])
print("T8")
for k, v in sorted(agg.items()): print("  ", k, {a: round(float(np.mean(b)), 3) for a, b in v.items()})
print("q52 best", sorted(q52, key=lambda r: -r["dice"])[:4], "worst", sorted(q52, key=lambda r: r["dice"])[:2])
print("q51", q51); print("overlap", [(o['img'], round(o['damage_p10']), round(o['undamaged_frac_above_damage_p10'], 4)) for o in ov])
```

task9.py

```
"""Task 9: region growing, split & merge, edge linking; IoU and boundary error; failure studies."""
import sys, json, collections
sys.path.insert(0, "/home/claude/work")
from common import *
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

G = build_gray_set()
R = {}
TARGET = "bright"      # bright-type damage (scratch, dust, crease, torn region)


def smooth(im): return cv2.GaussianBlur(cv2.medianBlur(im, 3), (0, 0), 1.0)


# ---- A. region growing (automatic seeds = strong bright anomalies, flood fill with similarity criterion) ----
def region_growing(im, Tg=30, seed_T=55, max_area_frac=0.03):
    s = smooth(im)
    bg = cv2.medianBlur(cv2.resize(s, (128, 128)), 9); bg = cv2.resize(bg, im.shape[::-1])
    seeds = ((s.astype(np.int16) - bg.astype(np.int16)) > seed_T).astype(np.uint8)
    n, lab = cv2.connectedComponents(seeds)
    out = np.zeros(im.shape, np.uint8)
    h, w = im.shape
    for i in range(1, n):
        ys, xs = np.where(lab == i)
        y, x = int(ys[len(ys) // 2]), int(xs[len(xs) // 2])
        if out[y, x]: continue
        m = np.zeros((h + 2, w + 2), np.uint8)
        cv2.floodFill(s.copy(), m, (x, y), 255, (Tg,) * 3, (Tg,) * 3, 8 | cv2.FLOODFILL_MASK_ONLY | cv2.FLOODFILL_FIXED_RANGE | (255
<< 8))
        reg = m[1:-1, 1:-1] > 0
        if reg.sum() <= max_area_frac * h * w: out |= reg.astype(np.uint8)       # reject leaked regions
    return out > 0


# ---- B. region splitting and merging (quadtree + adjacency merge) ----
def split_merge(im, split_std=14, min_size=4, merge_T=12):
    s = smooth(im).astype(np.float32); h, w = im.shape
    leaves = []
    def rec(y, x, sz):
        blk = s[y:y + sz, x:x + sz]
        if sz > min_size and blk.std() > split_std:
            hs = sz // 2
            for dy in (0, hs):
                for dx in (0, hs): rec(y + dy, x + dx, hs)
        else: leaves.append((y, x, sz, float(blk.mean())))
    rec(0, 0, h)    # h == w == power of two (512)
    n = len(leaves)
    lab = np.zeros((h, w), np.int32)
    for i, (y, x, sz, m) in enumerate(leaves): lab[y:y + sz, x:x + sz] = i
    parent = list(range(n))
    def find(a):
        while parent[a] != a: parent[a] = parent[parent[a]]; a = parent[a]
        return a
    means = [l[3] for l in leaves]
    pairs = set(zip(lab[:, :-1].ravel().tolist(), lab[:, 1:].ravel().tolist())) | set(zip(lab[:-1].ravel().tolist(),
lab[1:].ravel().tolist()))
    for a, b in pairs:
        if a != b and abs(means[a] - means[b]) < merge_T:
            ra, rb = find(a), find(b)
            if ra != rb: parent[rb] = ra
    root = np.array([find(i) for i in range(n)])
    reg = root[lab]
```

```
    # classify merged regions: damage if mean >> local background and small
    out = np.zeros((h, w), bool)
    bg = cv2.medianBlur(cv2.resize(s.astype(np.uint8), (128, 128)), 9); bg = cv2.resize(bg, (w, h)).astype(np.float32)
    for r in np.unique(reg):
        m = reg == r; area = m.sum()
        if area < 6 or area > 0.03 * h * w: continue
        if s[m].mean() - bg[m].mean() > 35: out |= m
    return out


# ---- C. edge linking and boundary detection ----
def edge_link(im, lo=50, hi=110, close=5):
    s = smooth(im); e = cv2.Canny(s, lo, hi)
    e = cv2.morphologyEx(e, cv2.MORPH_CLOSE, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (close, close)))
    cnts, _ = cv2.findContours(e, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    out = np.zeros(im.shape, np.uint8); h, w = im.shape
    bg = cv2.medianBlur(cv2.resize(s, (128, 128)), 9); bg = cv2.resize(bg, (w, h)).astype(np.float32)
    for c in cnts:
        a = cv2.contourArea(c)
        if len(c) < 4 or a > 0.03 * h * w: continue
        m = np.zeros(im.shape, np.uint8); cv2.drawContours(m, [c], -1, 255, -1)
        if a < 3: cv2.drawContours(m, [c], -1, 255, 2)          # thin structures (open lines) have ~zero area: keep their stroke
        mm = m > 0
        if s[mm].mean() - bg[mm].mean() > 35: out |= m
    return out > 0


def boundary_err(det, gt):
    """Symmetric mean boundary distance (pixels) between mask boundaries (Chamfer / mean surface distance)."""
    def bd(m): return cv2.morphologyEx(m.astype(np.uint8), cv2.MORPH_GRADIENT, np.ones((3, 3), np.uint8)) > 0
    bd_d, bd_g = bd(det), bd(gt)
    if bd_d.sum() == 0 or bd_g.sum() == 0: return float("nan")
    dt_g = cv2.distanceTransform((~bd_g).astype(np.uint8), cv2.DIST_L2, 3); dt_d = cv2.distanceTransform((~bd_d).astype(np.uint8),
cv2.DIST_L2, 3)
    return float(0.5 * (dt_g[bd_d].mean() + dt_d[bd_g].mean()))


METH = {"A Region growing": region_growing, "B Split & merge": split_merge, "C Edge linking": edge_link}
rows = []
for g in G:
    gt = g["bright"] > 0
    for n, f in METH.items():
        o, ms = timeit(f, g["img"], reps=1)
        rows.append(dict(img=g["id"], level=g["level"], method=n, iou=iou(o, gt), dice=dice(o, gt), bnd=boundary_err(o, gt), ms=ms,
frac=float(o.mean())))
R["t9"] = rows

# Sensitivity: tolerant IoU (1-px dilation of GT) because thin 1-2 px scratches are rarely pixel exact
k = np.ones((3, 3), np.uint8)
for r, g in zip(rows, [g for g in G for _ in METH]):
    pass

# component-wise recall per damage-type: scratches (long thin), dust (small round), tear (large) - using the synthetic ground truth
components
def component_recall(det, gt):
    n, lab, st, _ = cv2.connectedComponentsWithStats((gt > 0).astype(np.uint8), 8)
    rec = {"dust": [], "scratch/crease": [], "tear": []}
    for i in range(1, n):
        a = st[i, cv2.CC_STAT_AREA]; w_, h_ = st[i, cv2.CC_STAT_WIDTH], st[i, cv2.CC_STAT_HEIGHT]
        kind = "tear" if a > 1500 and min(w_, h_) > 30 else ("scratch/crease" if max(w_, h_) > 40 else "dust")
        cov = det[lab == i].mean(); rec[kind].append(cov > 0.5)
    return {k: (float(np.mean(v)) if v else float("nan")) for k, v in rec.items()}

comp = collections.defaultdict(lambda: collections.defaultdict(list))
for g in G:
    for n, f in METH.items():
        o = f(g["img"]); cr = component_recall(o, g["bright"])
        for kk, v in cr.items():
            if not np.isnan(v): comp[n][kk].append(v)
R["t9_component_recall"] = {n: {k: float(np.mean(v)) for k, v in d.items()} for n, d in comp.items()}

# Q60 wrong seed in region growing: low-contrast stain
rng = np.random.default_rng(7)
st = np.full((256, 256), 128, np.float32)
gtm = np.zeros((256, 256), np.uint8); cv2.ellipse(gtm, (128, 128), (50, 35), 20, 0, 360, 255, -1)
st[gtm > 0] = 112
st = cv2.GaussianBlur(st, (0, 0), 1.5) + rng.normal(0, 5, st.shape)
# gentle illumination gradient (natural tone) so that the 'background' range is wide
st += np.linspace(-12, 12, 256)[None, :]
st = np.clip(st, 0, 255).astype(np.uint8); sm = cv2.GaussianBlur(st, (0, 0), 1.0)
q60 = []
for seed_name, seed in (("inside stain (correct)", (128, 128)), ("just outside stain", (128, 210)), ("on stain boundary", (178,
128))):
```

```
    for Tg in (5, 10, 15, 20, 30):
        m = np.zeros((258, 258), np.uint8)
        cv2.floodFill(sm.copy(), m, seed, 255, (Tg,) * 3, (Tg,) * 3, 8 | cv2.FLOODFILL_MASK_ONLY | cv2.FLOODFILL_FIXED_RANGE | (255 <<
8))
        reg = m[1:-1, 1:-1] > 0
        q60.append(dict(seed=seed_name, Tg=Tg, area_px=int(reg.sum()), area_over_stain=float(reg.sum() / (gtm > 0).sum()),
iou=iou(reg, gtm > 0)))
R["q60"] = q60
fig, ax = plt.subplots(1, 4, figsize=(15, 3.8))
ax[0].imshow(st, cmap="gray", vmin=60, vmax=180); ax[0].set_title("Low-contrast stain (tone close to surround)"); ax[0].axis("off")
for x, (nm, seed, Tg) in zip(ax[1:], (("Seed inside, Tg=10", (128, 128), 10), ("Seed inside, Tg=20", (128, 128), 20), ("Seed outside,
Tg=10", (128, 210), 10))):
    m = np.zeros((258, 258), np.uint8); cv2.floodFill(sm.copy(), m, seed, 255, (Tg,) * 3, (Tg,) * 3, 8 | cv2.FLOODFILL_MASK_ONLY |
cv2.FLOODFILL_FIXED_RANGE | (255 << 8))
    x.imshow(m[1:-1, 1:-1], cmap="gray"); x.contour(gtm > 0, colors="r", linewidths=0.8); x.set_title(nm + " (red = true stain)");
x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t9_wrong_seed.png", dpi=75); plt.close()

# Q61 edge linking with gaps / overlapping lines
q61 = []
for gap in (0, 3, 6, 10, 16):
    t = np.full((200, 200), 120, np.uint8)
    pts = [((40, 40), (160, 40)), ((160, 40), (160, 160)), ((160, 160), (40, 160)), ((40, 160), (40, 40))]
    for (p1, p2) in pts: cv2.line(t, p1, p2, 235, 2)
    for (p1, p2) in pts:                                  # open a gap in the middle of every side
        mid = ((p1[0] + p2[0]) // 2, (p1[1] + p2[1]) // 2); cv2.circle(t, mid, gap // 2 + 1, 120, -1) if gap else None
    t = np.clip(t.astype(np.float32) + np.random.default_rng(1).normal(0, 4, t.shape), 0, 255).astype(np.uint8)
    gt_fill = np.zeros_like(t); cv2.rectangle(gt_fill, (40, 40), (160, 160), 255, -1)
    for close in (3, 7, 13, 21):
        e = cv2.Canny(cv2.GaussianBlur(t, (0, 0), 1), 50, 110)
        e = cv2.morphologyEx(e, cv2.MORPH_CLOSE, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (close, close)))
        cnts, _ = cv2.findContours(e, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
        m = np.zeros_like(t)
        for c in cnts: cv2.drawContours(m, [c], -1, 255, -1)
        q61.append(dict(gap=gap, close=close, iou_filled_region=iou(m > 0, gt_fill > 0)))
R["q61"] = q61

# figures
g = G[4]
panels = [("Degraded (G05)", g["img"]), ("GT bright damage", g["bright"])] + [(n, (f(g["img"]) * 255).astype(np.uint8)) for n, f in
METH.items()]
fig, ax = plt.subplots(1, 5, figsize=(17, 3.9))
for x, (n, o) in zip(ax, panels): x.imshow(o, cmap="gray"); x.set_title(n, fontsize=9); x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t9_seg.png", dpi=70); plt.close()

json.dump(R, open("/home/claude/work/results_t9.json", "w"), indent=1, default=float)
agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows:
    for k in ("iou", "dice", "bnd", "ms", "frac"): agg[r["method"]][k].append(r[k])
print("T9 mean over 9 photos")
for k, v in agg.items(): print("  ", k, {a: round(float(np.nanmean(b)), 3) for a, b in v.items()})
print("component recall", R["t9_component_recall"])
print("q60"); [print("  ", r) for r in q60 if r["Tg"] in (10, 20, 30)]
print("q61"); [print("  ", r) for r in q61 if r["close"] in (3, 13, 21)]
```

task910.py

```
"""Pipeline evaluation (Task 9 Q57-59) and Task 10 colour / high-resolution strategies."""
import sys, json, collections
sys.path.insert(0, "/home/claude/work")
from common import *
from pipeline import *
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr, structural_similarity as ssim

R = {}
G = build_gray_set()

# ---------- gray pipeline quality on 9 photos (512x512) ----------
rows = []
for g in G:
    x, out, dmg = restore_gray(g["img"])
    rows.append(dict(img=g["id"], level=g["level"],
                     psnr_in=float(psnr(g["clean"], g["img"], data_range=255)), ssim_in=float(ssim(g["clean"], g["img"],
data_range=255)),
                     psnr_enh=float(psnr(g["clean"], x, data_range=255)), ssim_enh=float(ssim(g["clean"], x, data_range=255)),
                     psnr_out=float(psnr(g["clean"], out, data_range=255)), ssim_out=float(ssim(g["clean"], out, data_range=255)),
                     dice=dice(dmg, g["bright"] > 0), iou=iou(dmg, g["bright"] > 0)))
R["gray_pipe"] = rows
```

```
# ---------- stage-wise timing at A4 (2480x3508), best of 3 -------------
rng = np.random.default_rng(0)
a4 = cv2.resize(G[3]["img"], (2480, 3508), interpolation=cv2.INTER_LINEAR)
a4 = np.clip(a4.astype(np.float32) + rng.normal(0, 6, a4.shape), 0, 255).astype(np.uint8)
best = None
for rep in range(3):
    T = {}; t = time.perf_counter(); restore_gray(a4, T); tot = (time.perf_counter() - t) * 1000
    if best is None or tot < best[0]: best = (tot, dict(T))
R["a4_timing_ms"] = dict(best[1]); R["a4_total_ms"] = best[0]
# the same without optional inpainting (enhancement only)
best2 = 1e9
for rep in range(3):
    t = time.perf_counter(); restore_gray(a4, None, inpaint=False); best2 = min(best2, (time.perf_counter() - t) * 1000)
R["a4_total_no_inpaint_ms"] = best2

# ---------- frequency-domain ops timing at A4 (why FFT filtering is not in the pipeline) ----------
t = time.perf_counter(); F = np.fft.rfft2(a4.astype(np.float32), s=(3508, 2480)); _ = np.fft.irfft2(F, s=(3508, 2480));
R["a4_fft_nopad_ms"] = (time.perf_counter() - t) * 1000
t = time.perf_counter(); F = np.fft.fft2(a4.astype(np.float32), s=(2 * 3508, 2 * 2480)); _ = np.fft.ifft2(F); R["a4_fft_padded_ms"] =
(time.perf_counter() - t) * 1000

# ---------- Task 10 ----------
C = build_color_set()
STRAT = {"A Retain": A_retain, "B Modify": B_modify, "C Redesign": C_redesign}

def delta_e(a_bgr, b_bgr):
    la = cv2.cvtColor(a_bgr, cv2.COLOR_BGR2LAB).astype(np.float32); lb = cv2.cvtColor(b_bgr, cv2.COLOR_BGR2LAB).astype(np.float32)
    la[..., 0] *= 100 / 255; lb[..., 0] *= 100 / 255; la[..., 1:] -= 128; lb[..., 1:] -= 128
    return float(np.sqrt(((la - lb) ** 2).sum(-1)).mean())

def edge_ret(o, ref):
    g = lambda z: cv2.cvtColor(z, cv2.COLOR_BGR2GRAY).astype(np.float32)
    s = lambda z: np.sqrt(cv2.Sobel(z, cv2.CV_32F, 1, 0) ** 2 + cv2.Sobel(z, cv2.CV_32F, 0, 1) ** 2)[30:-30, 30:-30].mean()
    return float(s(g(o)) / s(g(ref)))

def cast_metric(o):
    lab = cv2.cvtColor(o, cv2.COLOR_BGR2LAB).astype(np.float32)
    return float(lab[..., 2].mean() - 128), float(lab[..., 1].mean() - 128)    # b* (yellow) and a* (red) offsets

def skin_hue_shift(o, ref):
    """Mean |hue difference| (degrees) on skin-tone-like pixels of the clean photo (hue 0-50 deg, moderate saturation)."""
    hr = cv2.cvtColor(ref, cv2.COLOR_BGR2HSV); ho = cv2.cvtColor(o, cv2.COLOR_BGR2HSV)
    m = (hr[..., 0] * 2 < 50) & (hr[..., 1] > 50) & (hr[..., 1] < 200) & (hr[..., 2] > 70)
    if m.sum() < 50: return float("nan")
    d = np.abs(hr[..., 0].astype(np.float32) - ho[..., 0].astype(np.float32)); d = np.minimum(d, 180 - d) * 2
    return float(d[m].mean())

rows10 = []
imgs_out = {}
for c in C:
    inp = c["img"]; ref = c["clean"]; gtm = c["mask"] > 0
    entries = [("Input (degraded)", (inp, None, 0.0))]
    for n, f in STRAT.items():
        tm = {}; o, d = f(inp, tm); entries.append((n, (o, d, tm["total_ms"])))
    for n, (o, d, ms) in entries:
        gtb = cv2.dilate(c["mask"], np.ones((1, 1), np.uint8)) > 0
        rows10.append(dict(img=c["id"], scene=c["name"], method=n, dice=(dice(d, gtm) if d is not None else float("nan")),
                           psnr=float(psnr(ref, o, data_range=255)), ssim=float(ssim(ref, o, channel_axis=2, data_range=255)),
                           dE=delta_e(o, ref), edge=edge_ret(o, ref), b_cast=cast_metric(o)[0], a_cast=cast_metric(o)[1],
                           skin=skin_hue_shift(o, ref), ms512=ms))
        if c["id"] in ("C02", "C05", "C08"): imgs_out[(c["id"], n)] = o
R["t10"] = rows10

# ---------- timing at 4000x3000 colour (best of 3) ----------
big = cv2.resize(C[1]["img"], (4000, 3000), interpolation=cv2.INTER_CUBIC)
tim = {}
for n, f in STRAT.items():
    best = 1e9
    for rep in range(3):
        tm = {}; f(big, tm); best = min(best, tm["total_ms"])
    tim[n] = best
R["t10_timing_4000x3000_ms"] = tim

# ---------- Q69: inpainting hole-size study (fails for large regions) ----------
src = clean_sources()["astronaut"]
q69 = []
for r in (2, 5, 10, 20, 35, 55):
    msk = np.zeros(src.shape[:2], np.uint8); cv2.circle(msk, (256, 150), r, 255, -1)   # hole over the face (hair/eye region)
    broken = src.copy(); broken[msk > 0] = 235
    rec = cv2.inpaint(broken, msk, 3, cv2.INPAINT_TELEA)
    sel = msk > 0
    q69.append(dict(radius=r, hole_px=int(sel.sum()), psnr_hole=float(10 * np.log10(255 ** 2 / ((rec[sel].astype(np.float32) -
src[sel].astype(np.float32)) ** 2).mean())),
```

```
                    mean_abs_err=float(np.abs(rec[sel].astype(np.float32) - src[sel].astype(np.float32)).mean())))
R["q69"] = q69
fig, ax = plt.subplots(1, 4, figsize=(14, 3.8))
for x, r in zip(ax, (5, 20, 35, 55)):
    msk = np.zeros(src.shape[:2], np.uint8); cv2.circle(msk, (256, 150), r, 255, -1); br = src.copy(); br[msk > 0] = 235
    x.imshow(cv2.cvtColor(cv2.inpaint(br, msk, 3, cv2.INPAINT_TELEA)[80:230, 190:340], cv2.COLOR_BGR2RGB)); x.set_title(f"Inpainted
hole radius {r}px"); x.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t10_inpaint_holes.png", dpi=70); plt.close()

# ---------- figures ----------
fig, ax = plt.subplots(3, 5, figsize=(17, 10.4))
for i, cid in enumerate(("C02", "C05", "C08")):
    c = [x for x in C if x["id"] == cid][0]
    for j, (n, im) in enumerate([("Degraded input", c["img"]), ("A Retain", imgs_out[(cid, "A Retain")]), ("B Modify", imgs_out[(cid,
"B Modify")]),
                                 ("C Redesign", imgs_out[(cid, "C Redesign")]), ("Clean reference", c["clean"])]):
        ax[i, j].imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB)); ax[i, j].set_title(f"{cid} {n}", fontsize=9); ax[i, j].axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t10_compare.png", dpi=62); plt.close()

g = G[3]; x, out, dmg = restore_gray(g["img"])
fig, ax = plt.subplots(1, 4, figsize=(15, 4))
for a_, (n, im) in zip(ax, [("Degraded (G04)", g["img"]), ("Enhanced (before inpaint)", x), ("Restored (+inpaint, labelled)", out),
("Original undamaged", g["clean"])]):
    a_.imshow(im, cmap="gray", vmin=0, vmax=255); a_.set_title(n); a_.axis("off")
plt.tight_layout(); plt.savefig(FIGS + "/t9_pipeline_result.png", dpi=70); plt.close()

# save restored colour/gray examples for the document
for cid in ("C02", "C05", "C08"):
    cv2.imwrite(f"{FIGS}/restored_{cid}.png", imgs_out[(cid, "B Modify")])
cv2.imwrite(FIGS + "/restored_G04.png", out)

json.dump(R, open("/home/claude/work/results_t910.json", "w"), indent=1, default=float)

print("GRAY PIPELINE (mean)")
for k in ("psnr_in", "ssim_in", "psnr_enh", "ssim_enh", "psnr_out", "ssim_out", "dice", "iou"): print("  ", k,
round(float(np.mean([r[k] for r in rows])), 3))
print("A4 total ms", round(R["a4_total_ms"]), "no inpaint", round(R["a4_total_no_inpaint_ms"]))
for k, v in R["a4_timing_ms"].items(): print("   ", k, round(v))
print("A4 FFT nopad", round(R["a4_fft_nopad_ms"]), "padded", round(R["a4_fft_padded_ms"]))
agg = collections.defaultdict(lambda: collections.defaultdict(list))
for r in rows10:
    for k in ("dice", "psnr", "ssim", "dE", "edge", "b_cast", "a_cast", "skin", "ms512"): agg[r["method"]][k].append(r[k])
print("TASK10 (mean over 15)")
for k, v in agg.items(): print("  ", k, {a: round(float(np.nanmean(b)), 3) for a, b in v.items()})
print("timing 4000x3000", {k: round(v) for k, v in tim.items()})
print("q69", [(r["radius"], round(r["psnr_hole"], 1), round(r["mean_abs_err"], 1)) for r in q69])
```

task10b.py

```
"""Task 10 follow-up: parameter study for the 'Modify' strategy (white-balance strength x luminance mode)."""
import sys, json, collections
sys.path.insert(0, "/home/claude/work")
from common import *
from pipeline import *
from pipeline import _white_balance_lab
from task910_helpers import *
from skimage.metrics import peak_signal_noise_ratio as psnr, structural_similarity as ssim

C = build_color_set()


def lum_stretch(L8, flat=0.7, lo=1, hi=99, usm=0.5):
    x = cv2.medianBlur(L8, 3).astype(np.float32)
    x = flatten_illum(x, 16, flat)
    a, b = np.percentile(x, [lo, hi]); x = np.clip((x - a) * 255.0 / max(b - a, 1), 0, 255)
    x = np.clip(x + usm * (x - cv2.GaussianBlur(x, (0, 0), 1.5)), 0, 255)
    return x.astype(np.uint8)


def modify_variant(bgr, wb, mode, work=1000):
    h, w = bgr.shape[:2]; s = min(1.0, work / max(h, w))
    small = cv2.resize(bgr, (int(w * s), int(h * s)), interpolation=cv2.INTER_AREA) if s < 1 else bgr
    lab = _white_balance_lab(small, wb) if wb > 0 else cv2.cvtColor(small, cv2.COLOR_BGR2LAB).astype(np.float32)
    L = np.clip(lab[..., 0], 0, 255).astype(np.uint8)
    Lr = restore_gray(L, None, inpaint=False)[0] if mode == "CLAHE" else lum_stretch(L)
    dmg = detect_damage(L)
    a = cv2.GaussianBlur(lab[..., 1], (0, 0), 1.5); b = cv2.GaussianBlur(lab[..., 2], (0, 0), 1.5)
    out_s = cv2.cvtColor(np.dstack([Lr.astype(np.float32), a, b]).clip(0, 255).astype(np.uint8), cv2.COLOR_LAB2BGR)
    n, cc, st, _ = cv2.connectedComponentsWithStats(dmg.astype(np.uint8), 8)
    ok = np.zeros(n, bool); ok[1:] = st[1:, cv2.CC_STAT_AREA] <= 2500
    out_s = cv2.inpaint(out_s, cv2.dilate(ok[cc].astype(np.uint8), np.ones((3, 3), np.uint8)), 3, cv2.INPAINT_TELEA)
```

```
    return (cv2.resize(out_s, (w, h), interpolation=cv2.INTER_CUBIC) if s < 1 else out_s), \
           (cv2.resize(dmg.astype(np.uint8), (w, h), interpolation=cv2.INTER_NEAREST) > 0 if s < 1 else dmg)


rows = []
for wb in (0.0, 0.5, 0.85):
    for mode in ("CLAHE", "Stretch"):
        vals = collections.defaultdict(list)
        for c in C:
            o, d = modify_variant(c["img"], wb, mode)
            vals["psnr"].append(psnr(c["clean"], o, data_range=255)); vals["ssim"].append(ssim(c["clean"], o, channel_axis=2,
data_range=255))
            vals["dE"].append(delta_e(o, c["clean"])); vals["b_cast"].append(cast_metric(o)[0]); vals["edge"].append(edge_ret(o,
c["clean"]))
            vals["skin"].append(skin_hue_shift(o, c["clean"]))
        rows.append(dict(wb=wb, mode=mode, **{k: float(np.nanmean(v)) for k, v in vals.items()}))
# reference: untouched input
vals = collections.defaultdict(list)
for c in C:
    vals["psnr"].append(psnr(c["clean"], c["img"], data_range=255)); vals["ssim"].append(ssim(c["clean"], c["img"], channel_axis=2,
data_range=255))
    vals["dE"].append(delta_e(c["img"], c["clean"])); vals["b_cast"].append(cast_metric(c["img"])[0]);
vals["edge"].append(edge_ret(c["img"], c["clean"])); vals["skin"].append(skin_hue_shift(c["img"], c["clean"]))
ref_in = {k: float(np.nanmean(v)) for k, v in vals.items()}
json.dump(dict(rows=rows, input=ref_in), open("/home/claude/work/results_t10b.json", "w"), indent=1, default=float)
print("input", {k: round(v, 3) for k, v in ref_in.items()})
for r in rows: print({k: (round(v, 3) if isinstance(v, float) else v) for k, v in r.items()})
```